



Curvance

Competition

April 16, 2025

Contents

1	Introduction	3
1.1	About Cantina	3
1.2	Disclaimer	3
1.3	Risk assessment	3
1.3.1	Severity Classification	3
2	Security Review Summary	4
3	Findings	5
3.1	High Risk	5
3.1.1	Incorrect Curve LP pricing for 2 pool for non-correlated asset	5
3.1.2	CVE token bridging is completely broken and can result in loss of funds if used	5
3.1.3	Fee transfers using Wormhole native bridge will be lost forever because PMH doesn't call <code>completeTransferWithPayload</code>	7
3.1.4	Fee transfers will be lost when using both CCTP and Wormhole native bridge because assumes USDC address same between chains	8
3.1.5	Creation of new locks when number of locks is 0 should update user's claim index to <code>currentEpoch(block.timestamp) + 1</code>	9
3.1.6	Hardcoding the maximise params of <code>gmxFReader.getMarketTokenPrice(/*...*/)</code> returns maximum price always	10
3.1.7	Due to Bug in <code>processExpiredLock()</code> in <code>VeCVE</code> , users could drain rewards from the <code>CVELocker</code>	11
3.1.8	There is no decimal normalization when transferring tokens cross-chain using the <code>TokenBridge.transferTokensWithPayload()</code>	14
3.1.9	<code>VertexMultiplierMax</code> is never set, leading to no interest above vertex	15
3.1.10	Anyone can call <code>getBorrowRateWithUpdate()</code> to manipulate vertex multiplier	16
3.1.11	OCVE option exercise price is squared, leading to overpayment	16
3.1.12	<code>VeCVE</code> unlock penalty formula is backwards, providing immediate free unlocks	17
3.1.13	Gauge deposits can be temporarily bricked or made insolvent due to unallocated rewards miscalculation	18
3.1.14	The <code>userNextClaimIndex</code> will not be increased when <code>userPoints</code> is 0 causing attacker can use expired lock to steal rewards	20
3.1.15	<code>chainPoints</code> is never decremented in normal lock expiry, leading to wasted CVE	22
3.1.16	<code>_statusOf()</code> overwrite the <code>maxDebt</code> value	23
3.1.17	Attacker will claim all past epochs	24
3.1.18	<code>DToken</code> transfers are illegitimately restricted if a lender is insolvent	26
3.1.19	Wrong underlying price is used when removing collateral	26
3.1.20	Early <code>veCVE</code> unlock penalty multiplier is not converted to WAD	28
3.1.21	<code>CVELocker.recordEpochRewards</code> is called twice on a recipient chain, preventing reward claiming and stealing rewards	28
3.1.22	Posted collateral tokens are transferred without adjusting the user's posted <code>cToken</code> amount	29
3.1.23	Collateral tokens that have not been posted in the market may not be able to be transferred due to the liquidity check	30
3.1.24	Wrong decimal scalation will result in severe miscalculation of amounts	31
3.1.25	Missing <code>RateProvider</code> normalization ensures inaccurate pricing of <code>BalancerStablePool</code>	32
3.1.26	The CCTP receiver cannot call <code>MessageTransmitter.receiveMessage</code> causing can not to receive the USDC from the remote chain	34
3.1.27	Pending yield not cleared after claiming	35
3.1.28	Improper Setting of <code>rewardsPerCVE</code> in <code>ProtocolMessagingHub::receiveWormholeMessages</code>	35
3.2	Medium Risk	37
3.2.1	Cannot swap and lock rewards into <code>veCVE</code> due to <code>nonReentrant</code> modifier	37
3.2.2	Possible for \$800 USDC precision loss in <code>FeeAccumulator</code>	39
3.2.3	<code>Curve2Pool</code> asset pricing is manipulatable, even with reentrancy checks	39
3.2.4	No slippage control for <code>harvest</code> function (when compounding back to the LP) in <code>Aero-dromeStableCToken.sol</code> & volatile	40
3.2.5	Excess ETH when <code>BorrowZapper.sol#borrowAndBridge()</code> function is called not refunded	42
3.2.6	GMX callback contract does not handle deposit cancelation	43

3.2.7	Malicious users, when given allowance, might drain funds from owners	44
3.2.8	Incorrect interface definition makes contract undeployable	45
3.2.9	CTokenCompoundingWithExitFee.sol is not EIP-4626 compliant	46
3.2.10	Deleveraging position on CTokenCompoundingWithExitFee would always revert	47
3.2.11	If a reward token is re-added to GaugePool, a user could drain the contract of it	53
3.2.12	Unsold CVE within CurvanceDAOLBP will remain stuck	53
3.2.13	Convex Booster functions will revert on Arbitrum because Booster functions on L2 (Arbitrum) have a different interface	54
3.2.14	The protocol allows borrowing with small positions in which can result bad debts to protocol	54
3.2.15	If protocol leverage fee is set, all calls to deleverage() will revert	56
3.2.16	All calls to leverage() will fail if user has sufficient preexisting debt	57
3.2.17	Miscalculation in claimYieldForGauge() leads to yield manager insolvency	59
3.2.18	All claimed gas sent to the DAO if cToken has no pending WETH yield	60
3.2.19	Funds will be left in Zapper when performing redeems	61
3.2.20	Low value tokens can be reward rate stored incorrectly, losing user yield	61
3.2.21	Fee amount sent to one balance manager is more than expected in multiswap() and executeOTC()	62
3.2.22	In Convex2PoolCToken.sol and Convex3PoolCToken.sol and Aura CToken, the deposit should not able done when the pool is shut down	63
3.2.23	getPricesForAsset() will fail for all MTokens	63
3.2.24	User closes and opens a veCVE lock in the same epoch may result in a loss of points .	64
3.2.25	VertexInterestRate is derived incorrectly from vertexRatePerYear	66
3.2.26	Cross chain emissions rate updates can fail due to insufficient gas	67
3.2.27	getLower can return higher value due to ETHUSD conversion assumptions	68
3.2.28	Insufficient L2 uptime checks	70
3.2.29	Users will lose some funds when unfolding positions with deleverage()	70
3.2.30	deleverage() only allows one swap, which isn't sufficient for the use case	71
3.2.31	Reward distribution fails for all chains if a single chain would receive zero fees	71
3.2.32	Harvested Convex rewards cannot be swapped to native tokens due to missing re- ceive function	72
3.2.33	CToken yield stops compounding if the reward rate is set to zero	73
3.2.34	If the DAO deposits dToken reserves, totalReserves will be miscalculated resulting in an incorrect exchange and borrow rate	74
3.2.35	Balancer oracle should not be used while in recovery mode	74
3.2.36	Malicious Liquidity Providers, could artificially spike Interest rates, to steal value from Borrowes	77
3.2.37	GMCToken incorrectly attempts to claim funding fees from GMX	78
3.2.38	The averagePrice algorithm will report inaccurate prices in the event of a depeg . . .	78
3.2.39	FeeAccumulator.multiSwap may transfer too much CompoundFee	79
3.2.40	FeeAccumulator.executeEpochFeeRouter repeats payment of CompoundFee	80
3.2.41	Liquidators are not incentivized to socialize bad debt since liquidate is always cheaper than liquidateAccount	81
3.2.42	FeeAccumulator.migrateFeeAccumulator did not transfer native token	91
3.2.43	Convex2PoolCToken pool info's struct implementaion is completely broken on Arbi- trum/Polygon	92
3.2.44	CVE bridge will be replayed when chain hard fork	93
3.2.45	Utilization ratio Can Be Manipulated Above 100%	93
3.2.46	DTOKEN debt can't be repaid/liquidated all due to overflow	95
3.2.47	DTOKENS cannot be minted until gaugePool has started	98
3.2.48	epochReported is never set in BlastNativeYieldManager	100
3.2.49	Users voting power is zero in VeCVE, causing offchain voting failure	101
3.2.50	supportsInterface fails to return all supported interfaces in BlastCentralRegistry .	102
3.2.51	Rewards will be lost when calling setRewardPerSec() more than once for a given epoch	102
3.2.52	CVX rewards are not accounted for	103

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A competition provides a broad evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While competitions endeavor to identify and disclose all potential security issues, they cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities, therefore, any changes made to the code would require an additional security review. Please be advised that competitions are not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	<i>Must fix as soon as possible (if already deployed).</i>
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Curvance is a modular protocol for optimized liquidity management. It allows to collateralize, leverage, and earn boosted rewards across DeFi.

From Feb 27th to Apr 15th Cantina hosted a competition based on [curvance](#). The participants identified a total of **189** issues in the following risk categories:

- Critical Risk: 0
- High Risk: 28
- Medium Risk: 52
- Low Risk: 73
- Gas Optimizations: 0
- Informational: 36

The present report only outlines the **critical**, **high** and **medium** risk issues.

3 Findings

3.1 High Risk

3.1.1 Incorrect Curve LP pricing for 2 pool for non-correlated asset

Submitted by n1punp, also found by Bauchibred

Severity: High Risk

Context: [Curve2PoolLPAdaptor.sol#L175-L180](#)

Description: This price calculation is incorrect. Line179 should be `price = (price * price1) / WAD;` to correctly estimate the LP token price when the underlying tokens are not correlated.

Take the Curve 2 pool GEAR-ETH as an example. At the time of writing, the pool's `virtual_price` is 1026693802827566901 and GEAR price is \$0.0207 and ETH price is \$3211. So `price0` is the price of GEAR (0.0207e18) and `price1` is the price of ETH (3211e18).

Hence the variable `price` in Line 176 is then $2 * 1026693802827566901 * \sqrt{0.0207e18} / \sqrt{3211e18} = 0.005213578970463e18$. In Line 179, the `price` variable gets overwritten to $0.005213578970463e18 * 0.0207e18 / 1e18 = 0.0001079210847e18$.

However, if we evaluate the LP price based on the underlying tokens $\rightarrow$ we get all LP = $[(82.5e6 \text{ GEAR}) * (\$0.0207e18/\text{GEAR}) + (514.8 \text{ ETH}) * (\$3211e18/\text{ETH})] = 3.36e24$ So, 1 LP = $3.36e24 / 200773910327800888070125 = \16.73 . (needs to multiplied by 1e18 for precision control).

We can see that the 2 calculations do not align. However, if we were to fix Line179 to `price = (price * price1) / WAD` then we will get: `price = 0.005213578970463e18 * 3211e18 / 1e18 = 16.74080207415669e19`, which is close to the obtained spot calculation above.

With incorrect LP pricing, this can lead to critical issues in the system. More precisely, if `price0 > price1`, then the LP is overestimated, and can lead to potential bad debt in the system. if `price1 > price0`, then the LP is underestimated, and the users can get falsely liquidated.

Recommendation: Change L179 to `price = (price * price1) / WAD;`.

3.1.2 CVE token bridging is completely broken and can result in loss of funds if used

Submitted by Haxatron, also found by yttriumzz

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: CVE token bridging is currently broken. Let us see how the CVE bridge process works.

First, user calls `bridge` which calls `bridgeCVE` with the recipient.

- [CVE.sol#L229-L244](#):

```
function bridge(
    uint256 dstChainId,
    address recipient,
    uint256 amount
) external payable returns (uint64) {
    address messagingHub = centralRegistry.protocolMessagingHub();
    _burn(msg.sender, amount);
    _mint(messagingHub, amount);

    return
        IProtocolMessagingHub(messagingHub).bridgeCVE( value: msg.value )(
            dstChainId,
            recipient,
            amount
        );
}
```

Which calls `bridgeCVE` on the message hub.

- [ProtocolMessagingHub.sol#L349-L367](#):

```

function bridgeCVE(
    uint256 dstChainId,
    address recipient,
    uint256 amount
) external payable returns (uint64) {
    if (msg.sender != address(cve)) {
        _revert(_UNAUTHORIZED_SELECTOR);
    }

    _checkMessagingHubStatus();

    return _transferTokenViaWormhole(
        address(cve),
        dstChainId,
        recipient,
        amount,
        msg.value
    );
}

```

And that calls `_transferTokenViaWormhole` in `FeeTokenBridgingHub.sol`. Here, notice that the recipient passed as the initial parameter to `bridge()` function is used as the `to` address.

- `FeeTokenBridgingHub.sol#L179-L218`

```

function _transferTokenViaWormhole(
    address token,
    uint256 dstChainId,
    address to,
    uint256 amount,
    uint256 wormholeFee
) internal returns (uint64) {
    ITokenBridge tokenBridge = centralRegistry.tokenBridge();
    uint16 wormholeChainId = centralRegistry.wormholeChainId(dstChainId);
    IWormhole wormholeCore = centralRegistry.wormholeCore();
    uint256 messageFee = wormholeCore.messageFee();

    SwapperLib._approveTokenIfNeeded(token, address(tokenBridge), amount);

    bytes memory payload = abi.encode(uint8(1), feeToken, amount);

    uint64 sequence = tokenBridge.transferTokensWithPayload({
        value: messageFee
    })(
        token,
        amount,
        wormholeChainId,
        bytes32(uint256(uint160(to))),
        0,
        payload
    );

    IWormholeRelayer.VaaKey[]
        memory vaaKeys = new IWormholeRelayer.VaaKey[](1);
    vaaKeys[0] = IWormholeRelayer.VaaKey({
        emitterAddress: bytes32(uint256(uint160(address(tokenBridge)))),
        chainId: wormholeCore.chainId(),
        sequence: sequence
    });

    return
        centralRegistry.wormholeRelayer().sendVaasToEvm({
            value: wormholeFee - messageFee
        })(wormholeChainId, to, payload, 0, _GAS_LIMIT, vaaKeys);
}

```

But according to the Wormhole docs for the `sendVaasToEvm` function, the `to` address is supposed to be the address of the recipient bridge in the target chain and it must implement the `receiveWormholeMessages` function (see [the wormhole documentation](#)).

That means in order to bridge the tokens, the user must implement their own bridge and specify that as the recipient address. And even if that is true there is no way to specify the destination address receiving the tokens in the destination chain in the `payload` specified in `sendVaasToEvm`. That can't be right.

If this function is used with the recipient in its current state, then this cross-chain transaction is lost forever because the recipient definitely won't be a bridge.

What I think occurred is that the developer confused `_transferTokenViaWormhole` as a function that can bridge tokens, but in reality it isn't, It is actually meant to submit USDC fees to the destination chain.

Recommendation: Please see [the wormhole documentation](#) on how to bridge tokens.

3.1.3 Fee transfers using Wormhole native bridge will be lost forever because PMH doesn't call `completeTransferWithPayload`

Submitted by *Haxatron*, also found by *cryptostaker*, *Bernd Artmüller* and *Sujith Somraaj*

Severity: High Risk

Context: (No context files were provided by the reviewer)

According to the [README](#):

The primary distribution of USDC for CVE Locker rewards will be via Circle's CCTP via Wormhole's automatic relayer. The alternative of using Wormhole's native bridge is may be used but will be implemented on a case-by-case basis for each chain.

Therefore, if Wormhole's native token bridge is used, these USDC fee transfers will be lost forever.

The problem is that when the message is received on the destination messaging hub there is no logic to withdraw from Wormhole native bridge.

- [ProtocolMessagingHub.sol#L148-L185](#):

```
if (payloadId == 1) {
    (, address token, uint256 amount) = abi.decode(
        payload,
        (uint8, address, uint256)
    );

    address feeToken = centralRegistry.feeToken();

    // Make sure the token received is the
    // fee token (locker reward token), otherwise do not execute
    // epoch finalization.
    if (token == feeToken) {
        ICVELocker locker = ICVELocker(centralRegistry.cveLocker());

        // If the locker is shutdown, transfer fees to DAO
        // instead of recording epoch rewards.
        if (locker.isShutdown() == 2) {
            SafeTransferLib.safeTransfer(
                feeToken,
                centralRegistry.daoAddress(),
                amount
            );
            return;
        }

        // Transfer fees to locker and record newest epoch rewards.
        SafeTransferLib.safeTransfer(
            feeToken,
            address(locker),
            amount
        );
        locker.recordEpochRewards(amount);
        return;
    }

    // PayloadID = 4 Indicates receiving some crosschain information from
    // a remote chain. Such as Gauge emissions configuration,
    // locked token data, locker rewards data.
}
```

In particular, when `receiveWormholeMessages` is called, the tokens HAVE NOT been transferred yet, what must be done is that `completeTransferWithPayload` must be called on the destination chain token bridge, so that the tokens can be sent to the messaging hub.

Therefore this cross-chain transaction which uses Wormhole native bridge will fail because the USDC fee tokens have not been withdrawn to the messaging hub and therefore these fee tokens will be lost forever.

Note: Issue also applies to the `OneBalanceFeeManager`.

Recommendation: Please see the [wormhole documentation](#) for more details.

3.1.4 Fee transfers will be lost when using both CCTP and Wormhole native bridge because assumes USDC address same between chains

Submitted by Haxatron

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: Fee transfers using both CCTP and Wormhole native bridge will be lost forever because it assumes USDC address will be the same between chains.

Firstly, when sending fees it encodes the `feeToken` into the payload for both CCTP and Token bridge.

- [FeeTokenBridgingHub.sol#L123-L218](#)

```
function _transferFeeTokenViaCCTP(
    ITokenMessenger circleTokenMessenger,
    uint256 dstChainId,
    address to,
    uint256 amount,
    uint256 wormholeFee
) internal {
    // ...
    wormholeRelayer.sendToEvm{ value: wormholeFee }(
        wormholeChainId,
        to,
        abi.encode(uint8(1), feeToken, amount), // @audit - this is the payload sent
        0,
        0,
        _GAS_LIMIT,
        wormholeChainId,
        address(0),
        defaultDeliveryProvider,
        messageKeys,
        15
    );
}

function _transferTokenViaWormhole(
    address token,
    uint256 dstChainId,
    address to,
    uint256 amount,
    uint256 wormholeFee
) internal returns (uint64) {
    // ...
    bytes memory payload = abi.encode(uint8(1), feeToken, amount);
    // ...
    return
        centralRegistry.wormholeRelayer().sendVaasToEvm{
            value: wormholeFee - messageFee
        }(wormholeChainId, to, payload, 0, _GAS_LIMIT, vaaKeys);
}
```

Here the `feeToken` represents address of USDC in the source chain.

But in the destination chain message hub it skips sending the fees if the `token` stored in the payload does not match the `feeToken` in the destination chain in the `if (token == feeToken)` check, which will always be the case because USDC contract address is different between each chain, the transfers will be skipped entirely.

- [ProtocolMessagingHub.sol#L148-L181](#):

```

if (payloadId == 1) {
    (, address token, uint256 amount) = abi.decode(
        payload,
        (uint8, address, uint256)
    );

    address feeToken = centralRegistry.feeToken();

    // Make sure the token received is the
    // fee token (locker reward token), otherwise do not execute
    // epoch finalization.
    if (token == feeToken) { // <=
        ICVELocker locker = ICVELocker(centralRegistry.cveLocker());

        // If the locker is shutdown, transfer fees to DAO
        // instead of recording epoch rewards.
        if (locker.isShutdown() == 2) {
            SafeTransferLib.safeTransfer(
                feeToken,
                centralRegistry.daoAddress(),
                amount
            );
            return;
        }

        // Transfer fees to locker and record newest epoch rewards.
        SafeTransferLib.safeTransfer(
            feeToken,
            address(locker),
            amount
        );
        locker.recordEpochRewards(amount);
        return;
    }
}

```

Note: Issue also applies to the `OneBalanceFeeManager`.

Recommendation: Remove the `if (token == feeToken)` check.

3.1.5 Creation of new locks when number of locks is 0 should update user's claim index to `currentEpoch(block.timestamp) + 1`

Submitted by [Haxatron](#), also found by [KupiaSec](#)

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: When a user has 0 locks and creates a new lock, `updateUserClaimIndex` updates a `userNextClaimIndex[user]` to the `currentEpoch(block.timestamp)`

- [VeCVE.sol#L1150-L1155](#)

```

if (userLocks[recipient].length == 0) {
    cveLocker.updateUserClaimIndex(
        recipient,
        currentEpoch(block.timestamp)
    );
}

```

- [CVELocker.sol#L278-L281](#)

```

function updateUserClaimIndex(address user, uint256 index) external {
    _checkIsVeCVE();
    userNextClaimIndex[user] = index;
}

```

This is problematic because it means user can claim rewards for the epoch they locked in because `nextEpochToDeliver = currentEpoch(block.timestamp) + 1`

- [CVELocker.sol#L376-L387](#)

```
function epochsToClaim(address user) public view returns (uint256) {
    if (
        nextEpochToDeliver > userNextClaimIndex[user] &&
        veCVE.userPoints(user) > 0
    ) {
        unchecked {
            return nextEpochToDeliver - (userNextClaimIndex[user]); // @audit - this will be 1
        }
    }

    return 0;
}
```

Above we can see there will be 1 epoch for a user to claim once they joined and therefore as below they can claim the current epoch.

- CVELocker.sol#L463-L474

```
function _calculateRewards(
    address user,
    uint256 epochs
) internal returns (uint256) {
    uint256 startEpoch = userNextClaimIndex[user];
    uint256 rewards;

    for (uint256 i; i < epochs; ) {
        unchecked {
            rewards += _calculateRewardsForEpoch(user, startEpoch + i++);
        }
    }
}
```

Therefore, if the user claims the rewards from the current epoch immediately once they joined, then there won't be enough rewards to distribute to everybody because the user's points weren't factored in the accounting for the epoch they joined in.

This can be exploited by repeatedly bridging the veCVE locks which will reset the `userNextClaimIndex[user]` in other chains and keep stealing from other users.

Recommendation: In normal cases:

```
cveLocker.updateUserClaimIndex(
    recipient,
    currentEpoch(block.timestamp) + 1
);
```

But there is an edge case before the genesis epoch is delivered. In that case if `nextEpochToDeliver == 0`,

```
cveLocker.updateUserClaimIndex(
    recipient,
    0
);
```

3.1.6 Hardcoding the `maximise` params of `gmxCReader.getMarketTokenPrice(/*...*/) returns maximum price always`

Submitted by [GeneralKay](#), also found by [tvdung94](#)

Severity: High Risk

Context: GMAdaptor.sol#L139

Description: The `maximize` params is used to determine if to return the maximum or minimum price from the `gmxCReader.getMarketTokenPrice(...)` function call. However `true` is hardcoded for the `maximize` params thereby causing it to always return the maximum price

Prices are supposed to favour the protocol by returning lower prices for deposit and higher prices for withdraw.

```

// @dev get the market token's price
// @param dataStore DataStore
// @param market the market to check
// @param longTokenPrice the price of the long token
// @param shortTokenPrice the price of the short token
// @param indexTokenPrice the price of the index token
// @param maximize whether to maximize or minimize the market token price
// @return returns (the market token's price, MarketPoolValueInfo.Props)
function getMarketTokenPrice(
    address dataStore,
    MarketProps memory market,
    PriceProps memory indexTokenPrice,
    PriceProps memory longTokenPrice,
    PriceProps memory shortTokenPrice,
    bytes32 pnlFactorType,
    bool maximize
) external view returns (int256, InfoProps memory);

```

Recommendation: Consider allowing the `maximize` params of `gmxCReader.getMarketTokenPrice(...)` be based on the operation in order to favour the protocol.

3.1.7 Due to Bug in `processExpiredLock()` in VeCVE, users could drain rewards from the CVELocker

Submitted by [alix40](#)

Severity: High Risk

Context: [VeCVE.sol#L666-L748](#)

Description: Due to a bug in the `processExpiredLock()` function, it is possible for any user either intentionally (attacker) or unintentionally (normal user behavior) to drain funds from the CVELocker contract. For the loss of funds to occur, a user must satisfy the following conditions:

1. He should have a single lock in the VeCVE contract.
2. He should use `processExpiredLock()` function to unlock his funds and withdraw them exactly at the **unlock epoch**.

If any users fulfills those 2 conditions a user will still have his yay points from the lock (they don't get updated in the contract), and when he creates a new lock in the VeCVE contract, he will get rewards for those points, from **epoch 0** (as `userClaimIndex` will get reseted after he has no locks) to the epoch of the unlock.

Example Scenario: (for the coded proof of concept for this scenario check the proof of concept section)

- We are at epoch 26, `nextEpochToDeliver` in CVELocker is 27.
- Alice creates lock for 100 CVEs.
- After 52 Weeks, Alice uses `processExpiredLock()` to close the position and get her CVEs back.
- Alice will get rewards for epoch 26 to 51, but due to the bug she will still have 100 yay points in the VeCVE contract.
- Alice waits some times and decides to create a new lock, when creating the lock the method `VeCVE._claimRewards()` will be called at the start of the function, Alice will **get rewards from epoch 0 to epoch 51**. (0 because `userClaimIndex` will get reseted to 0 when Alice has no locks in VeCVE left and 51 as in 52 is the unlock epoch of the first lock, and the yay points will be reseted here finally, and Alice will stop receiving Rewards as she has 0 yay from that point).

Curvance updates the yay points of user in the VeCVE contract by the CVELocker contract in the `cveLocker.claimRewardsFor()` method. For a Lock A that expires at epoch 27, Curvance stores the amount locked as unlocked in the unlock epoch in the `userUnlocksByEpoch` mapping. And when The CVELocker will try to handle calculating reward for the user in the unlock epoch, the yay points of the user will be reduced and updated, so the user wouldn't have points for the Tokens that are unlocked starting from their unlock epoch.

The bug happens if the user uses `processExpiredLock()` to unlock his fund exactly at the unlock epoch. At that period there are no rewards, that have been distributed in the CVELocker as the epoch still didn't finish so the unlock epoch wouldn't be actually handled in the `_calculateRewardsForEpoch()` method.

And because of this `veCVE.updateUserPoints(user, epoch);` wouldn't be called the user will still has his yay points, even though he has no locks left in the `VeCVE` contract.

```
function _calculateRewardsForEpoch(
    address user,
    uint256 epoch
) internal returns (uint256) {
    if (veCVE.userUnlocksByEpoch(user, epoch) > 0) {
        // If they have tokens unlocking this epoch we need to decrease
        // their tokenPoints.
        veCVE.updateUserPoints(user, epoch);
    }

    return (veCVE.userPoints(user) * epochRewardsPerCVE[epoch]);
}
```

In processExpiredLock() function in the VeCVE contract, if after closing the lock the user has no locks left, the userNextClaimIndex in CVELocker will be reset to zero using the resetUserClaimIndex() function:

```
function resetUserClaimIndex(address user) external {
    _checkIsVeCVE();
    delete userNextClaimIndex[user];
}
```

And because of those 2 conditions, the user will get rewards for the yay points left in the contract starting from Epoch 0 (As `userNextClaimIndex` get reseted) to the unlock epoch (as only then will his yay points get correctly updated with `veCvE.updateUserPoints(user, epoch);` function)

Proof of concept: The coded proof of concept showcases the same example shown in the impact section. See the result:

```
Running 1 test for tests/token/VeCVE/functions/CreateLock.t.sol:CreateLockTest
[PASS] test_createLock_PoX() (gas: 2400516)
Logs:
  Initialize to epoch 26, so more funds are available to steal(like the example)
  NEXT Epoch to deliver: 26
  -----Alice Create Lock-----
  User points 10000000000000000000
  -----WARP 52 Weeks To Unlock Time-----
  NEXT Epoch to deliver: 52
  ----- Remove Lock with processExpiredLock() -----
  User points after removal and already geting rewards for locking 10000000000000000000
  USDC BALANCE After Removal and before Attack: 26000000000000000000
  Number of user locks in VeCVE after removal: 0
  ----- Create new normal Lock and steal Rewards-----
  USDC BALANCE After Creating Lock: 78000000000000000000

Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 935.07ms
Ran 1 test suites: 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

To run the PoC please add the following test to `tests/token/VeCVE/functions/CreateLock.t.sol`:

```
function test_createLock_PoX() public {
    uint256 amount = 100e18;
    rewardsData = RewardsData(false, false, false, false);
    deal(address(cve), address(this), amount);
    cve.approve(address(veCVE), amount);

    deal(_USDC_ADDRESS, address(cveLocker), 1000 * amount);
    console.log(
        "Initialize to epoch 26, so more funds are available to steal(like the example)"
    );
    for (uint256 i = 0; i < 26; i++) {
        vm.prank(centralRegistry.feeAccumulator());
        cveLocker.recordEpochRewards(_ONE);
        vm.warp(block.timestamp + 2 weeks);
    }
    console.log("NEXT Epoch to deliver:", cveLocker.nextEpochToDeliver());

    console.log("-----Alice Create Lock-----");
    veCVE.createLock(amount, false, rewardsData, "", 0);
    console.log("User points", veCVE.userPoints(address(this)));
    console.log(
        "-----WARP 52 Weeks To Unlock Time-----"
    );
}
```

```

);
for (uint256 i = 0; i < 26; i++) {
    vm.prank(centralRegistry.feeAccumulator());
    cveLocker.recordEpochRewards(_ONE);
    vm.warp(block.timestamp + 2 weeks);
}
console.log("NEXT Epoch to deliver:", cveLocker.nextEpochToDeliver());

console.log(
    "----- Remove Lock with processExpiredLock() -----"
);
veCVE.processExpiredLock(0, false, false, rewardsData, "", 0);
(uint256[] memory a, uint256[] memory timestamps) = veCVE
    .queryUserLocks(address(this));

console.log(
    "User points after removal and already getting rewards for locking",
    veCVE.userPoints(address(this))
);
console.log(
    "USDC BALANCE After Removal and before Attack: ",
    usdc.balanceOf(address(this))
);
console.log(
    "Number of user locks in VeCVE after removal: ",
    timestamps.length
);

console.log(
    "----- Create new normal Lock and steal Rewards-----"
);
// @audit-info create normal lock with 52 weeks lock duration
cve.approve(address(veCVE), amount);
veCVE.createLock(amount, false, rewardsData, "", 0);
console.log(
    "USDC BALANCE After Creating Lock: ",
    usdc.balanceOf(address(this))
);
}

```

Recommendation: As a possible mitigation, consider updating the unlocks and userpoints if the user calls in the same epoch as the unlock epoch.

```

function processExpiredLock(
    uint256 lockIndex,
    bool relock,
    bool continuousLock,
    RewardsData calldata rewardsData,
    bytes calldata params,
    uint256 aux
) external nonReentrant {
    // Claim any pending locker rewards.
    _claimRewards(msg.sender, rewardsData, params, aux);

    // Need to cache after _claimRewards as the user could have
    // created or modifier their locks with their pending rewards.
    Lock[] storage locks = userLocks[msg.sender];

    // Length is index + 1 so has to be less than array length.
    if (lockIndex >= locks.length) {
        _revert(_INVALID_LOCK_SELECTOR);
    }

    if (block.timestamp < locks[lockIndex].unlockTime && isShutdown != 2) {
        _revert(_INVALID_LOCK_SELECTOR);
    }

    ++ if (currentEpoch(locks[lockIndex].unlockTime) == currentEpoch(block.timestamp)){
    ++     _reducePoints(msg.sender, locks[lockIndex].amount);
    ++     _reduceTokenUnlocks(msg.sender, currentEpoch(locks[lockIndex].unlockTime),
    ↵ locks[lockIndex].amount);
    ++     }
}

```

3.1.8 There is no decimal normalization when transferring tokens cross-chain using the `TokenBridge.transferTokensWithPayload()`

Submitted by [cryptotaker](#)

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: There is no decimal normalization when transferring tokens cross-chain using the `TokenBridge.transferTokensWithPayload()` in the `FeeTokenBridgingHub.sol`. Wormhole only takes in 8 decimal places, so if the token being bridged has 18 decimals on the destination chain, the decimals have to be normalized to 18.

The [Wormhole Documentation](#) warns:

Due to constraints on some supported chains, all token amounts passed through the token bridge are truncated to a maximum of 8 decimals. Any chains implementation must make sure that of any token only ever `MaxUint64` units (post-shifting) are bridged into the wormhole network at any given time (all target chains combined), even though the slot is 32 bytes long (theoretically fitting `uint256`).

They give a few examples:

The amount "1" of a 18 decimal Ethereum token is originally represented as: 1000000000000000000, over the wormhole it is passed as: 100000000.

The amount "2" of a 4 decimal token is represented as 20000 and is passed over the wormhole without a decimal shift.

And the ways to handle them:

In case the chain supports the original decimal amount (known from the `AssetMeta`) it can do a decimal shift back to the original decimal amount. This allows for out-of-the-box interoperability of DeFi protocols across for example different EVM environments.

Otherwise the wrapped token should stick to the 8 decimals that the protocol uses.

Proof of concept: The protocol interacts with `tokenBridge.transferTokensWithPayload()` and `sendVaasToEvm()` to bridge tokens in `FeeTokenBridgingHub.sol`:

```
uint64 sequence = tokenBridge.transferTokensWithPayload{ // <=
    value: messageFee
}({
    token,
    amount,
    wormholeChainId,
    bytes32(uint256(uint160(to))),
    0,
    payload
});

IWormholeRelayer.VaaKey[]
memory vaaKeys = new IWormholeRelayer.VaaKey[](1);
vaaKeys[0] = IWormholeRelayer.VaaKey({
    emitterAddress: bytes32(uint256(uint160(address(tokenBridge)))),
    chainId: wormholeCore.chainId(),
    sequence: sequence
});

return
    centralRegistry.wormholeRelayer().sendVaasToEvm{ // <=
        value: wormholeFee - messageFee
    }(wormholeChainId, to, payload, 0, _GAS_LIMIT, vaaKeys);
```

On the receiving end through `receiveWormholeMessages()` in `ProtocolMessagingHub.sol`, the protocol did not normalize the decimals back to 18.

The protocol does not use the SDK version of Wormhole. This means that they are not using the `TokenSender` and `TokenReceiver` abstract contracts from Wormhole, and thus have to implement all the checks by themselves.

In the [Wormhole example without SDK](#), note that the decimals is normalized.

```
uint256 decimals = getDecimals(wrappedTokenAddress);
uint256 powerOfTen = 0;
if(decimals > 8) powerOfTen = decimals - 8;
IERC20(wrappedTokenAddress).transfer(recipient, transfer.amount * 10 ** powerOfTen); //
```

Recommendation: Recommend inheriting TokenSender and TokenReceiver when doing cross-chain token transfer, as all the checks are already written by Wormhole:

- `TokenBase.sol` is the abstract contract of TokenSender and TokenReceiver.
- `HelloToken.sol` is an example of how the token bridging is written after importing the SDK and inheriting TokenSender and TokenReceiver.

Finally, note that `receivePayloadAndTokens()` can be overridden to include the transaction execution on the destination chain.

3.1.9 VertexMultiplierMax is never set, leading to no interest above vertex

Submitted by [Zach Obront](#), also found by [JCN](#) and [Bernd Artmüller](#)

Severity: High Risk

Context: `DynamicInterestRateModel.sol`#L619-L623

Description: When the config parameters for `DynamicInterestRateModel.sol` are set in `_updateDynamicInterestRateModel()`, the `vertexMultiplierMax` value is never set.

Because it can never be updated elsewhere in the code, it will always remain set to 0.

When the utilization is above the vertex, we call `_updateForAboveVertex()`. This function is intended to update the `vertexMultiplier` based on the model, and then set the value with a floor of $1e18$.

However, the floor is implemented before setting a ceiling of `vertexMultiplierMax`, which means that the value will always be set to 0:

```
if (newMultiplier < WAD) {
    return WAD;
}

return
    newMultiplier < config.vertexMultiplierMax
    ? newMultiplier
    : config.vertexMultiplierMax;
```

In this example, we can see that `newMultiplier` will be greater than `WAD` so will skip the first check. It'll then be greater than `config.vertexMultiplierMax`, so it will return that value, which is 0.

This result is used to save the multiplier in storage:

```
_currentRates = _packRatesData(
    _updateForAboveVertex(config, util),
    uint64(block.timestamp + config.adjustmentRate)
);
```

Any time the vertex interest rate is calculated, we multiply the numerator by this value:

```
function _getVertexInterestRate(
    uint256 util
) public view returns (uint256) {
    // We divide by 1e36 (WAD_SQUARED) since we need to divide by WAD
    // twice to maintain precision.
    return
        (util * ratesConfig.vertexInterestRate * vertexMultiplier()) /
        WAD_SQUARED;
}
```

The result is that `_getVertexInterestRate()` will return 0, and therefore we will only pay the interest rate up to the vertex, no matter how high the utilization gets.

Proof of concept: The following test can be dropped into `UpdateDynamicInterestRateModel.t.sol` to demonstrate the vulnerability:


```

function testZach_vertexMultMaxNotSetCausesZeroMultiplier() public {
    // call update with reasonable values, including 15_000e4 for vertexMultiplierMax
    interestRateModel.updateDynamicInterestRateModel({
        baseRatePerYear: 1500,
        vertexRatePerYear: 1500,
        vertexUtilStart: 5500,
        adjustmentRate: 10 hours,
        adjustmentVelocity: 5500,
        vertexMultiplierMax: 15_000e4,
        decayRate: 150,
        vertexReset: true
    });

    // confirm that the update call leaves vertexMultiplierMax at 0
    (, , , , uint256 vertexMultiplierMax, , , , ) = interestRateModel
        .ratesConfig();
    assertEq(vertexMultiplierMax, 0);

    // we start with vertexMultiplier set based on vertexReset = true
    assertEq(interestRateModel.vertexMultiplier(), 1e18);

    // simulate a 100% utilization rate, which should move vertexMultiplier up
    vm.warp(block.timestamp + 1 days);
    interestRateModel.getBorrowRateWithUpdate(0, 1e18, 0);

    // instead, it sets it so 0, eliminating all additional interest above vertex
    assertEq(interestRateModel.vertexMultiplier(), 0);
}

```

Recommendation: Set the config.vertexMultiplier based on the passed argument in _updateDynamicInterestRateModel().

3.1.10 Anyone can call getBorrowRateWithUpdate() to manipulate vertex multiplier

Submitted by [Zach Obront](#), also found by [Ver0759](#) and [JCN](#)

Severity: High Risk

Context: [DynamicInterestRateModel.sol#L290-L294](#)

Description: The getBorrowRateWithUpdate() function is intended to be called by the DToken using the model to both retrieve the current borrow rate and update the vertex multiplier based on the utilization.

However, the function is permissionless and can be called by any user, passing any arbitrary values.

The result is that any user can call the function with 100% or 0% utilization in order to push the vertex multiplier up or down.

Recommendation: Validate that calls to this function can only be made by the relevant DToken.

3.1.11 OCVE option exercise price is squared, leading to overpayment

Submitted by [Zach Obront](#), also found by [Haxatron](#), [yttriumzz](#) and [deadrosesxyz](#)

Severity: High Risk

Context: [OCVE.sol#L244-L249](#)

Description: When OCVE options are exercised, we calculate the amount to pay as follows:

1. Get the cost in payment tokens in 18 decimals:

```
uint256 optionExerciseCost = (amount * paymentTokenPerCVE) / WAD;
```

2. Convert the payment token amount to the proper decimals:

```
uint256 payAmount = _adjustDecimals(
    optionExerciseCost,
    paymentTokenDecimals,
    18
);
```

3. Multiply the option exercise cost by the amount with the adjusted decimals:

```
/// Equivalent to `(optionExerciseCost * amount) / WAD` rounded up.
payAmount = FixedPointMathLib.mulDivUp(
    optionExerciseCost,
    payAmount,
    WAD
);
```

This final step is not necessary, and leads to an exercise price dramatically greater than intended. Instead of getting the exercise price expressed in the payment token, we get this value squared.

Proof of concept: The following test can be dropped into `ExerciseOption.t.sol` to demonstrate the vulnerability:

```
function testZach_OCVEMiscalc() public {
    // let's replace the contract with another that uses ETH for payments
    // this is helpful so we can avoid the decimal issue (submitted separately)
    oCVE = new OCVE(
        ICentralRegistry(address(centralRegistry)),
        _RETH_ADDRESS
    );

    (uint256 rethCurrentPrice, ) = oracleRouter.getPrice(
        _RETH_ADDRESS,
        true,
        true
    );
    assertEq(rethCurrentPrice, 1500e18);

    // if we set the strike price to 150e36, that implies that it should be 0.1 RETH each
    oCVE.setOptionsTerms(block.timestamp, 150e36);
    assertEq(oCVE.paymentTokenPerCVE(), 0.1e18);

    // let's load up all accounts so we can see transfer size
    uint256 amountToExercise = 1000e18;
    deal(address(oCVE), address(this), amountToExercise);
    deal(address(cve), address(oCVE), amountToExercise);
    uint256 hugeBalance = type(uint128).max;
    deal(_RETH_ADDRESS, address(this), hugeBalance);
    IERC20(_RETH_ADDRESS).approve(address(oCVE), hugeBalance);

    // now if we exercise for 1000e18, the result should be that we pay 100e18 RETH
    uint256 rethBalanceBefore = IERC20(_RETH_ADDRESS).balanceOf(
        address(this)
    );
    oCVE.exerciseOption(amountToExercise);
    assertEq(
        rethBalanceBefore - IERC20(_RETH_ADDRESS).balanceOf(address(this)),
        10_000e18
    );
}
```

Recommendation:

```
- payAmount = FixedPointMathLib.mulDivUp(
-     optionExerciseCost,
-     payAmount,
-     WAD
- );
```

3.1.12 VeCVE unlock penalty formula is backwards, providing immediate free unlocks

Submitted by Zach Obront, also found by Ver0759, tvdung94, alix40, twcctop Exvul, krikolkk, Bernd Artmüller, cryptotaker, Nyksx and KupiaSec

Severity: High Risk

Context: `VeCVE.sol#L1408-L1420`

Description: Users have the ability to expire their veCVE locks early using the `earlyExpireLock()` function. The intention is that this function charges them a penalty proportional to the amount of time that is left in the lock they are expiring.

However, the implementation of the penalty calculation is backwards. Rather than charging the full penalty immediately upon locking, and slowly decreasing the penalty through `unlockTime`, the penalty starts at zero and slowly increases.

We can see this because `unlockTime - block.timestamp` is the amount of time left in the lock. `LOCK_DURATION - (unlockTime - block.timestamp)` is the amount of time that has passed so far in the lock. Dividing this value by `LOCK_DURATION` gives us the proportion of the lock that has passed.

When we multiply this by `amount * penalty`, we get a value that starts at 0 and scales up to the full penalty as the lock passes.

Proof of concept: The following test can be dropped into `ProcessExpiredLock.t.sol` to demonstrate the vulnerability:

```
function testZach_BackwardsUnlockPenalty() public {
    // set the early unlock penalty to 5000 = 50%
    centralRegistry.setEarlyUnlockPenaltyMultiplier(5000);

    // create a new lock, which will have duration of 52 weeks
    veCVE.createLock(10e18, false, rewardsData, "", 0);
    (, uint40 unlockTime) = veCVE.userLocks(address(this), 1);
    assertApproxEqAbs(unlockTime, block.timestamp + 52 weeks, 1 weeks);

    // penalty is zero
    assertEq(veCVE.getUnlockPenalty(address(this), 1), 0);

    // skip halfway through the year and check
    vm.warp(block.timestamp + 26 weeks);
    (, unlockTime) = veCVE.userLocks(address(this), 1);
    assertApproxEqAbs(unlockTime, block.timestamp + 26 weeks, 1 weeks);

    // since penalty is scaling UP instead of DOWN, it's now 10e18 * 50% * 50% of the year
    // (note that this should be 2.5e18 but there is a separate issue with the decimals)
    assertEq(veCVE.getUnlockPenalty(address(this), 1), 2.5e4);
}
```

Recommendation:

```
return
    (amount *
-      ((penalty * (LOCK_DURATION - (unlockTime - block.timestamp))) /
+      ((penalty * (unlockTime - block.timestamp)) /
        LOCK_DURATION)) / WAD;
```

3.1.13 Gauge deposits can be temporarily bricked or made insolvent due to unallocated rewards miscalculation

Submitted by [Zach Obront](#), also found by [krikolkk](#), [deadrosesxyz](#) and [KupiaSec](#)

Severity: High Risk

Context: [GaugePool.sol#L371-L382](#)

Description: In the event that the first deposits to `GaugePool.sol` come partially through the first epoch, logic has been added to the `deposit()` function to send the unallocated rewards for the period without deposits back to the DAO.

However, this logic does not account for the possibility that deposits could be made before `startTime`. This is in fact possible, because `startTime` is set to the next epoch start time, which could be anywhere from 0-7 days in the future.

```
startTime = veCVE.nextEpochStartTime();
```

Because `firstDeposit` is only set when a deposit is made after `startTime`, the first depositor after `startTime` will trigger the returning of unallocated funds:

```

if (firstDeposit == 0) {
    // If first deposit, the new rewards from gauge start to this
    // point will be unallocated rewards.
    firstDeposit = block.timestamp;
    updatePool(token);

    uint256 rewardTokensLength = rewardTokens.length;
    for (uint256 i; i < rewardTokensLength; ) {
        // Query rewardToken then increment i.
        address rewardToken = rewardTokens[i++];
        uint256 unallocatedRewards = (poolAccRewardPerShare[token][
            rewardToken
        ] * totalSupply[token]) / WAD_SQUARED;
        if (unallocatedRewards > 0) {
            SafeTransferLib.safeTransfer(
                rewardToken,
                centralRegistry.daoAddress(),
                unallocatedRewards
            );
        }
    }
}

```

As we can see, this logic calls `updatePool()`, which has already been called previously in the function. This is because the `updatePool()` function has the following check, which returns early if there are no funds deposited so far:

```

if (totalDeposited == 0) {
    return;
}

```

It is assumed that, for the first deposit, when `updatePool()` is called before the deposit is made, this check will pass and therefore return early, without performing any logic.

On the other hand, when a deposit was made before `startTime`, the first call to `updatePool()` works as usual. The job of `updatePool()` is to increment `poolAccRewardPerShare` by `totalReward / totalSupply`.

However, when this was done in the first call to `updatePool()`, the newly minted tokens were not in the total supply yet. Therefore, the rewards per share are higher than they should be for the actual current total supply, which is used when calculated `unallocatedRewards`:

```

uint256 unallocatedRewards = (poolAccRewardPerShare[token][
    rewardToken
] * totalSupply[token]) / WAD_SQUARED;

```

The result is that `unallocatedRewards` is higher than the actual reward balance that the contract should be distributing at this time. This can cause the `deposit()` function to revert if we are close enough to the end of the epoch that it's greater than the reward token balance of the contract. If this isn't the case, it will leave the contract insolvent because the remaining rewards will not be sufficient to pay all the users who earn rewards.

Proof of concept: The following test can be dropped into `TestGaugePool.t.sol` to demonstrate the vulnerability:

```

function testZach_RevertOnSecondDeposit() public {
    // set up emission rates and fund the gauge pool with cve
    address mToken = tokens[0];
    address[] memory tokensParam = new address[](1);
    tokensParam[0] = mToken;
    uint256[] memory poolWeights = new uint256[](1);
    poolWeights[0] = 1e18;
    vm.prank(address(protocolMessagingHub));
    gaugePool.setEmissionRates(0, tokensParam, poolWeights);
    deal(address(cve), address(gaugePool), 1e18);

    // make a deposit before start time
    vm.prank(mToken);
    gaugePool.deposit(mToken, address(this), 100 ether);

    // fast forward to after start time
    vm.warp(gaugePool.startTime() + 2 weeks);

    // the unallocated rewards are overestimated, so the transfer reverts
    vm.prank(mToken);
    vm.expectRevert(abi.encodeWithSignature("TransferFailed()"));
    gaugePool.deposit(mToken, address(1), 1 ether);
}

```

Recommendation: The calculations could be adjusted to account for this situation, but the most logical fix is to allow `firstDeposit` to be set before `startTime`, which would eliminate the need to send unallocated rewards to the DAO.

3.1.14 The `userNextClaimIndex` will not be increased when `userPoints` is 0 causing attacker can use expired lock to steal rewards

Submitted by [yttriumzz](#)

Severity: High Risk

Context: [CVELocker.sol#L222](#)

Description: Before any changing the lock position in the VeCVE contract, `VeCVE._claimRewards` will be called to settle the user's rewards.

```

///// https://github.com/curvance/Curvance-CantinaCompetition/blob/c01e6bda82a60ac4842bf0d3363d63e89c870758/co
↪ ntracts/token/VeCVE.sol#L1126-L1136
function _claimRewards(
    address user,
    RewardsData memory rewardsData,
    bytes memory params,
    uint256 aux
) internal {
    uint256 epochs = cveLocker.epochsToClaim(user);
    if (epochs > 0) {
        cveLocker.claimRewardsFor(user, epochs, rewardsData, params, aux);
    }
}

```

`VeCVE._claimRewards` will further call `CVELocker.epochsToClaim`, and if its return value is 0, it will return directly without calling `CVELocker.claimRewardsFor`. As long as we can get `CVELocker.epochsToClaim` to return 0, we can bypass the settlement reward.

```

////// https://github.com/curvance/Curvance-CantinaCompetition/blob/c01e6bda82a60ac4842bf0d3363d63e89c870758/contracts/architecture/CVELocker.sol#L376-L387
function epochsToClaim(address user) public view returns (uint256) {
    if (
        nextEpochToDeliver > userNextClaimIndex[user] &&
        veCVE.userPoints(user) > 0
    ) {
        unchecked {
            return nextEpochToDeliver - (userNextClaimIndex[user]);
        }
    }

    return 0;
}

```

Let's look at the implementation of `CVELocker.epochsToClaim` above. As long as the user's `userPoints` are 0, we can let it return 0.

When we claim an expired lock reward, `VeCVE.updateUserPoints` will be triggered to update the user's `userPoints`. This method can be used to reduce the user's `userPoints` to 0. That is, we can use an expired lock to bypass the settlement reward. The advantage of this is that we can keep our `userNextClaimIndex` unchanged (this parameter involves the number of reward claims).

If we succeed in keeping `userNextClaimIndex` far in the past. We can call `createLock` again (at this point `userPoints` are 0 and settlement rewards are still bypassed) to increase our `userPoints` and then immediately call `claimRewards` to claim the reward based on the incorrect `userNextClaimIndex`.

The following proof of concept demonstrates that the attacker used an expired 1 CVE lock position to steal rewards worth $100 \text{ CVE} * 10 \text{ epoch}$ (100 CVE and 10 epoch can be arbitrary).

Proof of Concept:

```

diff --git a/tests/token/VeCVE/functions/CreateLock.t.sol b/tests/token/VeCVE/functions/CreateLock.t.sol
index d01ca88d..18e1c0b2 100644
--- a/tests/token/VeCVE/functions/CreateLock.t.sol
+++ b/tests/token/VeCVE/functions/CreateLock.t.sol
@@ -194,4 +194,62 @@ contract CreateLockTest is TestBaseVeCVE {

    assertEq(usdc.balanceOf(address(this)), amount / 2);
}

+
+ function test_yttriumzz_poc_0009()
+     public
+     setRewardsData(false, false, false)
+ {
+     // 1. config poc env
+     assertEq(cveLocker.nextEpochToDeliver(), 0);
+     deal(_USDC_ADDRESS, address(cveLocker), 1000000 * 1e6);
+
+     address user00 = address(0xACC00);
+     deal(address(cve), user00, 101 * 1e18);
+     vm.prank(user00);
+     cve.approve(address(veCVE), type(uint256).max);
+
+     // 2. user00 lock 1 CVE
+     vm.prank(user00);
+     veCVE.createLock(1e18, false, rewardsData, "", 0);
+     assertEq(cveLocker.userNextClaimIndex(user00), 0);
+     assertEq(veCVE.userUnlocksByEpoch(user00, 26), 1e18);
+
+     // 3. Lock time (26 + 1 epoch) passed
+     for (uint256 i = 0; i < 27; i++) {
+         vm.warp(block.timestamp + 2 weeks);
+         vm.prank(centralRegistry.feeAccumulator());
+         cveLocker.recordEpochRewards(1e6);
+         assertEq(cveLocker.nextEpochToDeliver(), i + 1);
+     }
+     assertEq(cveLocker.nextEpochToDeliver(), 27);
+
+     // 4. The lock time of user00 has expired.
+     //     He just claim the reward but does not call `processExpiredLock`,
+     //     so his `userPoints` is 0.
+     vm.prank(user00);

```

```

+     cveLocker.claimRewards(rewardsData, "", 0);
+     assertEq(veCVE.userPoints(user00), 0);
+     assertEq(usdc.balanceOf(user00), 26 * 1e6);
+
+     // 5. Pass 18 epoch (the longer time passes, the more users can steal)
+     for (uint256 i = 0; i < 18; i++) {
+         vm.warp(block.timestamp + 2 weeks);
+         vm.prank(centralRegistry.feeAccumulator());
+         cveLocker.recordEpochRewards(1e6);
+         assertEq(cveLocker.nextEpochToDeliver(), i + 28);
+     }
+     assertEq(cveLocker.nextEpochToDeliver(), 27 + 18);
+
+     // 6. After user00 lock 100 CVE,
+     //     he can immediately claim the rewards of 100 CVE for the past 10 epochs.
+     uint256 user00USDCBalanceBeforeHack = usdc.balanceOf(user00);
+     vm.startPrank(user00);
+     veCVE.createLock(100 * 1e18, false, rewardsData, "", 0);
+     cveLocker.claimRewards(rewardsData, "", 0);
+     vm.stopPrank();
+     assertEq(
+         usdc.balanceOf(user00) - user00USDCBalanceBeforeHack,
+         100 * 1e6 * 18
+     );
+ }
+ }

```

Recommendation: `userNextClaimIndex` should continue to accumulate even if `userPoints` is 0:

```

function epochsToClaim(address user) public view returns (uint256) {
    if (
-         nextEpochToDeliver > userNextClaimIndex[user] &&
-         veCVE.userPoints(user) > 0
+         nextEpochToDeliver > userNextClaimIndex[user]
    ) {
        unchecked {
            return nextEpochToDeliver - (userNextClaimIndex[user]);
        }
    }

    return 0;
}

```

3.1.15 `chainPoints` is never decremented in normal lock expiry, leading to wasted CVE

Submitted by [Zach Obront](#), also found by [Haxatron](#), [twcctop](#), [Exvul](#), [alix40](#), [yttriumzz](#), [Ch301](#), [Bernd Artmüller](#), [Nyksx](#) and [KupiaSec](#)

Severity: High Risk

Context: `VeCVE.sol#L901-L915`

Description: The total epoch rewards for a given chain are calculated pro rata by taking the total chainPoints of each chain and dividing the total rewards proportionately. This result is represented in `rewardsPerCVE` and that value, along with the necessary CVE to fund it, are sent to each chain.

`chainPoints` are incremented using the `_incrementPoints()` function each time a new lock is created, or changed to continuous. `chainPoints` are decremented using `_reducePoints()` any time locks are unlocked or changed away from continuous.

The one exception is that, when a lock expires, the updates to the user's points are done by the `CVELocker` contract. In `_calculateRewardsPerEpoch()`, it calls `veCVE.updateUserPoints()`:

```
function _calculateRewardsForEpoch(
    address user,
    uint256 epoch
) internal returns (uint256) {
    if (veCVE.userUnlocksByEpoch(user, epoch) > 0) {
        // If they have tokens unlocking this epoch we need to decrease
        // their tokenPoints.
        veCVE.updateUserPoints(user, epoch);
    }

    return (veCVE.userPoints(user) * epochRewardsPerCVE[epoch]);
}
```

Looking at the implementation of this function, we can see that it decrements `userPoints` but leaves `chainPoints` unchanged:

```
function updateUserPoints(address user, uint256 epoch) external {
    address _cveLocker = address(cveLocker);
    assembly {
        if iszero(eq(caller(), _cveLocker)) {
            mstore(0x00, _UNAUTHORIZED_SELECTOR)
            revert(0x1c, 0x04)
        }
    }

    unchecked {
        userPoints[user] =
            userPoints[user] -
            userUnlocksByEpoch[user][epoch];
    }
}
```

The result is that `chainPoints` will become greater than the sum of the individual `userPoints`, at approximately the rate of 2x per year (since locks are 1 year and this happens on each non-early unlock). This has two consequences:

- 1) Chains will be incorrectly allocated CVE based on these numbers.
- 2) Worse, the `rewardsPerCVE` will be based on the higher `chainPoints` estimates. Many of these points don't really exist, and therefore can never be claimed. The result is that a large portion of the CVE will be wasted and unclaimable.

Recommendation:

```
function updateUserPoints(address user, uint256 epoch) external {
    address _cveLocker = address(cveLocker);
    assembly {
        if iszero(eq(caller(), _cveLocker)) {
            mstore(0x00, _UNAUTHORIZED_SELECTOR)
            revert(0x1c, 0x04)
        }
    }

    unchecked {
        userPoints[user] =
            userPoints[user] -
            userUnlocksByEpoch[user][epoch];
+       chainPoints = chainPoints - userUnlocksByEpoch[user][epoch];
    }
}
```

3.1.16 `_statusOf()` overwrite the `maxDebt` value

Submitted by [Ch301](#), also found by [Naveen Kumar J](#) - [1nc0gn170](#), [bbl4de](#), [krikolkk](#), [inspex](#), [Bernd Artmüller](#), [Topmark](#), [Oxmint](#), [KupiaSec](#), [kaka](#) and [solthodox](#)

Severity: High Risk

Context: [LiquidityManager.sol#L224](#)

Description: In `LiquidityManager.sol` the function `_statusOf()`. It is to determine account's current status between `sumCollateral`, `maxDebt` and `sumDebt`. However, in the below snapshot code, we found that the `maxDebt` is kept overwritten if the user has two or more CTokens.

```
maxDebt = (collateralValue * tokenData[snapshot.asset].collRatio) / WAD;
```

`maxDebt` should keep collecting all the possible debt in `maxDebt`. So, It should be `maxDebt += ...`

This will affect the maximum leverage value in `PositionFolding.sol`, so users are not able to take use advantage of MAX LEVERAGE

Recommendation:

```
- maxDebt = (collateralValue * tokenData[snapshot.asset].collRatio) / WAD;  
+ maxDebt += (collateralValue * tokenData[snapshot.asset].collRatio) / WAD;
```

3.1.17 Attacker will claim all past epochs

Submitted by [Ch301](#), also found by [deadrosesxyz](#)

Severity: High Risk

Context: `CVELocker.sol`#L355-L366

Description: This issue related to `CVELocker.sol` and `VeCVE.sol`, with the fact that anyone can increase his `userNextClaimIndex[user]` using `CVELocker.sol`#`manageRewardsFor()` function.

1. The attacker needs to wait for at least `genesisEpoch + 1 Epoch`. The later he starts this attack the more rewards will steal.
2. Attacker call `createLock()`, with `continuousLock == false`. (this lock is only to bypass this check if `(userLocks[recipient].length == 0)` in `_lock()` in Step 5. So, `amount == 1e18` is enough here).
3. He will invoke `setDelegateApproval()` and set the delegation to another address he controls.
4. Now, trigger `manageRewardsFor()`, with `Epochs == 27`.

At this point attacker manipulates his `userNextClaimIndex[user]` index. (this will prevent him from claiming rewards for the next 27 Epoch. But, at the end of this attack he will claim everything).

5. He will create another lock, with `continuousLock == false`.
6. The attacker needs to wait one year (or start another attack).
7. The attacker initiates `processExpiredLock()` two times for both his locks, and will receive the initial amount of CVE.

Note: at this point, the `userNextClaimIndex(address(Attacker))` returns zero, but `userPoints(address(Attacker))` positive value in fact this is the amount of points of his second lock.

8. Now, he needs just to call `CVELocker.sol`#`claimRewards()`.
9. Repeat:

Steps 3 and 7 are connected to each other.

So, Step 3 will allow you to keep the points from the second locks (so, the more you lock in Step 5 the more you steal at the end) in Step 7.

The attacker can claim rewards from all the past epochs. with his rewards from locking CVE for 52 weeks and he will return the initial amount of CVE. So, no loss here.

Proof of concept: Please copy the following proof of concept in `IncreaseAmountAndExtendLockFor.t.sol`:

```
function test_POC_manageRewardsFor() public {  
    //user burn USDC  
    vm.startPrank(address(2));  
    uint user_USDC = usdc.balanceOf(address(2));  
    usdc.transfer(address(5), user_USDC);  
    vm.stopPrank();  
  
    deal(address(cve), address(2), 100e18);
```

```

deal(_USDC_ADDRESS, address(cveLocker), 100e18);

vm.warp(block.timestamp + 104 weeks);
// Record rewards per CVE for the epochs (two years)
//52 Epoch || 104 weeks
vm.startPrank(centralRegistry.feeAccumulator());
for (uint256 index = 0; index /*Epoch*/ < 52; index++) {
    cveLocker.recordEpochRewards(1e10);
}
uint nextEpochToDeliver = cveLocker.nextEpochToDeliver();
console.log(
    "After two year `nextEpochToDeliver`:",
    nextEpochToDeliver
);
vm.stopPrank();

//user createLock `Lock index == 0`. nb: Continuous Lock
vm.startPrank(address(2));
cve.approve(address(veCVE), 100e18);
veCVE.createLock(1e18, false, rewardsData, "", 0);
cveLocker.setDelegateApproval(address(3), true);
vm.stopPrank();

// Manipulate `userNextClaimIndex[]` value. 27 Epoch
vm.startPrank(address(3));
cveLocker.manageRewardsFor(address(2), 27);
vm.stopPrank();

uint userNextClaimIndex = cveLocker.userNextClaimIndex(address(2));
console.log("After userNextClaimIndex", userNextClaimIndex);

//user createLock `Lock index == 1`. nb: one year
vm.startPrank(address(2));
veCVE.createLock(10e18, false, rewardsData, "", 0);
vm.stopPrank();

//wait one year
vm.warp(block.timestamp + 53 weeks);
// Record rewards per CVE for the epochs (one years)
//26 Epoch || 52 weeks
vm.startPrank(centralRegistry.feeAccumulator());
for (uint256 index = 0; index /*Epoch*/ < 26; index++) {
    cveLocker.recordEpochRewards(1e5);
}
nextEpochToDeliver = cveLocker.nextEpochToDeliver();
console.log(
    "After three year `nextEpochToDeliver`:",
    nextEpochToDeliver
);
vm.stopPrank();

//User process his expired locks
vm.startPrank(address(2));
veCVE.processExpiredLock(1, false, false, rewardsData, "", 0);
veCVE.processExpiredLock(0, false, false, rewardsData, "", 0);
vm.stopPrank();

// The goal is `userPoints > 0` & `userNextClaimIndex == 0`
userNextClaimIndex = cveLocker.userNextClaimIndex(address(2));
uint userPoints = veCVE.userPoints(address(2));
console.log("userNextClaimIndex =", userNextClaimIndex);
console.log("userPoints =", userPoints);

//Time of harvest
//nb: at this point the user will claim Rewards for three years
vm.startPrank(address(2));
uint user_USDC_Balance = usdc.balanceOf(address(2));
console.log("Before harvest - user USDC Balance :", user_USDC_Balance);
cveLocker.claimRewards(rewardsData, "", 0);
user_USDC_Balance = usdc.balanceOf(address(2));
console.log("After harvest - user USDC Balance :", user_USDC_Balance);
vm.stopPrank();
}

```

Recommendation: A simple fix will be in `manageRewardsFor()`:

```

function manageRewardsFor(
+   address user
-   address user,
-   uint256 epochs
) external nonReentrant returns (uint256) {
    if (!_checkIsDelegate(user, msg.sender)) {
        _revert(_UNAUTHORIZED_SELECTOR);
    }

+   uint256 epochs = epochsToClaim(user);
+   if (epochs == 0) {
+       return ;
+   }

    // We check whether there are epochs to claim in reward manager
    // modules so we do not need to check here like in claimRewards.
    return _claimRewardsDirect(user, epochs);
}

```

3.1.18 DToken transfers are illegitimately restricted if a lender is insolvent

Submitted by *Bernd Artmüller*, also found by *KupiaSec*

Severity: High Risk

Context: [DToken.sol#L1116](#)

Description: Lenders that supply liquidity for a specific underlying token to a market receive dTokens in return. dTokens can later be redeemed for the underlying token to receive the accumulated interest.

dTokens can be transferred to other addresses via the `transfer` and `transferFrom` functions.

Internally, the `_transfer` function checks if the transfer is allowed by calling the `MarketManager.canTransfer` function, which checks the user's liquidity (i.e., if there's a collateral surplus or deficit) via the `_canRedeem` function.

If there is a liquidity deficit, i.e., the collateral is insufficient to cover the debt, the transfer is reverted.

However, this check does not make sense for dTokens, as they do not contribute to a user's liquidity, only posted collateral and borrowed tokens (debt) do. Moreover, dTokens can not be seized during liquidations.

Consequently, there is no need to prevent the transfer of dTokens in case of a liquidity deficit. On the contrary, this illegitimately restricts the lender's ability to transfer their dTokens.

Specifically, this issue is present if a lender has provided liquidity for a specific underlying token (thus received dTokens) and at the same time also borrowed the same underlying tokens via the dToken, while the posted collateral is insufficient to back the debt (i.e., collateral value dropped due to market volatility). In this case, the dTokens that the user added liquidity for can't be transferred, even though they do not contribute to the user's liquidity and are not accessible during liquidations.

Lenders are unable to transfer dTokens.

Recommendation: Consider removing the liquidity deficit check for dToken transfers.

3.1.19 Wrong underlying price is used when removing collateral

Submitted by *Nyksx*

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: When getting the underlying prices inside the `_hypotheticalLiquidityOf` function, if the asset has two feeds available and is cToken, the lower price feed will be used; if it isn't a cToken, the higher feed will be used. This is the correct way to calculate `maxDebt` because the calculation should favor the protocol.

```

if (snapshot.isCToken) {
    //...
    if (cr != 0) {
        // Cache collateral posted.
        posted = tokenData[snapshot.asset].accountPositions[
            account].collateralPosted;

        // If there is no collateral posted and its not a position
        // to be modified, clean up the position entry as the user
        // was liquidated.
        if (posted == 0) {
            //...
        } else {
            // There is collateral posted in this cToken, and the user
            // can take on more debt.
            maxDebt = _liquidityValue(
                maxDebt,
                posted,
                snapshot.exchangeRate,
                underlyingPrices[i],
                snapshot.decimals,
                cr
            );
        }
    }
}
}

```

The newDebt calculation is also correct because it uses the higher price feed and favors the protocol because of that.

```

else {
    // If they have a debt balance, increment their debt.
    if (snapshot.debtBalance > 0) {
        newDebt += _assetValue(
            snapshot.debtBalance,
            underlyingPrices[i],
            snapshot.decimals
        );
    }
}

```

For the redemption action, the function adds the redemption amount to newDebt to check if there is a liquidity deficit or not.

```

// Calculate impact of mTokenModified action.
if (action.mTokenModified == snapshot.asset) {
    //...
    if (snapshot.isCToken) {
        // If the cToken has a Collateralization Ratio,
        // increase their new debt.
        if (cr != 0) {
            // If they are trying to redeem more tokens than they have,
            // the transaction will fail before it gets to this point,
            // so no special case needed.
            if (posted == action.redeemTokens) {
                positionsToClose[i] = true;
                if (result.updateNeeded == 0) {
                    result.updateNeeded = 1;
                }
            }

            // Hypothetical redemption action.
            newDebt += _redemptionValue(
                action.redeemTokens,
                snapshot.exchangeRate,
                underlyingPrices[i],
                snapshot.decimals,
                cr
            );
        }
    } else {
        //...
    }
}
}

```

The problem is that it uses the lower price feed because the token is cToken, but it does not favor the protocol. Using the lower price feed when adding newDebt will result in lower debt for the user, which means the user can remove more collateral than it should.

Recommendation: When redeeming, the higher price needs to be used.

3.1.20 Early veCVE unlock penalty multiplier is not converted to WAD

Submitted by *Bernd Artmüller*, also found by *alix40*, *krikolkk*, *Zach Obront*, *Nyksx*, *cryptostaker* and *MaslarovK*

Severity: High Risk

Context: [CentralRegistry.sol#L622](#)

Description: The `setEarlyUnlockPenaltyMultiplier` setter function in the `CentralRegistry` contract sets the `earlyUnlockPenaltyMultiplier` storage variable to the provided value. `value` is provided in basis points and expected to be within the range of 3000 (30%) to 9000 (90%).

However, this penalty multiplier is expected to be in WAD, as seen in the `VeCVE._getUnlockPenalty` function, which calculates the penalty amount. As a result, as the multiplier is in basis points, the penalty amount will be significantly lower than expected, incentivizing early unlocks due to insufficient penalties.

Early veCVE unlock penalties are very low, incentivizing early unlocks and contradicting the lock-up period's intended purpose.

Recommendation: In the `setEarlyUnlockPenaltyMultiplier` function, consider using `_bpToWad` to scale value to WAD.

3.1.21 CVELocker.recordEpochRewards is called twice on a recipient chain, preventing reward claiming and stealing rewards

Submitted by *Bernd Artmüller*

Severity: High Risk

Context: [ProtocolMessagingHub.sol#L179](#), [FeeAccumulator.sol#L378](#)

Description: During fee distribution and reward epoch recording, executed by the `FeeAccumulator.executeEpochFeeRouter` function, chains (`CentralRegistry.supportedChains`) are looped twice:

1. In lines 791-815 of the `_executeEpochFeeRouter` function to distribute pro-rata fees via `sendFees`.
2. In lines 458-481 to notify the chains of the per epoch rewards via calling `ProtocolMessagingHub.sendWormholeMessages`.

This ultimately results in the `CVELocker`'s `recordEpochRewards` function being called twice on each receiving chain:

1. When pro-rata fees are received by the remote chain's messaging hub with payload id 1, calling the `recordEpochRewards` function in line 179.
2. When receiving the per epoch CVE rewards message as payload id 4 + `messageType` = 2 and calling the fee accumulator's `receiveExecutableLockData` function, which internally calls `recordEpochRewards` again.

This `FeeAccumulator.receiveExecutableLockData` function calls the `CVELocker.recordEpochRewards` function, which increments the `nextEpochToDeliver` variable.

Due to calling `recordEpochRewards` twice, the `nextEpochToDeliver` variable will be incremented twice, causing its value to drift away from the actual current epoch (i.e., becoming larger than the current epoch). Consequently, users (veCVE token holder) claiming their rewards will attempt to claim rewards for future epochs while the rewards are not yet available, stealing rewards from other users and the last users to claim will be unable to do so due to insufficient tokens to transfer.

Users claim rewards for future epochs and thus steal rewards from others.

Recommendation: Consider ensuring that the `recordEpochRewards` function is only called once per chain and per epoch.

3.1.22 Posted collateral tokens are transferred without adjusting the user's posted cToken amount

Submitted by *Bernd Artmüller*, also found by *alix40*, *tv dung94*, *KupiaSec* and *yttriumzz*

Severity: High Risk

Context: CTokenBase.sol#L434-L446

Description: Collateral tokens (cTokens) are transferrable via the `transfer` and `transferFrom` functions.

To prevent transferring cTokens that are posted as collateral and backing debt, both transfer functions call the `MarketManager.canTransferWithPrune` function to check if the transfer is allowed (e.g., checking if transfers are paused, whether the minimum hold period is met, ensuring sufficient collateral is backing the debt).

If the transfer is allowed, the cTokens are transferred to the specified address. However, the user's `collateralPosted` value is **not** adjusted to reflect the transferred amount.

Impact:

- Undercollateralized debt position.
- An attacker can transfer already posted collateral tokens from account A to another self-controlled account B without having the posted collateral adjusted. Thereafter, the attacker redeems the unadjusted collateral from the market via account A, effectively stealing collateral from other users.

Proof of concept: The following Pproof of concept demonstrates how a user can transfer already posted collateral tokens to another account, without having the posted collateral adjusted.

To run the PoC, add the test case to `tests/market/collateral/integrations/TestTokenInteractions.t.sol`:

```
function testCTokenTransferOnBorrowAllowsTransferringAllTokens() public {
    _prepareBALRETH(user1, 1 ether);

    // try mint()
    vm.startPrank(user1);
    balRETH.approve(address(cBALRETH), 1 ether);
    cBALRETH.deposit(1 ether, user1);
    marketManager.postCollateral(user1, address(cBALRETH), 1 ether);
    vm.stopPrank();

    // try borrow()
    vm.startPrank(user1);
    dDAI.borrow(500 ether);
    vm.stopPrank();

    // skip min hold period
    skip(20 minutes);

    // can't transfer full
    vm.startPrank(user1);
    vm.expectRevert(
        bytes4(keccak256("MarketManager__InsufficientCollateral()"))
    );
    cBALRETH.transfer(user2, 1 ether);
    vm.stopPrank();

    // can transfer/redeem partially
    vm.startPrank(user1);
    cBALRETH.transfer(user2, 0.2 ether);
    cBALRETH.transfer(user2, 0.2 ether);
    cBALRETH.transfer(user2, 0.2 ether);
    cBALRETH.transfer(user2, 0.2 ether);
    cBALRETH.transfer(user2, 0.2 ether);
    vm.stopPrank();

    assertFalse(marketManager.flaggedForLiquidation(user1)); // @audit-info user1 is _not_ flagged for
    ↪ liquidation due to collateral posted being stale, even though there is no collateral backing the debt
    assertEquals(cBALRETH.balanceOf(user1), 0 ether);
    assertEquals(cBALRETH.balanceOf(user2), 1 ether);
    assertEquals(cBALRETH.exchangeRateCached(), 1 ether);

    (bool hasPosition, uint256 balanceOf, uint256 collateralPostedOf) = marketManager.tokenDataOf(user1,
    ↪ address(cBALRETH));
```

```

    assertTrue(hasPosition);
    assertEq(balanceOf, 0);
    assertEq(collateralPostedOf, 1 ether); // @audit-info collateral is still posted
}

```

Recommendation: Consider adjusting the user's `collateralPosted` value in both the `transfer` and `transferFrom` functions to reflect the transferred `cToken` amount.

3.1.23 Collateral tokens that have not been posted in the market may not be able to be transferred due to the liquidity check

Submitted by *Bernd Artmüller*, also found by *KupiaSec*, *alix40* and *yttriumzz*

Severity: High Risk

Context: `MarketManager.sol`#L724-L728

Description: Collateral tokens (`cTokens`) are transferrable via the `transfer` and `transferFrom` functions. To prevent transferring `cTokens` that are posted as collateral and backing debt, both transfer functions call the `MarketManager.canTransferWithPrune` function to check if (uncomplete list):

1. The transfer is allowed (e.g., checking if transfers are paused).
2. Whether the minimum hold period is met.
3. Ensuring sufficient collateral is backing the debt.

The third check, ensuring sufficient collateral is backing the debt, is performed by calling the `_canRedeem` function, which then checks the account's liquidity via the `_hypotheticalLiquidityOf` function in lines 1494-1502:

```

1461: function _canRedeem(
1462:     address mToken,
1463:     address account,
1464:     uint256 amount
1465: ) internal view returns (uint256, bool[] memory){
...
// [...]
1490:
1491:     // Check account liquidity with hypothetical redemption.
1492:     // We check liquidity even for dToken redemptions to prevent any
1493:     // potential invariant manipulation creating a scenario of insolvency.
1494:     (HypotheticalData memory result, bool[] memory positionsToClose) = _hypotheticalLiquidityOf(
1495:         account,
1496:         HypotheticalAction({
1497:             mTokenModified: mToken,
1498:             redeemTokens: amount,
1499:             borrowAmount: 0,
1500:             errorCodeBreakpoint: 2
1501:         })
1502:     );
// ...
// [...]

```

The transfer amount, `amount` is considered as a redemption in the `_hypotheticalLiquidityOf` function, i.e., increasing the `newDebt` value by the transfer amount (in USD terms).

This `newDebt` value is compared to the maximum debt (`maxDebt`) and the difference is calculated as either a `collateralSurplus` or `liquidityDeficit` in lines 391-404:


```

391: // These will not underflow/overflow as condition is checked prior.
392: // Returns excess liquidity on hypothetical positions.
393: if (maxDebt > newDebt) {
394:     unchecked {
395:         result.collateralSurplus = maxDebt - newDebt;
396:     }
397:
398:     return (result, positionsToClose);
399: }
400:
401: // Returns shortfall on hypothetical positions.
402: unchecked {
403:     result.liquidityDeficit = newDebt - maxDebt;
404: }

```

If there's a liquidity deficit, the token transfer is prevented.

However, this can result in a situation where a user has enough collateral backing the debt, but transferring the remaining collateral is prevented due to the assumption that the resulting collateral is insufficient to back the debt.

Consider the following example: A user has 200 collateral token, 100 tokens are posted in the market while the other 100 tokens are not. The ctoken has a collateral ratio of 80%, allowing the user to borrow 80 debt tokens (assuming collateral to debt token price is 1:1).

The user borrowed 80 debt tokens, backed by the 100 cTokens.

However, transferring the other 100 collateral tokens that have not been posted in the market is prevented due to the liquidity check in `_hypotheticalLiquidityOf` function. Specifically, `newDebt` is calculated as $80 + (100 * 0.8) = 160$, which exceeds the posted collateral of $100 * 80\% = 80$. As a result, the transfer is prevented and the user is not able to freely transfer the collateral tokens, even though there is enough collateral backing the debt.

Impact: Users are unable to transfer collateral tokens (both posted and non-posted).

Recommendation: Consider checking in the `canTransferWithPrune` function if the transfer amount requires the posted collateral to be touched and used, and if so, check the liquidity. Otherwise, allow the transfer to proceed.

3.1.24 Wrong decimal scalation will result in severe miscalculation of amounts

Submitted by *solthodox*, also found by *MaslarovK*, *yttriumzz*, *Zach Obront*, *deadrosesxyz* and *yixxas*

Severity: High Risk

Context: OCVE.sol#L238

Description: In the `exerciseOption` function of OCVE the user burns an amount of OCVE to get the right to buy CVE at the price specified at the contract. CVE has 18 decimals and this won't have any impact if `paymentToken` does so too. But in the case the `paymentToken` has != 18 decimals (e.g. USDC) the buyer will pay either extremely high amounts if the payment token has lesser than 18 decimals, or extremely low amounts if it has more than 18 decimals (quite unlikely realistically).

Proof of concept: The `optionExerciseCost` is scaled to 18 decimals, because `amount` and `paymentTokenPerCVE` are in 18 decimals and divided by 18 decimals again ($1e18 * 1e18 / 1e18$). Let's say for this example that the payment token is USDC (6 decimals) and the USD price of CVE is 2. That means that we must pay 2 USDC for every CVE we want to get. Let's say we want to get 1 CVE. To calculate the pay amount we want to reduce the decimals from 18 to 6. It turns out the code does the opposite:

```

// 18 decimals
// 1e18 * 2e8 / 1e18 = 2e8
uint256 optionExerciseCost = (amount * paymentTokenPerCVE) / WAD;
//....
uint256 payAmount = _adjustDecimals(
    optionExerciseCost, // amount(18 decimals)
    paymentTokenDecimals, // from decimals(6)
    18 // to decimals
);

```



```
function _adjustDecimals(
    uint256 amount, // 2e18
    uint8 fromDecimals, // 6
    uint8 toDecimals // 18
) internal pure returns (uint256) {
    //...
    // we enter here:
    else if (fromDecimals < toDecimals) {
        // result: 2e18 * 10 ** (18 - 6) = 2e30
        return amount * 10 ** (toDecimals - fromDecimals);
    }
    //...
}
```

As demonstrated in this example the contract will ask for 2e30 USDC units(2e22 USDC) instead of 2e6 units (2 USDC).

Recommendation: Scale decimals in the opposite direction:

```
uint256 payAmount = _adjustDecimals(
    optionExerciseCost,
    18,
    paymentTokenDecimals
);
```

3.1.25 Missing RateProvider normalization ensures inaccurate pricing of BalancerStablePool

Submitted by *cuthalion0x*, also found by *GeneralKay* and *pks271*

Severity: High Risk

Context: [BalancerStablePoolAdaptor.sol#L107-L121](#)

Description: Balancer's MetaStablePool and ComposableStablePool store a RateProvider for each underlying token which expresses each token's price with respect to a common base. This is useful for yield-bearing tokens in particular, such as RocketPool's rETH or Lido's wstETH. Indeed, the test pool used in this code base within TestBalancerStablePoolAdaptor.t.sol holds two tokens: WETH and rETH.

When pricing these BPT, it is imperative to perform all price operations in normalized units. For example, if using the classical min-price algorithm, the min-value comparisons become meaningless if prices are not expressed in the same units. Similarly, the average-price algorithm used here will produce meaningless results if the prices are not normalized prior to summation.

The Balancer pool's getRate() function already accounts for the underlying token rates, so in our example WETH/rETH pool, the rETH rate is already accounted for. If we include it within our averagePrice and then multiply again by pool.getRate(), the rate will be double-counted and the resulting price will be too high. To solve this, we must normalize each constituent price by dividing out the RateProvider rate prior to computing the average.

Proof of concept: The following diff enhances the TestBalancerStablePoolAdaptor test by comparing the computed price to an expected value rather than simply asserting that it is greater than zero:

```
diff --git a/tests/oracles/OracleRouter/integrations/TestBalancerStablePoolAdaptor.t.sol
↪ b/tests/oracles/OracleRouter/integrations/TestBalancerStablePoolAdaptor.t.sol
index 9adcac1e..f72c9a42 100644
--- a/tests/oracles/OracleRouter/integrations/TestBalancerStablePoolAdaptor.t.sol
+++ b/tests/oracles/OracleRouter/integrations/TestBalancerStablePoolAdaptor.t.sol
@@ -7,6 +7,9 @@ import { IVault } from "contracts/oracles/adaptors/balancer/BalancerBaseAdaptor.
import { ChainlinkAdaptor } from "contracts/oracles/adaptors/chainlink/ChainlinkAdaptor.sol";
import { ICentralRegistry } from "contracts/interfaces/ICentralRegistry.sol";
import { OracleRouter } from "contracts/oracles/OracleRouter.sol";
+import { IBalancerPool } from "contracts/interfaces/external/balancer/IBalancerPool.sol";
+import { IERC20 } from "contracts/interfaces/IERC20.sol";
+import { console2 } from "forge-std/console2.sol";

contract TestBalancerStablePoolAdaptor is TestBaseOracleRouter {
    address internal constant _BALANCER_VAULT =
@@ -22,10 +25,12 @@ contract TestBalancerStablePoolAdaptor is TestBaseOracleRouter {
    bytes32 private WETH_RETH_POOLID =
        0x1e19cf2d73a72ef1332c882f20534b6519be02760002000000000000000000112;
```

```

+   uint256 private WETH_RETH_TVL_USD = 58_666_383e18; // from Balancer web UI at fork block
+
+   BalancerStablePoolAdaptor adaptor;
+
+   function setUp() public override {
-       _fork(18031848);
+       _fork(19656276);
+
+       _deployCentralRegistry();
+       _deployOracleRouter();
@@ -88,13 +93,36 @@ contract TestBalancerStablePoolAdaptor is TestBaseOracleRouter {
+       oracleRouter.addApprovedAdaptor(address(adaptor));
+       oracleRouter.addAssetPriceFeed(WETH_RETH, address(adaptor));
+
+       (uint256 wethPrice, ) = oracleRouter.getPrice(
+           WETH,
+           true,
+           false
+       );
+       console2.log('WETH price: ', wethPrice);
+
+       (uint256 rethPrice, ) = oracleRouter.getPrice(
+           RETH,
+           true,
+           false
+       );
+       console2.log('RETH price: ', rethPrice);
+
+       uint256 expectedPriceFromTvl = WETH_RETH_TVL_USD * 1e18 / IERC20(WETH_RETH).totalSupply();
+       console2.log('expected RETH/WETH price from TVL: ', expectedPriceFromTvl);
+
+       uint256 expectedPriceFromRate = IBalancerPool(WETH_RETH).getRate() * wethPrice / 1e18;
+       console2.log('expected RETH/WETH price from ETH rate: ', expectedPriceFromRate);
+
+       (uint256 price, uint256 errorCode) = oracleRouter.getPrice(
+           WETH_RETH,
+           true,
+           false
+       );
+       console2.log('computed RETH/WETH price: ', price);
+
+       assertEq(errorCode, 0);
-       assertGt(price, 0);
+       assertApproxEqRel(price, expectedPriceFromTvl, 0.01e18); // 1% error allowed
+       assertApproxEqRel(price, expectedPriceFromRate, 0.01e18);
+   }
+
+   function testRevertAfterAssetRemove() public {

```

At commit [76669e36](#), the enhanced test fails with a price that is too high because it double-counts the rETH rate. Please consult the logs below:

```

WETH price: 307433610000000000000000
RETH price: 339375962079000000000000
expected RETH/WETH price from TVL: 3174908455794615477293
expected RETH/WETH price from ETH rate: 3173314294197332376565
computed RETH/WETH price: 3338167971780883793528

```

Recommendation: The following diff patches the issue by normalizing each constituent price by the rate from its corresponding RateProvider:

```

diff --git a/contracts/oracles/adaptors/balancer/BalancerStablePoolAdaptor.sol
↪ b/contracts/oracles/adaptors/balancer/BalancerStablePoolAdaptor.sol
index f0bb348d..e148a7fa 100644
--- a/contracts/oracles/adaptors/balancer/BalancerStablePoolAdaptor.sol
+++ b/contracts/oracles/adaptors/balancer/BalancerStablePoolAdaptor.sol
@@ -118,7 +118,16 @@ contract BalancerStablePoolAdaptor is BalancerBaseAdaptor {

    // We did not have an error, so we can add the price
    // to the average, and increment number of prices.
-    averagePrice += price;
+    // @audit We must first normalize the price using the rate from the RateProvider.
+    // @audit If there is no RateProvider, assume a rate of 1 (note that `rateProviderDecimals` is
↪ unreliable in this case).
+    address rateProvider = data.rateProviders[i];
+    uint256 normalizedPrice;
+    if (rateProvider == address(0)) {
+        normalizedPrice = price;
+    } else {
+        normalizedPrice = (price * (10 ** data.rateProviderDecimals[i])) /
↪ IRateProvider(rateProvider).getRate();
+    }
+    averagePrice += normalizedPrice;
+    ++numPrices;
}

```

After introducing this recommendation, the enhanced test succeeds with the following logs:

```

WETH price: 307433610000000000000000
RETH price: 339375962079000000000000
expected RETH/WETH price from TVL: 3174908455794615477293
expected RETH/WETH price from ETH rate: 3173314294197332376565
computed RETH/WETH price: 3173702335773619101965

```

3.1.26 The CCTP receiver cannot call `MessageTransmitter.receiveMessage` causing can not to receive the USDC from the remote chain

Submitted by *yttriumzz*, also found by *Haxatron*

Severity: High Risk

Context: `FeeTokenBridgingHub.sol#L144`

Description: The `FeeTokenBridgingHub` contract has two methods to send tokens to the peer, one of which is through CCTP, which is implemented in `FeeTokenBridgingHub._transferFeeTokenViaCCTP`.

```

///// https://github.com/curvature/Curvature-CantinaCompetition/blob/c01e6bda82a60ac4842bf0d3363d63e89c870758/co
↪ ntracts/architecture/FeeTokenBridgingHub.sol#L139-L145
uint64 nonce = circleTokenMessenger.depositForBurnWithCaller(
    amount,
    centralRegistry.cctpDomain(dstChainId),
    bytes32(uint256(uint160(to))),
    feeToken,
    bytes32(uint256(uint160(to)))
);

```

As shown in the above code, it uses `circleTokenMessenger.depositForBurnWithCaller` to interact with CCTP. Let's take a look at the documentation for this function.

Same as `depositForBurn` but with an additional parameter, `destinationCaller`. This parameter specifies which address has permission to call `receiveMessage` on the destination domain for the message.

reference: <https://developers.circle.com/stablecoins/docs/evm-smart-contracts#depositforburnwithcaller>

It limits the callers of `receiveMessage` on the target chain. The `_transferFeeTokenViaCCTP` function uses `to` as the address that the target chain caller has permission to call `receiveMessage`. This is wrong because:

- 1) The receiver does not implement the code to interact with `receiveMessage`.

2) The `to` address is a local chain address rather than the target chain address.

This results in the USDC transferred from remote chain being unable to be withdrawn.

Recommendation: Use CCTP's `depositForBurn` function or implement a function that interacts with `receiveMessage`:

```
function _transferFeeTokenViaCCTP(
    ITokenMessenger circleTokenMessenger,
    uint256 dstChainId,
    address to,
    uint256 amount,
    uint256 wormholeFee
) internal {
    IWormholeRelayer wormholeRelayer = centralRegistry.wormholeRelayer();
    uint16 wormholeChainId = centralRegistry.wormholeChainId(dstChainId);

    SwapperLib._approveTokenIfNeeded(
        feeToken,
        address(circleTokenMessenger),
        amount
    );

-   uint64 nonce = circleTokenMessenger.depositForBurnWithCaller(
+   uint64 nonce = circleTokenMessenger.depositForBurn(
        amount,
        centralRegistry.cctpDomain(dstChainId),
        bytes32(uint256(uint160(to))),
        feeToken,
        bytes32(uint256(uint160(to)))
-   bytes32(uint256(uint160(to)))
    );
}
```

3.1.27 Pending yield not cleared after claiming

Submitted by 0xEVom, also found by krikolkk, LordofTera and Zach Obront

Severity: High Risk

Context: [BlastNativeYieldManager.sol#L161-L179](#), [BlastNativeYieldManager.sol#L327-L343](#)

Description: The `BlastNativeYieldManager` contract keeps track of pending WETH and USDB yield for each token in the `pendingWETHYield` and `pendingUSDBYield` mappings. This pending yield can be claimed by tokens via the `claimYieldForGauge()` and `claimYieldForAutoCompounding()` functions.

However, the pending yield amounts are never cleared or reset to zero after being claimed in these functions. Specifically, the pending yield is added to the `WETHYield` and `USDBYield` variables to be claimed, but the `pendingWETHYield` and `pendingUSDBYield` mappings are not updated.

This means that any pending yield will be claimable again in future calls to `claimYieldForGauge()` and `claimYieldForAutoCompounding()`, leading to the same pending yield being claimed multiple times. This will lead to tokens claiming far more yield than they should be entitled to.

Recommendation: After adding any pending yield to the `WETHYield` and `USDBYield` variables in `claimYieldForGauge()` and `claimYieldForAutoCompounding()`, the pending yield for that token should be cleared. This can be done by setting the token's pending yield to zero in the mappings:

```
pendingWETHYield[msg.sender] = 0;
pendingUSDBYield[msg.sender] = 0;
```

This ensures that pending yield can only be claimed once, preventing tokens from claiming excess yield.

3.1.28 Improper Setting of `rewardsPerCVE` in `ProtocolMessagingHub::receiveWormholeMessages`

Submitted by KupiaSec

Severity: High Risk

Context: [ProtocolMessagingHub.sol#L179](#)

Description: The `rewardsPerCVE` variable dictates the rewards assigned to each escrowed CVE for the subsequent reward epoch.

In `_executeEpochFeeRouter` function `rewardsPerCVE` is calculated at L827:

```
// FeeAccumulator.sol
824: feeTokenBalanceForChain =
825:     (((feeTokenBalance * WAD) / totalLockedTokens) * lockedTokens) /
826:     WAD;
827:     uint256 epochRewardsPerCVE = (feeTokenBalance * WAD) / //@audit-info calculate epoch reward per
    ↪ cve
828:     totalLockedTokens;
```

`rewardsPerCVE` represents a ratio between the `feeTokenBalance` and `totalLockedTokens`. It signifies the rewards allocated to one vote-escrowed CVE for the forthcoming reward epoch.

However, in the `ProtocolMessagingHub::receiveWormholeMessages` function, at L179, the `rewardsPerCVE` variable is erroneously configured:

```
// ProtocolMessagingHub.sol
97: function receiveWormholeMessages(
98:     bytes memory payload,
99:     bytes[] memory /* additionalMessages */,
100:     bytes32 srcAddress,
101:     uint16 srcChainId,
102:     bytes32 deliveryHash
103: ) external payable {
104:     _checkMessagingHubStatus();
105:
    //[lines of code]
159:     if (token == feeToken) {
160:         ICVELocker locker = ICVELocker(centralRegistry.cveLocker());
161:
162:         // If the locker is shutdown, transfer fees to DAO
163:         // instead of recording epoch rewards.
164:         if (locker.isShutdown() == 2) {
165:             SafeTransferLib.safeTransfer(
166:                 feeToken,
167:                 centralRegistry.daoAddress(),
168:                 amount
169:             );
170:             return;
171:         }
172:
173:         // Transfer fees to locker and record newest epoch rewards.
174:         SafeTransferLib.safeTransfer(
175:             feeToken,
176:             address(locker),
177:             amount
178:         );
179:         locker.recordEpochRewards(amount); //@audit-issue epoch reward should not be amount
180:         return;
181:     }
    //[lines of code]
283: }
```

This oversight enables certain users to claim more rewards than their entitlement, resulting in a deficit of rewards for other users. Consequently, this discrepancy hinders affected users from claiming their fair share of rewards.

The `rewardsPerCVE` value is incorrectly set too high, causing first claimers to get more rewards than intended, depleting the available rewards and leaving subsequent claimers with nothing to claim.

The likelihood is HIGH due to the incorrect logic applied in setting the epoch rewards.

Proof of concept:

- Upon arrival of a Wormhole message, the `ProtocolMessagingHub::receiveWormholeMessages` function is triggered.
- The function transfers the specified amount of `feeToken` to the locker, simultaneously setting the `rewardsPerCVE` equal to this amount.
- As a consequence, initial claimers receive rewards exceeding the intended allocation, depleting the available rewards and consequently leaving subsequent claimers with nothing to claim.

Recommendation: Correct the function to accurately set rewardsPerCVE, rather than the amount of fee-Token.

3.2 Medium Risk

3.2.1 Cannot swap and lock rewards into veCVE due to nonReentrant modifier

Submitted by *Haxatron*, also found by *AuditorPraise*, *deadrosesxyz*, *yttriumzz*, *Bernd Artmüller*, *alix40*, *glcanvas* and *KupiaSec*

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: Curvature allows users to specify a rewardsData struct which allows users to choose whether to receive rewards or swap them into CVE and lock as veCVE when calling createLock.

- [VeCVE.sol#L285-L302](#):

```
function createLock(
    uint256 amount,
    bool continuousLock,
    RewardsData calldata rewardsData,
    bytes calldata params,
    uint256 aux
) external nonReentrant {
    _canLock(amount);

    SafeTransferLib.safeTransferFrom(
        cve,
        msg.sender,
        address(this),
        amount
    );

    // Claim any pending locker rewards.
    _claimRewards(msg.sender, rewardsData, params, aux);
}
```

Which will eventually call _lockRewardsAsVeCVE if user wants to lock their rewards.

- [CVELocker.sol#L584-L636](#)

```
function _lockRewardsAsVeCVE(
    address user,
    bool isFreshLock,
    bool continuousLock,
    uint256 lockIndex
) internal returns (uint256) {
    uint256 lockAmount = IERC20(cve).balanceOf(address(this));

    IERC20(cve).approve(address(veCVE), lockAmount);

    // Because this call is nested within call to claim all rewards
    // there will never be any rewards to process,
    // and thus no potential secondary lock so we can just pass
    // empty reward data to the veCVE calls.
    if (isFreshLock) {
        veCVE.createLockFor(
            user,
            lockAmount,
            continuousLock,
            RewardsData({
                asCVE: false,
                shouldLock: false,
                isFreshLock: false,
                isFreshLockContinuous: false
            }),
            "",
            0
        );
    }

    return lockAmount;
}

// Because this call is nested within call to claim all rewards
```

```

// there will never be any rewards to process,
// and thus no potential secondary lock so we can just pass
// empty reward data to the veCVE calls.
veCVE.increaseAmountAndExtendLockFor(
    user,
    lockAmount,
    lockIndex,
    continuousLock,
    RewardsData({
        asCVE: false,
        shouldLock: false,
        isFreshLock: false,
        isFreshLockContinuous: false
    }),
    "",
    0
);

return lockAmount;
}

```

Issue is that both `increaseAmountAndExtendLockFor` and `createLockFor` use `nonReentrant` modifier.

[VeCVE.sol#L460-L468](#)

```

```solidity
function increaseAmountAndExtendLockFor(
 address recipient,
 uint256 amount,
 uint256 lockIndex,
 bool continuousLock,
 RewardsData calldata rewardsData,
 bytes calldata params,
 uint256 aux
) external nonReentrant {
 ...

```

- [VeCVE.sol#L317-L324](#)

```

function createLockFor(
 address recipient,
 uint256 amount,
 bool continuousLock,
 RewardsData calldata rewardsData,
 bytes calldata params,
 uint256 aux
) external nonReentrant {

```

Solady's `ReentrancyGuard` uses the common lock-based approach where a lock variable is used to determine whether a `nonReentrant` function is still executing.

- [ReentrancyGuard.sol#L28-L42](#)

```

modifier nonReentrant() virtual {
 /// @solidity memory-safe-assembly
 assembly {
 if eq(sload(_REENTRANCY_GUARD_SLOT), 2) {
 mstore(0x00, 0xab143c06) // `Reentrancy()`.
 revert(0x1c, 0x04)
 }
 sstore(_REENTRANCY_GUARD_SLOT, 2)
 }
 _;
 /// @solidity memory-safe-assembly
 assembly {
 sstore(_REENTRANCY_GUARD_SLOT, 1)
 }
}

```

Therefore the swapping and lock rewards into `veCVE` fail because `createLock` is also `nonReentrant` and we cannot have two `nonReentrant` functions from the same contract executing at the same time.

### 3.2.2 Possible for \$800 USDC precision loss in FeeAccumulator

Submitted by *Haxatron*

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** WAD precision is used to calculate epochRewardsPerCVE.

- [FeeAccumulator.sol#L827-L828](#)

```
uint256 epochRewardsPerCVE = (feeTokenBalance * WAD) /
totalLockedTokens;
```

But precision loss can be around \$800 USDC per epoch based on max total supply of CVE.

Max total supply is 420 million CVE, with the 2x multiplier from continuous lock the max value of totalLockedTokens would be 840 million CVE  $> 8 \times 10^{26}$  CVE (because 1 CVE is 18 decimals). Because of this precision loss of reward token can go up to  $8 \times 10^{26}$ .

We know feeToken is USDC at 6 decimals, then the max precision loss is  $8 \times 10^{26} / 10^6 \times \text{WAD} = 800$  USDC.

Therefore every epoch (2 weeks), as much as 800 USDC can be locked in the FeeAccumulator although they will still be included in the balance to be sent on the next epoch.

However this goes against the following:

Are there contracts which through reasonable use could suffer from stuck funds?

**Recommendation:** Use RAY (1e27) precision.

### 3.2.3 Curve2Pool asset pricing is manipulatable, even with reentrancy checks

Submitted by *n1punp*

**Severity:** Medium Risk

**Context:** [Curve2PoolAssetAdaptor.sol#L145-L158](#)

**Description:** The current Curve asset pricing uses the spot token balances to compute the asset price (by sampling the swap of 1/100 the token balances), with an additional check for reentrancy. However, this check can be insufficient, with the emergency of both Flashbots and L2s.

In Flashbots, the attacker can bid up and bribe validators to take control of consecutive txs in batch (or consecutive blocks if needed). In this case, the attacker can pack a sequence of sandwich attack around the attack tx that consumes the price. And so the attack tx will not see the spot price change.

In L2s, it is also possible to imitate the above behavior, since most of the Sequencers operate in a FIFO basis. This means the attacker can simply send a sequence of txs in series to perform the attack, with very low chances of getting interfered.

This means that the current check may not prove to be sufficient for evaluating a fair Curve2Pool asset price. Even though this requires the attacker to have upfront capital for the attack, we can historically see that attackers now are willing to put their capital at risk to perform the attack over multiple blocks. (see [this article](#) on Rodeo Finance's exploit where the attacker manipulates the price around the target transaction).

The attack vector can simply be a simple sandwich attack bundled together in a series of txs that manipulates the token balances, making the sample swap output amount out be manipulated as high as the attacker wants. This will mean the asset token price will be over-estimated, and the attacker can put this asset as collateral and borrow other assets of higher value.

This manipulatable pricing can then lead to a bad debt in the system.

**Recommendation:** Do not use spot price, even if the reentrancy check is enforced, since it can still be bypassed.



### 3.2.4 No slippage control for harvest function (when compounding back to the LP) in AerodromeStableCToken.sol & volatile

Submitted by *n1punp*

**Severity:** Medium Risk

**Context:** AerodromeStableCToken.sol#L203-L233

**Description:** No slippage control for harvest function (when compounding back to the LP) in AerodromeStableCToken.sol & AerodromeVolatileCToken.sol (and velodrome contracts as well).

The harvest function first swaps the reward token (AERO) using swapData (to token0) in L180. Then it tries to deposit single-side using token0 via 3-steps as:

1. Calculate the optimal deposit swap amount.
2. Perform swap from token0 → token1, and...
3. Add liquidity to the pool.

However, this single-side deposit is prone to MEV sandwich attack. The attacker can first make the pool imbalance (more token0, less token1). Then the step 1) that calculates the deposit optimal deposit swap amount will return minimal swap amount (since the pool is already imbalanced). Step 2 then executes this minimal swap amount, and step 3 will provide large number of token0 and small token1 to the imbalanced pool → the user can get almost 0 LP tokens back since the pool is very imbalanced and the worth of token0 is really low.

Here is the modified TestAerodromeStableCToken.t.sol:

```
function testUsdcDaiStablePool() public {
 uint256 assets = 100e18;
 deal(address(USDC_DAI), user1, assets);
 deal(address(USDC_DAI), address(this), 42069);

 USDC_DAI.approve(address(cUSDCDAI), 42069);
 marketManager.listToken(address(cUSDCDAI));

 vm.prank(user1);
 USDC_DAI.approve(address(cUSDCDAI), assets);

 // console.log(cUSDCDAI.balanceOf(user1));

 vm.prank(user1);
 cUSDCDAI.deposit(assets, user1);

 assertEq(
 cUSDCDAI.totalAssets(),
 assets + 42069,
 "Total Assets should equal user deposit plus initial mint."
);

 vm.startPrank(gauge.voter());
 deal(address(AERO), gauge.voter(), 10e18);
 AERO.approve(address(gauge), 10e18);
 gauge.notifyRewardAmount(10e18);
 vm.stopPrank();

 // Advance time to earn CRV and CVX rewards
 vm.warp(block.timestamp + 1 days);

 // Mint some extra rewards for Vault.
 uint256 earned = gauge.earned(address(cUSDCDAI));
 uint256 amount = (earned * 84) / 100;
 SwapperLib.Swap memory swapData;
 swapData.inputToken = address(AERO);
 swapData.inputAmount = amount;
 swapData.outputToken = address(DAI);
 swapData.target = address(veloRouter);
 IVeloRouter.Route[] memory routes = new IVeloRouter.Route[](1);
 routes[0].from = address(AERO);
 routes[0].to = address(DAI);
 routes[0].stable = false;
 routes[0].factory = address(veloPairFactory);
 swapData.call = abi.encodeWithSelector(
```

```

 IVeloRouter.swapExactTokensForTokens.selector,
 amount,
 0,
 routes,
 address(cUSDCDAI),
 type(uint256).max
);

 // begin attack
 console.log(IERC20(cUSDCDAI.asset()).totalSupply()); // print pre-balance

 address hacker = address(0x1234);

 AerodromeStableCToken.StrategyData memory sd;
 (
 sd.gauge,
 sd.pairFactory,
 sd.router,
 sd.token0,
 sd.token1,
 sd.decimalsA,
 sd.decimalsB
) = cUSDCDAI.strategyData();

 uint amt0 = IERC20(sd.token0).balanceOf(cUSDCDAI.asset()) * 10; // 10x the pool token1
 console.log(
 IERC20(sd.token0).balanceOf(cUSDCDAI.asset()),
 IERC20(sd.token1).balanceOf(cUSDCDAI.asset())
);
 deal(sd.token0, hacker, amt0);

 vm.prank(hacker);
 IERC20(sd.token0).approve(address(sd.router), amt0);

 // 1. make pool imbalance (more token0)
 IVeloRouter.Route[] memory routes1 = new IVeloRouter.Route[](1);
 routes1[0].from = sd.token0;
 routes1[0].to = sd.token1;
 routes1[0].stable = true;
 routes1[0].factory = IVeloPool(cUSDCDAI.asset()).factory();
 vm.prank(hacker);
 uint256[] memory amountsOut = IVeloRouter(sd.router)
 .swapExactTokensForTokens(
 amt0,
 0,
 routes1,
 hacker,
 block.timestamp
);
 uint amt1 = amountsOut[amountsOut.length - 1];

 console.log(
 IERC20(sd.token0).balanceOf(cUSDCDAI.asset()),
 IERC20(sd.token1).balanceOf(cUSDCDAI.asset())
);

 // 2. let the user harvest rewards
 cUSDCDAI.harvest(abi.encode(swapData));

 console.log(
 IERC20(sd.token0).balanceOf(cUSDCDAI.asset()),
 IERC20(sd.token1).balanceOf(cUSDCDAI.asset())
);

 // 3. make the pool balance again
 vm.prank(hacker);
 IERC20(sd.token1).approve(address(sd.router), amt1);

 routes1[0].from = sd.token1;
 routes1[0].to = sd.token0;
 routes1[0].stable = true;
 routes1[0].factory = IVeloPool(cUSDCDAI.asset()).factory();
 vm.prank(hacker);
 IVeloRouter(sd.router).swapExactTokensForTokens(
 amt1,
 0,

```

```

 routes1,
 hacker,
 block.timestamp
);

 console.log(
 IERC20(sd.token0).balanceOf(cUSDCDAI.asset()),
 IERC20(sd.token1).balanceOf(cUSDCDAI.asset())
);

 console.log(IERC20(cUSDCDAI.asset()).totalSupply()); // print post-balance

 assertEq(
 cUSDCDAI.totalAssets(),
 assets + 42069,
 "Total Assets should equal user deposit plus initial mint."
);

 vm.warp(block.timestamp + 8 days);

 // Mint some extra rewards for Vault.
 earned = gauge.earned(address(cUSDCDAI));
 amount = (earned * 84) / 100;
 swapData.inputAmount = amount;
 swapData.call = abi.encodeWithSelector(
 IVeloRouter.swapExactTokensForTokens.selector,
 amount,
 0,
 routes,
 address(cUSDCDAI),
 type(uint256).max
);
 cUSDCDAI.harvest(abi.encode(swapData));

 vm.warp(block.timestamp + 7 days);

 assertGt(
 cUSDCDAI.totalAssets(),
 assets + 42069,
 "Total Assets should greater than original deposit plus initial mint."
);

 vm.prank(user1);
 cUSDCDAI.withdraw(assets, user1, user1);
}

```

The above performs step 1) and 3) that sandwiches the harvest step 2 (NOTE: the hacker can still utilize Flashbots to bundle txs together, even if the caller is access-controlled).

The result from the above test is: The LP token only increments from 61389621417353898 → 61389627736242604 ( $\Delta = 6318888706$ )

Instead of the original (with no attack): 61389621417353898 → 61389671030534817 ( $\Delta = 49613180919$ ).

The manipulated harvest only gives  $6318888706 / 49613180919 = 12\%$  of the real amount. The attack can drain even more if the swap amount is set correctly.

This means that the auto-compounding feature will get MEV'd and will lose all the reward tokens due to no proper slippage control in the optimal deposit part.

*Note: This issue also applies to `AerodromeVolatileCToken.sol`, and `VelodromeStableCToken.sol` and `VelodromeVolatileCToken.sol` as well.*

**Recommendations:** Add slippage control value from off-chain, or add oracle price checks to the optimal deposit logic.

### 3.2.5 Excess ETH when BorrowZapper.sol#borrowAndBridge() function is called not refunded

Submitted by [GeneralKay](#)

**Severity:** Medium Risk

**Context:** `BorrowZapper.sol`#L74-L79, `FeeTokenBridgingHub.sol`#L157, `FeeTokenBridgingHub.sol`#L165

**Description:** The `BorrowZapper.sol#borrowAndBridge()` function is payable and requires ETH for bridging through the highlighted `_sendFeeToken()` function of the parent `FeeTokenBridgingHub.sol` contract.

The issue is that ETH is sent along the `borrowAndBridge()` function call can be more than the required `wormholeFee` passed to `wormholeRelayer.sendToEvm{ value: wormholeFee }`.

When Excess ETH is sent with `borrowAndBridge()` function call, users are not refunded the extra ETH.

**Impact:** Excess ETH sent with `borrowAndBridge()` function call will be lost to the `BorrowZapper.sol` contract.

**Reference:** See how `msg.value` is validated against cost in the wormhole [docs](#).

**Recommendation:** Consider Implementing any of the following:

1. Refund Excess ETH when `msg.value` is more than `wormholeFee`, or
2. Ensure `mgs.value == wormholeFee` this way.

```
// File: BorrowZapper.sol

function borrowAndBridge(
 address dToken,
 uint256 borrowAmount,
 SwapperLib.Swap memory swapData,
 uint256 dstChainId
) external payable nonReentrant {
 uint256 balancePrior = IERC20(feeToken).balanceOf(address(this));

 // Borrow on behalf of caller.
 DToken(dToken).borrowFor(msg.sender, address(this), borrowAmount);

 address underlying = DToken(dToken).underlying();

 // Check if swapping is necessary.
 if (underlying != feeToken) {
 if (
 swapData.target == address(0) ||
 swapData.inputToken != underlying ||
 swapData.outputToken != feeToken ||
 swapData.inputAmount != borrowAmount
) {
 revert BorrowZapper__InvalidSwapData();
 }

 // Validate target contract is an approved swapper.
 if (!centralRegistry.isSwapper(swapData.target)) {
 revert BorrowZapper__InvalidSwapper(swapData.target);
 }
 unchecked {
 SwapperLib.swap(centralRegistry, swapData);
 }
 } else {
 if (swapData.target != address(0)) {
 revert BorrowZapper__InvalidSwapData();
 }
 }
}

++ uint256 wormholeFee = _quoteWormholeFee(dstChainId, true);
++ require(msg.value == wormholeFee, "More ETH than required");

 // Bridge the fee token to `dstChainId` via Wormhole.
 _sendFeeToken(
 dstChainId,
 msg.sender,
 IERC20(feeToken).balanceOf(address(this)) - balancePrior
);
}
```

### 3.2.6 GMX callback contract does not handle deposit cancelation

Submitted by [GeneralKay](#)

**Severity:** Medium Risk

### Context: GMCToken.sol#L179

**Description:** GMCToken.sol#L179 passes address(this) as the callback contract when forming the createDeposit(...) params. GMX expects that the callback function which is the GMCToken.sol to implement the afterDepositExecution(...) and afterDepositCancellation(...) callback functions so that GMX will call the functions for successful deposit and cancelled deposit respectively.

However the GMCToken.sol only implement the afterDepositExecution(...) callback function and not the afterDepositCancellation(...) callback function. This shows the assumption that deposit will never be cancelled which is not true because the GMX docs state that deposit can be cancelled for a lot of reasons.

According to the [GMX deposit notes](#):

- Handle deposit cancellations

When createDeposit(...) is cancelled, GMX tries to call the afterDepositCancellation(...) on the GMCToken.sol contract but since it is not available, the transaction will revert causing the deposit to be stuck.

[GMX Deposit notes](#) recommends that integration should handle deposit cancellation.

**Impact:** Loss of createDeposit asset when deposit is cancelled and blocking deposit to GMX.

**Reference:** [GMX deposit notes](#).

**Recommendation:** Implement the afterDepositCancellation(...) as the GMX docs recommend to handle deposit cancellation.

### 3.2.7 Malicious users, when given allowance, might drain funds from owners

Submitted by [tvdung94](#), also found by [n1punp](#), [krikolkk](#) and [KupiaSec](#)

**Severity:** Medium Risk

**Context:** CTokenCompounding.sol#L382

**Description:** The amount of allowance spent in this \_withdraw() is incorrect. Because of that, malicious users, when given allowance from owners, could withdraw entire owners' balances instead of expected amount.

- Wrong amount of allowance to spend:

```
_spendAllowance(owner, msg.sender, allowed - shares);

function _spendAllowance(address owner, address spender, uint256 amount) internal virtual {
 /// @solidity memory-safe-assembly
 assembly {
 // Compute the allowance slot and load its value.
 mstore(0x20, spender)
 mstore(0x0c, _ALLOWANCE_SLOT_SEED)
 mstore(0x00, owner)
 let allowanceSlot := keccak256(0x0c, 0x34)
 let allowance_ := sload(allowanceSlot)
 // If the allowance is not the maximum uint256 value.
 if add(allowance_, 1) {
 // Revert if the amount to be transferred exceeds the allowance.
 if gt(amount, allowance_) {
 mstore(0x00, 0x13be252b) // `InsufficientAllowance()`.
 revert(0x1c, 0x04)
 }
 // Subtract and store the updated allowance.
 sstore(allowanceSlot, sub(allowance_, amount))
 }
 }
}
```

Consider this scenario:

1. An owner gives an attacker allowance to withdraw 10 shares. (Allowed = 10).
2. An attacker withdraws 10 shares (shares = 10). Because allowed - shares = 10 - 10 = 0, allowance amount is not reduced at all.

3. The attacker could just withdraw again. Repeat until the owner's balance is depleted.

**Recommendation:** Consider this change:

```
_spendAllowance(owner, msg.sender, shares);
```

### 3.2.8 Incorrect interface definition makes contract undeployable

Submitted by *krikolkk*, also found by *OxEVom*

**Severity:** Medium Risk

**Context:** [ICentralRegistry.sol#L132-L133](#)

**Description:** The method `marketManagers` is defined incorrectly inside the `ICentralRegistry` interface. The field `marketManagers` of the `CentralRegistry` contract, which is marked as `public`, thus generating the `marketManagers` view function, is of type `address[]`, meaning that the generated method will have the signature `marketManagers(uint)` and the return type would be `address`, therefore any calls to `marketManagers` using `ICentralRegistry` will revert.

According to the function's natspec, the expected behavior is for it to return the array of all Curvance market managers on the specific chain. However, this natspec also seems incorrect and is just a copy of the `marketManagers` field natspec.

This function is currently called only in `BlastNativeYieldManager`'s constructor, making the contract creation fail and revert without reason, making it undeployable.

The **likelihood** of this happening is **high** as it WILL happen, and the contract will be undeployable. Also, if there were some use cases in the future leveraging this interface, it would assume the correctness of this function. The **impact** of this issue is **low** as no funds are at risk.

**Proof of concept:**

```
// SPDX-License-Identifier: UNLICENSED

pragma solidity ^0.8.17;

import { TestBaseMarket } from "tests/market/TestBaseMarket.sol";
import { BlastNativeYieldManager } from "contracts/architecture/blastNative/BlastNativeYieldManager.sol";
import { BlastCentralRegistry } from "contracts/architecture/blastNative/BlastCentralRegistry.sol";
import { IBlastCentralRegistry } from "contracts/interfaces/blast/IBlastCentralRegistry.sol";
import "forge-std/console.sol";

contract TestBlastNativeYieldManager is TestBaseMarket {
 function test_DeployBlastNativeYieldMangerRevertsWithoutReason() public {
 uint nonce = vm.getNonce(msg.sender);
 address expectedAddress = computeCreateAddress(msg.sender, nonce + 1);
 vm.startPrank(msg.sender);

 BlastCentralRegistry registry = new BlastCentralRegistry(
 _ZERO_ADDRESS,
 _ZERO_ADDRESS,
 _ZERO_ADDRESS,
 block.timestamp,
 address(0),
 _USDC_ADDRESS,
 expectedAddress
);

 new BlastNativeYieldManager(IBlastCentralRegistry(address(registry)));

 vm.stopPrank();
 }
}
```

The test above fails with the following output:

```
Running 1 test for tests/blastNative/TestBlastNativeYieldManager.sol:TestBlastNativeYieldManager
[FAIL. Reason: EvmError: Revert] test_DeployBlastNativeYieldMangerRevertsWithoutReason() (gas: 2881410)
Test result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 5.75ms
```

**Recommendation:** Change the function definition in the `ICentralRegistry` interface to the correct one

```

- /// @notice Address array for all Curvance Market Managers on this chain.
- function marketManagers() external view returns (address[] memory);
+ /// @notice Address of a Curvance Market Manager on index `index`
+ function marketManagers(uint256 index) external view returns (address);

```

To fulfill the need of the BlastNativeYieldManager contract, consider adding another function to the CentralRegistry contract that would return the marketManagers length:

```

+ function marketManagersLength() external view virtual returns (uint256) {
+ return marketManagers.length;
+ }

```

Also, include the new function in the ICentralRegistry interface:

```

+ function marketManagersLength() external view returns (uint256);

```

After that we can update the BlastNativeYieldManager contract constructor like this:

```

constructor(IBlastCentralRegistry centralRegistry_) {
 if (
 !ERC165Checker.supportsInterface(
 address(centralRegistry_),
 type(IBlastCentralRegistry).interfaceId
)
) {
 revert BlastNativeYieldManager__InvalidCentralRegistry();
 }

 centralRegistry = centralRegistry_;

- address[] memory marketManagers = centralRegistry_.marketManagers();
- uint256 numMarkets = marketManagers.length;
+ uint256 numMarkets = centralRegistry_.marketManagersLength();
+ address marketManager;

 // Register any previously configured markets here, if any.
 for (uint256 i; i < numMarkets;) {
- isMarketManager[marketManagers[i++]] = true;
+ marketManager = centralRegistry_.marketManagers(i++);
+ isMarketManager[marketManager] = true;
 }

 // Set gas fees yield to claimable and then pass Governor
 // permissioning to Curvance DAO.
 CHAIN_YIELD_MANAGER.configureClaimableGas();
 CHAIN_YIELD_MANAGER.configureGovernor(centralRegistry_.daoAddress());
}

```

### 3.2.9 CTokenCompoundingWithExitFee.sol is not EIP-4626 compliant

Submitted by [alix40](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Impact:** All EIP-4626 standard compliant vaults **must** include any fees in the previewWithdraw() or previewRedeem() Not complying with a must statement in the EIP-4626 standard could result in errors and problems when integrating with other defi protocols.

**Proof of concept:** The CTokenCompoundingWithExitFee.sol contract adds a fee on undelying tokens withdrawals. The contract however inherits and uses the same preview function from CTokenBase.sol, which doesn't include any fees on withdrawals.

```
function _previewWithdraw(
 uint256 assets,
 uint256 ta
) internal view returns (uint256 shares) {
 uint256 totalShares = totalSupply();

 shares = totalShares == 0
 ? assets
 : FixedPointMathLib.mulDivUp(assets, totalShares, ta);
}
```

As we can see the function `_previewWithdraw()` used in the `previewRedeem()` and `previewWithdraw()` functions, doesn't include any fees.

The EIP-4626 states the following concerning the `previewWithdraw()` and `previewRedeem()` functions [link](#)  
 MUST be inclusive of withdrawal fees. Integrators should be aware of the existence of withdrawal fees.

According to the standard it is **necessary** to include any withdrawal fees in the `previewRedeem()` and `previewWithdraw()` functions

**Recommendation:** It is recommended to override `previewRedeem()` and `previewWithdraw()` functions in the `CTokenCompoundingWithExitFee.sol` contract and to include the withdrawal fee, in the computations.

### 3.2.10 Deleveraging position on CTokenCompoundingWithExitFee would always revert

Submitted by [alix40](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Impact:** The contract `CTokenCompoundingWithExitFee` inherits the function `withdrawByPositionFolding()` from the `CTokenCompounding` contract. The inherited function however doesn't include the exit fee when interacting with the `PositionFolding` contract, which would lead in the deleveraging transaction reverting.

**Proof of concept:** The inherited function `withdrawByPositionFolding()` from the `CTokenCompounding` contract does not include the exit fee, which would result in the `PositionFolding` contract receiving less assets than the number passed in the `onRedeem()` callback, which would then revert.

```
function withdrawByPositionFolding(
 address owner,
 uint256 assets,
 bytes calldata params
) external nonReentrant {
 // Validate that the position folding contract is calling.
 if (msg.sender != marketManager.positionFolding()) {
 _revert(_UNAUTHORIZED_SELECTOR);
 }

 // Cache pendingRewards, _totalAssets, balanceOf.
 uint256 pending = _calculatePendingRewards();
 uint256 ta = _totalAssets + pending;
 uint256 balancePrior = balanceOf(owner);

 // We use a modified version of maxWithdraw with newly vested assets.
 if (assets > _convertToAssets(balancePrior, ta)) {
 // revert with "CTokenCompounding_WithdrawMoreThanMax".
 _revert(0x2735eaab);
 }

 // No need to check for rounding error, previewWithdraw rounds up.
 uint256 shares = _previewWithdraw(assets, ta);

 // Update gauge pool values for `owner`.
 _gaugePool().withdraw(address(this), owner, shares);
 // @audit-issue on the CTokenCompoundingWithExitFee, processWithdraw is overridden, and will withdraw assets -> exit fee
 _processWithdraw(
 msg.sender,
 msg.sender,
```



```

 owner,
 assets,
 shares,
 ta,
 pending
);
 // @audit onRedeem receive a false amount (exit fee not included), which would lead in the transaction to
 // ↪ the PositionFolding contract to revert
 // Callback to PositionFolding that executes cToken specific logic.
 IPositionFolding(msg.sender).onRedeem(
 address(this),
 owner,
 assets,
 params
);
 // ...

```

As a proof of concept, to prove that deleveraging on CTokenCompoundingWithExitFee is not possible, we forked the testsuite tests/market/position-folding/integrations/TestPositionFolding.t.sol and modified it to use cBALRETHWithExitFee instead of cBALRETH.

When running the PoC, we will see that leveraging on the cBALRETHWithExitFee will work as expected (as it deposits funds in the cToken and doesn't Withdraw), while deleveraging on the cBALRETH will revert with the message [FAIL. Reason: PositionFolding\_\_InvalidAmount()].

To run the proof of concept, please create this file tests/market/position-folding/integrations/TestPositionFoldingWithExitFee.t.sol:

### Proof of concept:

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.17;

import { MockDataFeed } from "contracts/mocks/MockDataFeed.sol";
import { IMToken } from "contracts/interfaces/market/IMToken.sol";
import { IUniswapV2Router } from "contracts/interfaces/external/uniswap/IUniswapV2Router.sol";
import { SwapperLib } from "contracts/libraries/SwapperLib.sol";
import { CTokenPrimitive } from "contracts/market/collateral/CTokenPrimitive.sol";
import { MockCallDataChecker } from "contracts/mocks/MockCallDataChecker.sol";
import "forge-std/console.sol";
import "tests/market/TestBaseMarket.sol";

contract User {}

contract TestPositionFoldingWithExitFee is TestBaseMarket {
 address internal constant _UNISWAP_V2_ROUTER =
 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;

 address public owner;
 address public user;
 MockDataFeed public mockUsdcFeed;
 MockDataFeed public mockDaiFeed;
 MockDataFeed public mockWethFeed;
 MockDataFeed public mockRethFeed;

 receive() external payable {}

 fallback() external payable {}

 function setUp() public override {
 super.setUp();

 owner = address(this);
 user = user1;

 // use mock pricing for testing
 mockUsdcFeed = new MockDataFeed(_CHAINLINK_USDC_USD);
 chainlinkAdaptor.addAsset(
 _USDC_ADDRESS,
 address(mockUsdcFeed),
 0,
 true
);
 dualChainlinkAdaptor.addAsset(
 _USDC_ADDRESS,

```

```

 address(mockUsdcFeed),
 0,
 true
);
 mockDaiFeed = new MockDataFeed(_CHAINLINK_DAI_USD);
 chainlinkAdaptor.addAsset(_DAI_ADDRESS, address(mockDaiFeed), 0, true);
 dualChainlinkAdaptor.addAsset(
 _DAI_ADDRESS,
 address(mockDaiFeed),
 0,
 true
);
 mockWethFeed = new MockDataFeed(_CHAINLINK_ETH_USD);
 chainlinkAdaptor.addAsset(
 _WETH_ADDRESS,
 address(mockWethFeed),
 0,
 true
);
 dualChainlinkAdaptor.addAsset(
 _WETH_ADDRESS,
 address(mockWethFeed),
 0,
 true
);
 mockRethFeed = new MockDataFeed(_CHAINLINK_RETH_ETH);
 chainlinkAdaptor.addAsset(
 _RETH_ADDRESS,
 address(mockRethFeed),
 0,
 false
);
 dualChainlinkAdaptor.addAsset(
 _RETH_ADDRESS,
 address(mockRethFeed),
 0,
 false
);
 _prepareUSDC(user, 200000e6);
 _prepareDAI(user, 200000e18);
 _prepareBALRETH(user, 1 ether);

 // start epoch
 gaugePool.start(address(marketManager));
 vm.warp(gaugePool.startTime());
 vm.roll(block.number + 1000);

 mockUsdcFeed.setMockUpdatedAt(block.timestamp);
 mockDaiFeed.setMockUpdatedAt(block.timestamp);
 mockWethFeed.setMockUpdatedAt(block.timestamp);
 mockRethFeed.setMockUpdatedAt(block.timestamp);

 (, int256 ethPrice, ,) = mockWethFeed.latestRoundData();
 chainlinkEthUsd.updateAnswer(ethPrice);

 // setup dDAI
 {
 _prepareDAI(owner, 200000e18);
 dai.approve(address(dDAI), 200000e18);
 marketManager.listToken(address(dDAI));
 // add MToken support on price router
 oracleRouter.addMTokenSupport(address(dDAI));
 }

 // setup cBALRETHWithExitFee
 {
 // support market
 _prepareBALRETH(owner, 1 ether);
 balRETH.approve(address(cBALRETHWithExitFee), 1 ether);
 marketManager.listToken(address(cBALRETHWithExitFee));
 // set collateral factor
 marketManager.updateCollateralToken(
 IMToken(address(cBALRETHWithExitFee)),
 7000,
 4000,

```

```

 3000,
 200,
 400,
 10,
 1000
);
 address[] memory tokens = new address[](1);
 tokens[0] = address(cBALRETHWithExitFee);
 uint256[] memory caps = new uint256[](1);
 caps[0] = 100_000e18;
 marketManager.setCTokenCollateralCaps(tokens, caps);
}

// set position folding
MarketManager(marketManager).setPositionFolding(
 address(positionFolding)
);

// vm.warp(gaugePool.startTime());
// vm.roll(block.number + 1000);

// // set gauge settings of next epoch
// address[] memory tokensParam = new address[](2);
// tokensParam[0] = address(dDAI);
// tokensParam[1] = address(cBALRETHWithExitFee);
// uint256[] memory poolWeights = new uint256[](2);
// poolWeights[0] = 100;
// poolWeights[1] = 100;
// vm.prank(protocolMessagingHub);
// gaugePool.setEmissionRates(1, tokensParam, poolWeights);
// vm.prank(protocolMessagingHub);
// cue.mintGaugeEmissions(300 * 2 weeks, address(gaugePool));
// vm.warp(gaugePool.startTime() + 1 * 2 weeks);

// provide enough liquidity for leverage
provideEnoughLiquidityForLeverage();

centralRegistry.addSwapper(_UNISWAP_V2_ROUTER);
centralRegistry.setExternalCallDataChecker(
 _UNISWAP_V2_ROUTER,
 address(new MockCallDataChecker(_UNISWAP_V2_ROUTER))
);
}

function provideEnoughLiquidityForLeverage() internal {
 address liquidityProvider = makeAddr("liquidityProvider");
 _prepareDAI(liquidityProvider, 200000e18);
 _prepareBALRETH(liquidityProvider, 10 ether);
 // mint dDAI
 vm.startPrank(liquidityProvider);
 dai.approve(address(dDAI), 200000 ether);
 dDAI.mint(200000 ether);
 // mint cBALRETH
 balRETH.approve(address(cBALRETHWithExitFee), 10 ether);
 cBALRETHWithExitFee.deposit(10 ether, liquidityProvider);
 vm.stopPrank();
}

function testInitialize() public {
 assertEq(
 address(positionFolding.centralRegistry()),
 address(centralRegistry)
);
 assertEq(
 address(positionFolding.marketManager()),
 address(marketManager)
);
}

function testLeverageWithExitFee() public {
 vm.startPrank(user);

 // approve
 balRETH.approve(address(cBALRETHWithExitFee), 1 ether);

 // mint

```

```

assertGt(cBALRETHWithExitFee.deposit(1 ether, user1), 0);
marketManager.postCollateral(
 user,
 address(cBALRETHWithExitFee),
 1 ether
);
assertEq(cBALRETHWithExitFee.balanceOf(user), 1 ether);

uint256 balanceBeforeBorrow = dai.balanceOf(user);
// borrow
dDAI.borrow(100 ether);
assertEq(balanceBeforeBorrow + 100 ether, dai.balanceOf(user));

// try leverage with 50% of max
uint256 amountForLeverage = (positionFolding
 .queryAmountToBorrowForLeverageMax(user, address(dDAI)) * 50) /
 100;

PositionFolding.LeverageStruct memory leverageData;
leverageData.borrowToken = dDAI;
leverageData.borrowAmount = amountForLeverage;
leverageData.collateralToken = CTokenPrimitive(
 address(cBALRETHWithExitFee)
);
leverageData.swapData.inputToken = address(dai);
leverageData.swapData.inputAmount = amountForLeverage;
leverageData.swapData.outputToken = _WETH_ADDRESS;
leverageData.swapData.target = _UNISWAP_V2_ROUTER;
address[] memory path = new address[](2);
path[0] = address(dai);
path[1] = _WETH_ADDRESS;
leverageData.swapData.call = abi.encodeWithSignature(
 "swapExactTokensForTokens(uint256,uint256,address[],address,uint256)",
 amountForLeverage,
 0,
 path,
 address(positionFolding),
 block.timestamp
);
leverageData.zapperCall.inputToken = _WETH_ADDRESS;
uint256[] memory amountsOut = IUniswapV2Router(_UNISWAP_V2_ROUTER)
 .getAmountsOut(amountForLeverage, path);
leverageData.zapperCall.inputAmount = amountsOut[1];

address[] memory tokens = new address[](2);
tokens[0] = _RETH_ADDRESS;
tokens[1] = _WETH_ADDRESS;
leverageData.zapperCall.target = address(complexZapper);
leverageData.zapperCall.call = abi.encodeWithSelector(
 ComplexZapper.enterBalancer.selector,
 address(0),
 ComplexZapper.ZapperData(
 _WETH_ADDRESS,
 leverageData.zapperCall.inputAmount,
 address(balRETH),
 0,
 false
),
 new SwapperLib.Swap[](0),
 _BALANCER_VAULT,
 _BAL_WETH_RETH_POOLID,
 tokens,
 address(positionFolding)
);

positionFolding.leverage(leverageData, 500);

(uint256 dDAIBalance, uint256 dDAIBorrowed,) = dDAI.getSnapshot(user);
assertEq(dDAIBalance, 0);
assertEq(dDAIBorrowed, 100 ether + amountForLeverage);

(
 uint256 cBALRETHWithExitFeeBalance,
 uint256 cBALRETHWithExitFeeBorrowed,
) = cBALRETHWithExitFee.getSnapshot(user);

```

```

 assertGt(cBALRETHWithExitFeeBalance, 1.5 ether);
 assertEq(cBALRETHWithExitFeeBorrowed, 0 ether);

 vm.stopPrank();
}

function testDeLeverageWithExitFee() public {
 testLeverageWithExitFee();
 // Warp until collateral posting wait time ends
 vm.warp(block.timestamp + 20 minutes);
 dDAI accrueInterest();

 vm.startPrank(user);

 PositionFolding.DeleverageStruct memory deleverageData;

 (, uint256 dDAIBorrowedBefore,) = dDAI.getSnapshot(user);
 (uint256 cBALRETHWithExitFeeBalanceBefore, ,) = cBALRETHWithExitFee
 .getSnapshot(user);

 deleverageData.collateralToken = CTokenPrimitive(
 address(cBALRETHWithExitFee)
);
 deleverageData.collateralAmount = 0.3 ether;
 deleverageData.borrowToken = dDAI;

 deleverageData.zapperCall.inputToken = address(balRETH);
 deleverageData.zapperCall.inputAmount = deleverageData
 .collateralAmount;

 address[] memory tokens = new address[](2);
 tokens[0] = _RETH_ADDRESS;
 tokens[1] = _WETH_ADDRESS;
 deleverageData.zapperCall.target = address(complexZapper);
 deleverageData.zapperCall.call = abi.encodeWithSelector(
 ComplexZapper.exitBalancer.selector,
 ComplexZapper.BPTRedemption(
 _BALANCER_VAULT,
 _BAL_WETH_RETH_POOLID,
 true,
 1
),
 ComplexZapper.ZapperData(
 address(balRETH),
 deleverageData.zapperCall.inputAmount,
 _WETH_ADDRESS,
 0,
 false
),
 tokens,
 new SwapperLib.Swap[](0),
 address(positionFolding)
);

 uint256 amountForDeleverage = 0.3 ether;
 deleverageData.swapData.inputToken = _WETH_ADDRESS;
 deleverageData.swapData.inputAmount = amountForDeleverage;
 deleverageData.swapData.outputToken = address(dai);
 deleverageData.swapData.target = _UNISWAP_V2_ROUTER;
 address[] memory path = new address[](2);
 path[0] = _WETH_ADDRESS;
 path[1] = address(dai);
 deleverageData.swapData.call = abi.encodeWithSignature(
 "swapExactTokensForTokens(uint256,uint256,address[],address,uint256)",
 amountForDeleverage,
 0,
 path,
 address(positionFolding),
 block.timestamp
);
 uint256[] memory amountsOut = IUniswapV2Router(_UNISWAP_V2_ROUTER)
 .getAmountsOut(amountForDeleverage, path);
 deleverageData.repayAmount = amountsOut[1];

 cBALRETHWithExitFee.approve(
 address(positionFolding),

```

```

 type(uint256).max
);
 positionFolding.deleverage(deleverageData, 500);

 (uint256 dDAIBalance, uint256 dDAIBorrowed,) = dDAI.getSnapshot(user);
 assertEq(dDAIBalance, 0);
 assertEq(
 dDAIBorrowed,
 dDAIBorrowedBefore - deleverageData.repayAmount
);

 (
 uint256 cBALRETHWithExitFeeBalance,
 uint256 cBALRETHWithExitFeeBorrowed,

) = cBALRETHWithExitFee.getSnapshot(user);
 assertEq(
 cBALRETHWithExitFeeBalance,
 cBALRETHWithExitFeeBalanceBefore - deleverageData.collateralAmount
);
 assertEq(cBALRETHWithExitFeeBorrowed, 0);

 vm.stopPrank();
}
}

```

**Recommendation:** There are multiple ways to mitigate this issue, the simplest solution would be to override `withdrawByPositionFolding()` and include the exit fee when calling the `onRedeem()` callback.

```

IPositionFolding(msg.sender).onRedeem(
 address(this),
 owner,
-- assets,
++ _removeExitFeeFromAssets(assets)
 params
);

```

### 3.2.11 If a reward token is re-added to GaugePool, a user could drain the contract of it

Submitted by [deadrosesxyz](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** Consider the following scenario:

1. Reward token exists within GaugePool. Some time passes, `accRewardPerShare == 100e36` (it is scaled by `WAD_SQUARED`).
2. Reward token gets removed.
3. User A joins the pool for the first time. Since the reward token is accrued, `_calcDebt` does not calculate `rewardDebt` for that token.
4. Reward token gets re-added again.
5. User can now claim rewards for the entire `accRewardPerShare` as they don't have a `rewardDebt` stored.

**Recommendation:** Upon removing a reward token, clear the `accRewardPerShare` or don't allow for a token to be ever re-added.

### 3.2.12 Unsold CVE within CurvanceDAO LBP will remain stuck

Submitted by [deadrosesxyz](#), also found by [Haxatron](#) and [yttriumzz](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** Within the LBP contract, CVE is sent to be sold. The problem is that in case not all CVE is sold, the rest of it will remain stuck within the contract with no possible way for the DAO to reclaim it.

**Recommendation:** Add a function to reclaim any unsold CVE from within the contract.

### 3.2.13 Convex Booster functions will revert on Arbitrum because Booster functions on L2 (Arbitrum) have a different interface

Submitted by *GeneralKay*

**Severity:** Medium Risk

**Context:** [Convex3PoolCToken.sol#L279](#), [Convex3PoolCToken.sol#L82-L85](#)

**Description:** The issue is that the codebase assumes that all convex booster functions on all chains have the same interface which is false. The differences are stated in the docs [here](#)

For example the `booster.deposit(...)` function on mainnet takes 3 parameters as input while on Arbitrum the same `booster.deposit(...)` takes in just 2 parameters.

Also the `booster.poolInfo(...)` function returns 6 arguments on mainnet while it returns 5 arguments on Arbitrum. Using the same interface for these functions on all networks will cause the functions to revert.

The Convex [docs](#) explains the differences in the booster contract for L2s like Arbitrum.

The `IBooster` interface used in the codebase can only work for the booster contract on Ethereum mainnet.

Curvature will be deployed on Arbitrum in addition to other EVM L2 networks as stated in their [docs](#).

Curvature is a modular multi-chain money market, for yield-bearing assets, and any other ERC-20 tokens. Initially to be launched on Ethereum, Arbitrum, Blast, Base, Optimism, and Polygon zkEVM

The `Convex3PoolCToken.sol`, `AuraCToken.sol` and `Convex2PoolCToken.sol` can not even be deployed to Arbitrum because it uses the mainnet version `poolInfo` function in the constructor of the contracts. This is because it expects 6 return values instead of 5 for Arbitrum. This can be deployed to mainnet but not L2s like Arbitrum. Here is the [PoolInfo struct for the L2 version](#) and here is the [booster contract on Arbitrum](#) to check the function differences.

**Impact:**

- `Convex3PoolCToken.sol`, `AuraCToken.sol` and `Convex2PoolCToken.sol` cannot be deployed on Arbitrum because of the `poolInfo` function in their constructors.
- The `booster.deposit(...)` functions in the above contracts will also revert because it expects just 2 parameters unlike the mainnet version that expects 3.

**Reference:** [Here](#) is a similar issue report in a past contest.

**Recommendation:** Consider implementing a separate implementation of `Convex3PoolCToken.sol`, `AuraCToken.sol` and `Convex2PoolCToken.sol` that adheres to the L2 Booster interface. See the Convex [docs](#).

### 3.2.14 The protocol allows borrowing with small positions in which can result bad debts to protocol

Submitted by *oceansky*, also found by *JCN*, *alix40*, *pks271* and *Bauchibred*

**Severity:** Medium Risk

**Context:** [DToken.sol#L1231](#), [MarketManager.sol#L315](#)

**Description:** The collateral posting and borrow functions has no checking of the required minimum amount to post collateral and borrow, which means anyone can setup a multiple collateral posting and borrowing positions with small amount as long as it is covered by their collateral deposits and caps.

```

File: DToken.sol
1231: function _borrow(
1232: address account,
1233: uint256 amount,
1234: address recipient
1235:) internal {
1236: // Check if we have enough underlying held to support the borrow.
1237: // We add _BASE_UNDERLYING_RESERVE to the calculation to ensure that
1238: // the market never actually runs out of assets and may introduce
1239: // invariant manipulation.
1240: // This also acts as a protective mechanism against trying to
1241: // manipulate totalBorrows above total underlying assets inside
1242: // the system since there will always be at least
1243: // _BASE_UNDERLYING_RESERVE excess inside the market.
1244: if (
1245: marketUnderlyingHeld() - totalReserves <
1246: amount + _BASE_UNDERLYING_RESERVE
1247:) {
1248: revert DToken__InsufficientUnderlyingHeld();
1249: }
1250:
1251: // Calculate current account debt then add `amount`.
1252: // Then update account exchange rate, and total borrow balances.
1253: _debtOf[account].principal = debtBalanceCached(account) + amount;
1254: _debtOf[account].accountExchangeRate = marketData.exchangeRate;
1255: totalBorrows = totalBorrows + amount;
1256:
1257: // Transfer underlying to `recipient`.
1258: SafeTransferLib.safeTransfer(underlying, recipient, amount);
1259:
1260: emit Borrow(account, amount);

```

```

File: MarketManager.sol
309: /// @notice Posts `tokens` of `cToken` as collateral inside this market.
310: /// @dev The collateral token must have collateralization
311: /// enabled (collRatio > 0).
312: /// @param account The account posting collateral.
313: /// @param cToken The address of the cToken to post collateral for.
314: /// @param tokens The amount of `cToken` to post as collateral, in shares.
315: function postCollateral(
316: address account,
317: address cToken,
318: uint256 tokens
319:) external {
320: if (tokens == 0) {
321: _revert(_INVALID_PARAMETER_SELECTOR);
322: }
323:
324: // If they are trying to post collateral for someone else,
325: // make sure it is done via the cToken contract itself.
326: if (msg.sender != account) {
327: _checkIsToken(cToken);
328: }
329:
330: if (!tokenData[cToken].isListed) {
331: _revert(_TOKEN_NOT_LISTED_SELECTOR);
332: }
333:
334: if (!IMToken(cToken).isCToken()) {
335: _revert(_INVALID_PARAMETER_SELECTOR);
336: }
337:
338: AccountPosition storage accountPositions = tokenData[cToken].accountPositions[
339: account
340:];
341:
342: // Precondition invariant check.
343: if (
344: accountPositions.collateralPosted + tokens >
345: IMToken(cToken).balanceOf(account)
346:) {
347: revert MarketManager__InsufficientCollateral();
348: }
349:
350: _postCollateral(account, accountPositions, cToken, tokens);

```



```
351: }
352:
```

The problem with this no minimum collateral posting and borrowing amount is in the liquidation part in which the liquidator may not be incentivized to liquidate the account when it is necessary because of the very low amount to be recovered from collateral compared to the cost of gas of transaction.

Even though, there is a bad debt socialization to mitigate this, this should not be done frequently because it will affect the lenders which will absorb the losses.

**Proof of concept:** Let's illustrate the scenario with assumption that the exchange rate between underlying tokens, ctokens and usd is 1:1:1.

1. User A uses 10 different accounts to post collateral tokens with 100 ctokens each totalling to 1000 ctokens.
2. User A uses the 10 accounts from step 1 to borrow 90 underlying tokens for each account from dtoken contract.
3. Suddenly, the price of collateral underlying tokens falls at the liquidation price.
4. At this moment, the cost of gas per liquidation operation is already above 100 usd due to high demand activity in the Ethereum blockchain.
5. The liquidators won't attempt to liquidate the underwater accounts since the operation is not profitable (collateral value to claim is lower than the gas cost).
6. The 10 small accounts won't be liquidated and the lenders will absorb the losses through bad debt socialization.

This is only a simplified illustration but if this can be exploited by several people. This can bring down the whole protocol into a bad state.

**Impact:** Small positions can't be liquidated which will result in bad debt to protocol.

**Recommendation:** Implement a minimum collateral posting and borrowing position amount that is enough for liquidator to be profitable.

### 3.2.15 If protocol leverage fee is set, all calls to `deleverage()` will revert

*Submitted by Zach Obront, also found by JCN, bbl4de, Ch301, alix40 and twccstop Exvul*

**Severity:** Medium Risk

**Context:** [PositionFolding.sol#L437-L445](#)

**Description:** All calls to `deleverage()` call back to the `onRedeem()` function in `PositionFolding.sol`.

Before performing the deleveraging, this function checks the protocol's leverage fee, calculates the amount that should be taken, and sends it to the DAO.

```
uint256 fee = (collateralAmount * getProtocolLeverageFee()) / 10000;
if (fee > 0) {
 collateralAmount -= fee;
 SafeTransferLib.safeTransfer(
 collateralUnderlying,
 centralRegistry.daoAddress(),
 fee
);
}
```

However, the protocol leverage fee is expressed in WAD, not in basis points. This can be seen when it is set in the Central Registry:

```
function setProtocolLeverageFee(uint256 value) external {
 _checkElevatedPermissions();

 // Leverage fee cannot be more than 2%.
 if (value > 200) {
 _revert(_PARAMETERS_MISCONFIGURED_SELECTOR);
 }
 // Convert the parameters from basis points to `WAD` format
 // while inefficient we want to minimize potential human error
 // as much as possible, even if it costs a bit extra gas on config.
 protocolLeverageFee = _bpToWad(value);

 emit FeeSet("Leverage", value);
}
```

Therefore, the fee will be many times greater than the collateral amount. When we try to subtract the fee from `collateralAmount` in the `if` block, it will underflow and revert.

**Proof of concept:** The following line can be added to the `testDeLeverage()` test in `TestPositionFolding.t.sol` to set the protocol leverage fee, which causes the function to revert due to underflow:

```
function testDeLeverage() public {
 testLeverage();
+ centralRegistry.setProtocolLeverageFee(100);

 // Warp until collateral posting wait time ends
 vm.warp(block.timestamp + 20 minutes);
 dDAI accrueInterest();
 // ...
}
```

### Recommendation:

```
- uint256 fee = (collateralAmount * getProtocolLeverageFee()) / 10000;
+ uint256 fee = (collateralAmount * getProtocolLeverageFee()) / WAD;
if (fee > 0) {
 collateralAmount -= fee;
 SafeTransferLib.safeTransfer(
 collateralUnderlying,
 centralRegistry.daoAddress(),
 fee
);
}
```

### 3.2.16 All calls to `leverage()` will fail if user has sufficient preexisting debt

Submitted by [Zach Obront](#), also found by [Bernd Artmüller](#)

**Severity:** Medium Risk

**Context:** [PositionFolding.sol#L561-L566](#)

**Description:** When calling `leverage()` to create a folded position, the `queryAmountToBorrowForLeverageMax()` function is used to determine the maximum amount we can borrow.

After pulling the user's total collateral, total debt, and max debt (based on the collateral ratio of each piece of collateral), the following calculation is performed:

```
uint256 maxLeverage = ((sumCollateral - sumDebt) *
 MAX_LEVERAGE *
 sumCollateral) /
 (sumCollateral - maxDebt) /
 DENOMINATOR -
 sumCollateral;
```

We then subtract the debt already taken from this value and divide by the price of the borrow token to get the maximum number of borrow tokens that can still be borrowed.

```
return ((maxLeverage - sumDebt) * 1e18) / price;
```

However, the original calculation is incorrect. To understand why, let's work backwards from the final calculation. The `maxLeverage` value in this calculation should represent the total amount of borrows that would be allowed against our collateral if all borrows were reinvested as collateral, assuming no debt, and priced in USD. By subtracting the USD denominated debt we've already taken, we're left with the USD denominated debt we can still take. By dividing this by price, we're left with the amount of borrow tokens we can still borrow.

But does the original calculation give us this value? We can examine by breaking it apart into multiple pieces:

```
((sumCollateral / (sumCollateral - maxDebt)) * (MAX_LEVERAGE / DENOMINATOR) * (sumCollateral - sumDebt)) -
↳ sumCollateral
```

This formula has four pieces:

- 1) `sumCollateral / (sumCollateral - maxDebt)`: calculate the terminal maximum leverage multiplier.
- 2) `MAX_LEVERAGE / DENOMINATOR`: multiply by 99% to minimize transaction failure.
- 3) `sumCollateral - sumDebt`: multiply the above result by the delta between collateral and debt.
- 4) `- sumCollateral`: subtract the amount of collateral from the result, so we're just left with the borrows and not the amount we've contributed.

However, we can see that Step 3 is not correct. We should be multiplying by `sumCollateral` rather than `sumCollateral - sumDebt` in order to get the full amount that can be borrowed. We then subtract the debt in the final calculation to see how much is remaining.

This error can cause three consequences:

1. If `sumDebt` is a great enough portion of `sumCollateral`, the `maxLeverage` calculation can underflow and revert when it subtracts `sumCollateral`.
2. If this is not the case, the result of `maxLeverage` can be smaller than `sumDebt`, which can cause the final return value to underflow and revert.
3. If neither of these happen, the max leverage amount is significantly understated in the calculation, leading to full leverage not being possible.

**Proof of concept:** We can isolate the `maxLeverage` calculation as its own function with dummy values:

```
function _calculateMaxLeverage(
 uint sumDebt
) internal returns (uint maxLeverage) {
 uint sumCollateral = 1e18;
 uint maxDebt = 0.8e18;
 uint DENOMINATOR = 1e4;
 uint MAX_LEVERAGE = 0.99e4;
 uint price = 1e18;

 maxLeverage =
 ((sumCollateral - sumDebt) * MAX_LEVERAGE * sumCollateral) /
 (sumCollateral - maxDebt) /
 DENOMINATOR -
 sumCollateral;
}
```

Based on these values, the `maxLeverage` should equal `3.96e18`. This can be validated because borrowing `4e18` (the value before the 99% adjustment) would lead to a total collateral amount of `5e18`, which would lead to `maxDebt = 5e18 * 80% = 4e18`, which is exactly what we borrowed.

We can see that if we pass `sumDebt = 0`, this is the value we get (because in that case, `sumCollateral - sumDebt = sumCollateral`, which gives us the correct formula). However, as we increase `sumDebt`, we get different values, until it reaches `0.7e18` (a valid value) and reverts.

```
function testZach_LeverageCalculation() public {
 uint price = 1e18;
 for (uint sumDebt; sumDebt < 0.8e18; sumDebt += 0.1e18) {
 uint maxLeverage = _calculateMaxLeverage(sumDebt);
 uint maxBorrow = ((maxLeverage - sumDebt) * 1e18) / price;
 console.log(sumDebt / 0.1e18, maxLeverage, maxBorrow);
 }
}
```

Logs:

```
0 3950000000000000000 3950000000000000000
1 3455000000000000000 3355000000000000000
2 2960000000000000000 2760000000000000000
3 2465000000000000000 2165000000000000000
4 1970000000000000000 1570000000000000000
5 1475000000000000000 975000000000000000
6 980000000000000000 380000000000000000
```

## Recommendation:

```
- uint256 maxLeverage = ((sumCollateral - sumDebt) *
+ uint256 maxLeverage = (sumCollateral *
 MAX_LEVERAGE *
 sumCollateral) /
 (sumCollateral - maxDebt) /
 DENOMINATOR -
 sumCollateral;
```

### 3.2.17 Miscalculation in `claimYieldForGauge()` leads to yield manager insolvency

Submitted by [Zach Obront](#), also found by [krikolkk](#) and [0xEVom](#)

**Severity:** Medium Risk

**Context:** [BlastNativeYieldManager.sol#L143](#)

**Description:** When an `mToken` calls `claimYieldForGauge()` on the Blast Native Yield Manager, we use the delta between the WETH balances before and after to determine if some dust is remaining, which is sent to the DAO:

```
uint256 gasYield = CHAIN_YIELD_MANAGER.claimMaxGas(
 msg.sender,
 address(this)
);

uint256 WETHPrior = WETH_YIELD_MANAGER.balanceOf(address(this));

// ...

if (gasYield > 0) {
 IWETH(address(WETH_YIELD_MANAGER)).deposit{ value: gasYield }();
 WETHYield += gasYield;
}
```

As we can see, the gas is claimed before `WETHPrior` is snapshotted, but it is not yet deposited into the WETH contract. As a result, the `WETHPrior` balance does not include this gas payment.

Throughout the function, we increment `WETHYield` to equal the total amount that will be paid out (WETH yield plus gas). This amount is then sent as rewards to the gauge.

```
gaugePool.setRewardPerSec(
 yieldDestination,
 nextEpoch,
 address(WETH_YIELD_MANAGER),
 WETHPerSecond // WETHPerSecond is WETHYield / 2_WEEKS
);
```

Finally, once all this logic is done, we snapshot the WETH balance after. By decrementing the prior balance by the amount claimed, we aim to find the amount that should be left, and send any excess to the DAO:

```

uint256 WETHAfter = WETH_YIELD_MANAGER.balanceOf(address(this));

// We offset the prior to what we expect to have been removed from
// prior balance after streaming native yield via gauge system.
WETHPrior -= WETHYield;

if (WETHAfter != WETHPrior) {
 // If somehow the contract pulled more yield than allotted,
 // panic and kill the action.
 if (WETHAfter < WETHPrior) {
 revert BlastNativeYieldManager__InvariantError();
 }

 // If we didnt revert prior we now know that we have more
 // WETH than expected.

 SafeTransferLib.safeTransfer(
 address(WETH_YIELD_MANAGER),
 centralRegistry.daoAddress(),
 WETHAfter - WETHPrior
);
}

```

However, this calculation is not correct. When we calculate the target balance for after, we take `WETHPrior - WETHYield`. If we compare this with the changes in the real balance (`WETHPrior + gas claimed - WETHYield`), we can see that in the "normal" case, `WETHAfter` will be `gas claimed` higher than the target.

As a result, we send the full `gas claimed` amount to the DAO, in addition to sending the correct amount to the gauge.

This leads to an insolvency in the contract, where the amount of WETH expected by the pending rewards is greater than the amount of WETH the contract actually holds.

**Recommendation:** The claimed gas fees should be deposited into the WETH contract before `WETHPrior` is snapshotted.

### 3.2.18 All claimed gas sent to the DAO if cToken has no pending WETH yield

Submitted by [Zach Obront](#)

**Severity:** Medium Risk

**Context:** [BlastNativeYieldManager.sol#L156-L169](#)

**Description:** When `claimYieldForGauge()` is called, we claim all gas and pending WETH & USDB, and send it to the token's gauge to distribute as rewards.

First, the gas is claimed and deposited into WETH. This increases the `WETHYield` variable.

Then, we attempt to claim the WETH yield. If `claimWETHYield = false` or if the current `pendingWETH[msg.sender] = 0`, we skip the if clause where `WETHPerSecond` is set.

```

if (claimWETHYield) {
 uint256 pendingWETH = pendingWETHYield[msg.sender];

 // Recognize USDB yield, if necessary.
 if (pendingWETH > 0) {
 WETHYield += pendingWETH;
 WETHPerSecond = WETHYield / EPOCH_WINDOW;
 }
}

```

This `WETHPerSecond` value is used to determine whether we call the gauge pool with rewards. In this case, because it remains set to 0, this clause is skipped.

Finally, excess WETH is sent to the DAO. In this situation, the DAO will silently (and possibly without even noticing) take all the WETH from the gauge that should have been distributed to users.

**Recommendation:** `WETHPerSecond` should be set outside of the if clause, as long as `WETHYield > 0`.

### 3.2.19 Funds will be left in Zapper when performing redeems

Submitted by [Zach Obront](#)

**Severity:** Medium Risk

**Context:** [SimpleZapper.sol#L183-L212](#)

**Description:** When using the Zappers to perform redeems, there are additional funds left in the Zapper contract that aren't sent to the user.

We can use `redeemAndSwap()` on `SimpleZapper.sol` as an example:

1. `exitCurvance()` redeems our Curvance position, and ensures that AT LEAST `swapperData.inputAmount` is returned from the redemption.

```
if (assets < expectedAssets) {
 revert SimpleZapper__ExecutionError();
}
```

However, it is possible that MORE than `swapperData.inputAmount` was returned. In this case, we don't update this value based on the difference.

2. We then execute the swap using the specified `inputAmount` and leave any excess funds in the Zapper:

```
// Execute swap into `swapperData.outputToken`.
uint256 outAmount = SwapperLib.swap(centralRegistry, swapperData);

_transferToRecipient(swapperData.outputToken, recipient, outAmount);
```

These funds can be stolen by another user by calling `swapAndRepay()` with no swap data, so that we just end up repaying with the existing balance of the Zapper contract.

**Recommendation:** Update `swapperData.inputAmount` to equal the amount of assets returned, so that all funds returned from the redemption are returned to the user.

### 3.2.20 Low value tokens can be reward rate stored incorrectly, losing user yield

Submitted by [Zach Obront](#), also found by [GeneralKay](#) and [KupiaSec](#)

**Severity:** Medium Risk

**Context:** [CTokenCompounding.sol#L621-L639](#)

**Description:** When storing the reward rate, we bitmask to the lower 128 bits in order to pack the storage slot with the current timestamp.

```
newRewardRate := and(newRewardRate, _BITMASK_REWARD_RATE)

result := or(
 newRewardRate,
 or(
 shl(_BITPOS_VEST_END, newVestPeriod),
 shl(_BITPOS_LAST_VEST, timestamp())
)
)
```

This bitmasking assumes that the value of `newRewardRate` should always be less than or equal to `type(uint128).max`.

If we trace back where this call is coming from, we can see that it's called by the `_setNewVaultData()` function:

```
function _setNewVaultData(
 uint256 yieldToVest,
 uint256 periodToVest
) internal {
 // Set rewardRate equal to prorated `yieldToVest` over `periodToVest`,
 // in `WAD` (1e18).
 _vaultData = _packVaultData(
 FixedPointMathLib.mulDiv(yieldToVest, 1e18, periodToVest),
 block.timestamp + periodToVest
);
}
```

Whatever the `yieldToVest` that is passed to this function, we multiply by `1e18` and divide by `1 days` (which is equivalent to multiplying by `1.15e13`) and then pass it along.

Therefore, to cause the overflow, we just need a `yieldToVest` value that is greater than `type(uint128).max / 1.15e13`  $\approx 2.9e25$ .

While this is unlikely to occur, for tokens with high decimal values it is certainly within the range of possibilities of what might occur.

In these cases, the funds will be transferred to the CToken, but since `rewardRate()` isn't updated sufficiently, `_calculatePendingRewards()` will set `pendingRewards` too low, which will not increase `totalAssets` properly, and will lead to the value not accruing to the token holders.

**Proof of concept:** Let's take a hypothetical example that makes the issue clear. If we assume that a CToken has accrued  $\sim 2.9e25$  of a reward token, we can calculate the correct reward rate as:

```
2.9e25 * 1e18 / 1 days = 3.3e38
```

This means that  $3.3e38 / 1e18 = 3.3e20$  of the token should be rewarded each second. Instead, if we actually perform the update, we see:

```
function testZach_YieldOverflow() public {
 deal(address(CONVEX_STETH_ETH_POOL), address(cSTETH), 2.9400396502e25);
 cSTETH.harvest(abi.encode(new SwapperLib.Swap[] (0)));
 console.log(cSTETH.rewardRate());
}
```

```
Logs:
357832832921688864528084840
```

This value is equal to  $3.5e26$ , which means that rewards of just  $3.5e26 / 1e18 = 3.5e8$  will be rewarded each second instead.

**Recommendation:** Don't pack the storage slot for these important values.

### 3.2.21 Fee amount sent to one balance manager is more than expected in `multiswap()` and `executeOTC()`

Submitted by [tvdung94](#), also found by [yttriumzz](#)

**Severity:** Medium Risk

**Context:** [FeeAccumulator.sol#L245-L250](#)

**Description:** According to the comments in protocol, 1/16 or 6.25% of fee token inside fee accumulator should be sent to gelato 1balance.

In terms of funds inside fee accumulator, 1/16 or 6.25% of fee token should be sent and deposited to Gelato 1Balance on polygon.

(See [FeeAccumulator.sol#L775-L776](#))

The problem is that in `multiSwap()` and `executeOTC()`, the amount of fee token sent to the protocol is 1/15 ( $\sim 6.67\%$ ).

**Recommendation:** Consider this change:

```
SafeTransferLib.safeTransfer(
 feeToken,
 oneBalanceFeeManager,
 (feeTokenBalance * vaultCompoundFee()) /
 centralRegistry.protocolHarvestFee() // <=
);
```

### 3.2.22 In Convex2PoolCToken.sol and Convex3PoolCToken.sol and Aura CToken, the deposit should not be able to be done when the pool is shut down

Submitted by [ladboy233](#), also found by [Bauchibred](#)

**Severity:** Medium Risk

**Context:** [Convex2PoolCToken.sol#L276](#), [Convex3PoolCToken.sol#L276](#), [0xF403C135812408BFbE8713b5A23a04b3D48AAE31#code#L974](#), [AuraCToken.sol#L347](#)

**Description:** In [Convex2PoolCToken.sol](#) and [Convex3PoolCToken.sol](#) and [Aura CToken](#), the deposit should not be able to be done when the pool is shut down.

**Proof of concept:** In [Convex2PoolCToken.sol](#) and [Convex3PoolCToken.sol](#), there is a function `_afterDeposit` that deposits the token to the convex finance booster:

- [Convex2PoolCToken.sol](#) [Convex3PoolCToken.sol](#)

```
function _afterDeposit(uint256 assets, uint256) internal override {
 IBooster booster = strategyData.booster;
 SafeTransferLib.safeApprove(asset(), address(booster), assets);
 booster.deposit(strategyData.pid, assets, true);
}
```

**Impact:** This `_afterDeposit` is triggered when harvest the reward. However, there is an absence of a validation step to check if the pool has been shut down.

- Refer to Booster contract:

```
function deposit(uint256 _pid, uint256 _amount, bool _stake) public returns(bool){
 require(!isShutdown,"shutdown");
 PoolInfo storage pool = poolInfo[_pid];
 require(pool.shutdown == false, "pool is closed");
}
```

The code will always try to deposit the collected reward. After swapping collected reward to the assets. However, when deposit asset when the booster / corresponding pool is shutdown, it will revert the harvest transaction and leads to loss of pending reward. Note, the Aura finance has the same problem:

- Line of code:

```
function _afterDeposit(uint256 assets, uint256) internal override {
 IBooster booster = strategyData.booster;
 SafeTransferLib.safeApprove(asset(), address(booster), assets);
 booster.deposit(strategyData.pid, assets, true);
}
```

**Recommendation:** Before executing the deposit logic in `_afterDeposit`, add a condition to check the shutdown state of the pool and the booster.

### 3.2.23 `getPricesForAsset()` will fail for all MTokens

Submitted by [Zach Obront](#), also found by [yttriumzz](#)

**Severity:** Medium Risk

**Context:** [OracleRouter.sol#L371-L372](#)

**Description:** When `getPricesForAsset()` is called with an MToken, we substitute the underlying asset as the asset so we can use it for oracle calls.



```

bool isMToken = mTokenAssets[asset].isMToken;
if (isMToken) {
 asset = mTokenAssets[asset].underlying;
}

```

However, after the prices are gathered, we assume that `asset` still represents the MToken and attempt to make a call to the MToken contract.

```

if (isMToken) {
 uint256 exchangeRate = IMToken(asset).exchangeRateCached();
 // ...
}

```

This call will fail on all underlying tokens, and therefore the call to `getPricesForAsset()` will revert.

**Proof of concept:** The following test can be dropped into `GetPricesForAsset.t.sol` to demonstrate the vulnerability:

```

function testZach_MTokenFail() public {
 oracleRouter.addApprovedAdaptor(address(chainlinkAdaptor));
 chainlinkAdaptor.addAsset(
 address(mUSDC),
 _CHAINLINK_USDC_USD,
 0,
 true
);
 oracleRouter.addAssetPriceFeed(
 _USDC_ADDRESS,
 address(chainlinkAdaptor)
);
 oracleRouter.addMTokenSupport(address(mUSDC));

 vm.expectRevert();
 OracleRouter.FeedData[] memory feedDatas = oracleRouter
 .getPricesForAsset(address(mUSDC), true);
}

```

**Recommendation:** Use a new variable to store underlying asset, which can be used for the oracle calls, while maintaining the MToken address to call to query the exchange rate.

### 3.2.24 User closes and opens a VeCVE lock in the same epoch may result in a loss of points

Submitted by *yttriumzz*

**Severity:** Medium Risk

**Context:** [VeCVE.sol#L901-L915](#)

**Description:** When `CVELocker` settles rewards, if the user has unlocked tokens in a certain epoch, `CVE-Locker` will call `updateUserPoints` function to reduce the user's points. But it doesn't clear `userUnlocksByEpoch` itself.

Assume that within epochA, if the user's last lock expires and is closed, his `userNextClaimIndex` will be cleared, but `userUnlocksByEpoch[epochA]` is still left. Then he opens a new lock in epochA, and `userUnlocksByEpoch[epochA]` will be applied to the user's new lock again, causing the user's points to be repeatedly subtracted.

The following proof of concept shows that the user closed an expired lock and opened a new lock within the same epoch. His new lock is affected by the legacy `userUnlocksByEpoch`, resulting in a point of 0.

**Proof of Concept:**

```

diff --git a/tests/token/VeCVE/functions/CreateLock.t.sol b/tests/token/VeCVE/functions/CreateLock.t.sol
index d01ca88d..a76f79d5 100644
--- a/tests/token/VeCVE/functions/CreateLock.t.sol
+++ b/tests/token/VeCVE/functions/CreateLock.t.sol
@@ -194,4 +194,56 @@ contract CreateLockTest is TestBaseVeCVE {

 assertEq(usdc.balanceOf(address(this)), amount / 2);
}
+

```

```

+ function test_yttriumzz_poc_0010()
+ public
+ setRewardsData(false, false, false)
+ {
+ // 1. config poc env
+ deal(_USDC_ADDRESS, address(cveLocker), 10000e6);
+ uint256 genesisEpochTimestamp = veCVE.genesisEpoch();
+ vm.warp(genesisEpochTimestamp);
+ vm.prank(centralRegistry.feeAccumulator());
+ cveLocker.recordEpochRewards(1e6);
+ assertEq(cveLocker.nextEpochToDeliver(), 1);
+
+ address user00 = address(0xACC00);
+ deal(address(cve), user00, 100 * 1e18);
+ vm.prank(user00);
+ cve.approve(address(veCVE), type(uint256).max);
+
+ // 2. user00 create the first lock
+ vm.prank(user00);
+ veCVE.createLock(1e18, false, rewardsData, "", 0);
+ assertEq(veCVE.userPoints(user00), 1e18);
+ assertEq(veCVE.userUnlocksByEpoch(user00, 26), 1e18);
+
+ // 3. 26 epoch passed
+ for (uint256 i = 1; i < 1 + 26; i++) {
+ vm.warp(genesisEpochTimestamp + i * 2 weeks);
+ vm.prank(centralRegistry.feeAccumulator());
+ cveLocker.recordEpochRewards(1e6);
+ assertEq(cveLocker.nextEpochToDeliver(), 1 + i);
+ }
+ assertEq(cveLocker.nextEpochToDeliver(), 27);
+
+ // 4. user00 close the first lock and create the second lock within the same epoch
+ vm.startPrank(user00);
+ veCVE.processExpiredLock(0, false, false, rewardsData, "", 0);
+ assertEq(veCVE.userPoints(user00), 0);
+ veCVE.createLock(1e18, false, rewardsData, "", 0);
+ assertEq(veCVE.userPoints(user00), 1e18);
+ vm.stopPrank();
+
+ // 5. 1 epoch has passed, the user claim the reward and trigger the bug.
+ // As a result, the user's points are repeatedly subtracted and become 0.
+ vm.warp(genesisEpochTimestamp + 27 * 2 weeks);
+ vm.prank(centralRegistry.feeAccumulator());
+ cveLocker.recordEpochRewards(1e6);
+ assertEq(cveLocker.nextEpochToDeliver(), 28);
+ assertEq(veCVE.userPoints(user00), 1e18);
+ vm.prank(user00);
+ cveLocker.claimRewards(rewardsData, "", 0); // !!!!! trigger bug
+ assertEq(veCVE.userPoints(user00), 0);
+ }
}

```

**Recommendation:** If userUnlocksByEpoch is already applied, delete it:

```

function updateUserPoints(address user, uint256 epoch) external {
 address _cveLocker = address(cveLocker);
 assembly {
 if iszero(eq(caller(), _cveLocker)) {
 mstore(0x00, _UNAUTHORIZED_SELECTOR)
 revert(0x1c, 0x04)
 }
 }

 unchecked {
 userPoints[user] =
 userPoints[user] -
 userUnlocksByEpoch[user][epoch];
 }

+ delete userUnlocksByEpoch[user][epoch];
}

```

### 3.2.25 VertexInterestRate is derived incorrectly from vertexRatePerYear

Submitted by [Zach Obront](#)

**Severity:** Medium Risk

**Context:** [DynamicInterestRateModel.sol#L630-L632](#)

**Description:** When setting the parameters for the interest rate model, we input a `baseRatePerYear` and `vertexRatePerYear`, which are translated into the rates that are saved in storage. The `baseRatePerYear` is translated into `config.baseInterestRate` as follows:

```
config.baseInterestRate =
 (INTEREST_COMPOUND_RATE * baseRatePerYear * WAD) /
 (_SECONDS_PER_YEAR * vertexUtilStart);
```

This is taking the rate per year and dividing it by the number of 10 minute increments in a year, then dividing by `vertexUtilStart`. In other words, it's ensuring that, when the utilization is `vertexUtilStart`, the `baseRatePerYear` is applied each year. When the utilization is lower than `vertexUtilStart`, we get a linear proportion of `baseRatePerYear`.

- `config.vertexInterestRate` is calculated differently:

```
config.vertexInterestRate =
 (INTEREST_COMPOUND_RATE * vertexRatePerYear) /
 _SECONDS_PER_YEAR;
```

This takes the vertex rate per year and simply divides it by the number of 10 minute increments in a year. Therefore, the `vertexInterestRate` saved in storage represents the increase in rate per 1 wei increase in utilization.

When this value is used in `_getVertexInterestRate()`, we divide by `1e18` an extra time:

```
function _getVertexInterestRate(
 uint256 util
) public view returns (uint256) {
 // We divide by 1e36 (WAD_SQUARED) since we need to divide by WAD
 // twice to maintain precision.
 return
 (util * ratesConfig.vertexInterestRate * vertexMultiplier()) /
 WAD_SQUARED;
}
```

The result is that the `vertexInterestRate` is applied per `1e18` increase in utilization rate. This applies dramatically less of an increase than expected, since the total amount the utilization can go above `vertexStartUtil` is much less than `1e18`.

Let's use the rates in `UpdateDynamicInterestRateModel.t.sol` as an example:

- `baseRatePerYear` = 15%
- `vertexStartUtil` = 55%
- `vertexRatePerYear` = 15%

The result is that, the rate scales linearly up to 15% at a utilization of 55%. This is good!

After 55%, we apply a rate of 15% per `1e18` increase, which comes out to 0.15% per 1% increase. Therefore, at a 100% utilization, the rate will only have increased to 21.75%.

Despite the same "rate per year" inputs, the result is that the base rate increases at 0.27% per 1% increase in utilization, while the vertex rate increases at a much lower 0.15% per 1% increase in utilization. This is the opposite dynamic of what we would like.

**Recommendation:** Set `vertexInterestRate` properly so that `vertexRatePerYear` represents the full amount of rate that would be charged on the vertex portion when `util` = 100%.

```
config.vertexInterestRate =
 (INTEREST_COMPOUND_RATE * vertexRatePerYear) /
- _SECONDS_PER_YEAR;
+ (_SECONDS_PER_YEAR * (WAD - vertexUtilStart));
```

Then adjust `_getVertexInterestRate()` to remove the extra division:

```
function _getVertexInterestRate(
 uint256 util
) public view returns (uint256) {
 // We divide by 1e36 (WAD_SQUARED) since we need to divide by WAD
 // twice to maintain precision.
 return
 (util * ratesConfig.vertexInterestRate * vertexMultiplier()) /
- WAD_SQUARED;
+ WAD;
}
```

### 3.2.26 Cross chain emissions rate updates can fail due to insufficient gas

Submitted by [Zach Obront](#), also found by [yttriumzz](#)

**Severity:** Medium Risk

**Context:** [ProtocolMessagingHub.sol#L234-L257](#)

**Description:** When messages are sent cross chain using Wormhole, a fixed gas limit of 250000 is set.

```
function _sendWormholeMessages(
 uint256 dstChainId,
 address toAddress,
 uint256 messageFee,
 uint8 payloadId,
 bytes memory payload
) internal returns (uint64) {
 // Validate that we are aiming for a supported chain.
 if (centralRegistry.supportedChainData(dstChainId).isSupported < 2) {
 revert ProtocolMessagingHub__ChainIsNotSupported();
 }

 return
 centralRegistry.wormholeRelayer().sendPayloadToEvm{
 value: messageFee
 }(
 centralRegistry.wormholeChainId(dstChainId),
 toAddress,
 abi.encode(payloadId, payload), // payload
 0, // No receiver value since we're just passing a message.
 _GAS_LIMIT
);
}
```

```
uint256 internal constant _GAS_LIMIT = 250_000;
```

This is insufficient for many calls aimed at updating the gauge emissions, as this action requires many SSTOREs and is thus gas intensive. Based on my testing (below), an call to just 1 gauge to update 5 MTo-kens will use more than the allocated gas.

In the event that the gas limit is too small, the transaction will fail on the receiving side and will not be replayable.

**Proof of concept:** In `ReceiveWormholeMessages.t.sol`, the tests are run without any gauges for simplicity. To prove out the gas costs, we need to create an environment where Wormhole messages can be received, but gauges are also deployed.

To do this, you can copy this entire file into the repo and run the test with `forge test --match-test testZach_OutOfGasWhenSettingCrossChainEmissions`:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.17;

import { TestBaseMarket } from "tests/market/TestBaseMarket.sol";
import { ProtocolMessagingHub } from "contracts/architecture/ProtocolMessagingHub.sol";

contract ZachReceiveWormhole is TestBaseMarket {
 address public srcMessagingHub;

 function setUp() public override {
```

```

super.setUp();

gaugePool.start(address(marketManager));
vm.warp(gaugePool.startTime());

srcMessagingHub = makeAddr("SrcMessagingHub");

centralRegistry.addChainSupport(
 address(srcMessagingHub),
 address(srcMessagingHub),
 address(cve),
 42161,
 1,
 1,
 23
);
}

function testZach_OutOfGasWhenSettingCrossChainEmissions() public {
 uint NUM_TOKENS = 5;

 address[] memory gaugePools = new address[](1);
 uint256[] memory emissionTotals = new uint256[](1);
 address[] [] memory tokens = new address[] [] (1);
 uint256[] [] memory emissions = new uint256[] [] (1);

 gaugePools[0] = address(gaugePool);
 emissionTotals[0] = NUM_TOKENS * _ONE;
 uint256 chainLockedAmount = _ONE;
 uint256 messageType = 3;

 tokens[0] = new address[] (NUM_TOKENS);
 for (uint256 i = 0; i < NUM_TOKENS; i++) {
 tokens[0][i] = address(uint160(i));
 }

 emissions[0] = new uint256[] (NUM_TOKENS);
 for (uint256 i = 0; i < NUM_TOKENS; i++) {
 emissions[0][i] = _ONE;
 }

 vm.prank(_WORMHOLE_RELAYER);
 uint startGas = gasleft();
 protocolMessagingHub.receiveWormholeMessages(
 abi.encode(
 4,
 abi.encode(
 gaugePools,
 emissionTotals,
 tokens,
 emissions,
 chainLockedAmount,
 messageType
)
),
 new bytes[] (0),
 bytes32(uint256(uint160(address(srcMessagingHub)))),
 23,
 bytes32("0x01")
);
 assertGt(startGas - gasleft(), 250_000);
}
}

```

**Recommendation:** Change `_GAS_LIMIT` to `DEFAULT_GAS_LIMIT` and override it with a custom calculation in situations where gas is likely to be more expensive.

### 3.2.27 `getLower` can return higher value due to ETHUSD conversion assumptions

Submitted by [Zach Obront](#), also found by [Haxatron](#) and [yttriumzz](#)

**Severity:** Medium Risk

**Context:** [OracleRouter.sol#L765-L776](#)

**Description:** When `getPrice()` is called on the Oracle Router, we pass a `getLower` argument, which determines whether we take the higher or lower price. This is used to ensure that discrepancies always resolve in the protocol's favor.

We also pass a value for `inUSD`, which determines if the return value is in USD or ETH. If the oracle doesn't support the option we choose, a second oracle call to ETHUSD is performed and the price is adjusted based on this value.

```
if (data.inUSD != inUSD) {
 uint256 newPrice;
 (newPrice, data.hadError) = _getETHUSD(getLower);
 if (data.hadError) {
 return FeedData({ price: 0, hadError: true });
 }

 data.price = uint240(
 _convertETHUSD(data.price, newPrice, data.inUSD)
);
}
```

In the event that we get a price in ETH and want it in USD, we multiply the result by the ETH price. In the event that we get a price in USD and want it in ETH, we divide by the ETH price.

To enforce the requested `getLower` when converting ETH to USD, we should get the ETH price with the same value (ie for lower price, get lower ETH price). However, to enforce the requested `getLower` when converting USD to ETH, we need the opposite ETH price (i.e. for a lower price, we should get the higher ETH price).

As we can see in the snippet above, we always pass the same `getLower` value to `_getETHUSD()`. As a result, when we get a USD price that we want in ETH, this results in increasing the price, which can lead to a lower price that is greater than the higher price.

**Proof of concept:** The following test contract can be dropped into the test suite to demonstrate the vulnerability:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.17;

import { OracleRouter } from "contracts/oracles/OracleRouter.sol";
import { TestBaseOracleRouter } from "../TestBaseOracleRouter.sol";
import { ICentralRegistry } from "contracts/interfaces/ICentralRegistry.sol";
import { PriceReturnData } from "contracts/interfaces/IOracleAdaptor.sol";

contract Oracle {
 address constant FXS_TOKEN = 0x3432B6A60D23Ca0dFCa7761B7ab56459D9C964D0;
 address constant ETH_ADDRESS = 0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEE;

 function isSupportedAsset(address asset) external view returns (bool) {
 return (asset == ETH_ADDRESS || asset == FXS_TOKEN);
 }

 function getPrice(
 address asset,
 bool inUSD,
 bool getLower
) external view returns (PriceReturnData memory rd) {
 if (asset == FXS_TOKEN) {
 // for simplicity, let's assume that...
 // 1) we don't offer ETH support for FXS, so always return USD
 // 2) the oracles are identical, so getLower == !getLower
 // 3) current price is $6, so just return that
 return PriceReturnData(6e18, false, true);
 } else if (asset == ETH_ADDRESS) {
 // price of ETH in ETH is 1
 if (!inUSD) return PriceReturnData(1e18, false, false);

 // price of ETH in USD works, but let's have a range
 if (getLower) return PriceReturnData(3900e18, false, true);
 if (!getLower) return PriceReturnData(4000e18, false, true);
 } else {
 // only these two tokens are supported
 return PriceReturnData(0, true, true);
 }
 }
}
```

```

}

contract TestOracleRouter is TestBaseOracleRouter {
 address constant FXS_TOKEN = 0x3432B6A60D23Ca0dFCa7761B7ab56459D9C964D0;
 address constant ETH_ADDRESS = 0xEeeeeEeeeEeEeeEeEeEeeeeEeeeeeeEEeE;

 function testZach_GetLowerIsHigher() public {
 OracleRouter oracleRouter = new OracleRouter(
 ICentralRegistry(address(centralRegistry))
);
 Oracle feed = new Oracle();

 oracleRouter.addApprovedAdaptor(address(feed));
 oracleRouter.addAssetPriceFeed(ETH_ADDRESS, address(feed));
 oracleRouter.addAssetPriceFeed(FXS_TOKEN, address(feed));

 (uint lower,) = oracleRouter.getPrice({
 asset: FXS_TOKEN,
 inUSD: false,
 getLower: true
 });
 (uint higher,) = oracleRouter.getPrice({
 asset: FXS_TOKEN,
 inUSD: false,
 getLower: false
 });

 assert(lower > higher);
 }
}

```

## Recommendation:

```

if (data.inUSD != inUSD) {
 uint256 newPrice;
- (newPrice, data.hadError) = _getETHUSD(getLower);
+ bool ethUsdLower = inUSD ? getLower : !getLower;
+ (newPrice, data.hadError) = _getETHUSD(ethUsdLower);
 if (data.hadError) {
 return FeedData({ price: 0, hadError: true });
 }

 data.price = uint240(
 _convertETHUSD(data.price, newPrice, data.inUSD)
);
}

```

### 3.2.28 Insufficient L2 uptime checks

Submitted by [Zach Obront](#), also found by [ladboy233](#), [Bauer](#) and [0xffchain](#)

**Severity:** Medium Risk

**Context:** [OracleRouter.sol#L823-L841](#)

**Description:** The `OracleRouter` implements a function to check whether the L2 sequencer is up (and has been up for the past 60 minutes).

This is important because, in the case that an L2's sequencer is down and only accepting forced transactions through the L1 inbox (or has just come back online), the state of the chain can be unpredictable and prone to manipulation.

Currently, this check is only performed in the `_getETHUSD()` function. This function is only called in the situation where the oracle returns the price in the wrong asset (ie in USD instead of ETH).

In cases where the oracle is priced in the correct asset, the check will be skipped.

**Recommendation:** Move the call to `_isSequencerValid()` into the higher level `getPrice()` functions, so it is checked on all oracle calls.

### 3.2.29 Users will lose some funds when unfolding positions with `deleverage()`

Submitted by [Zach Obront](#), also found by [JCN](#) and [KupiaSec](#)

**Severity:** Medium Risk

**Context:** [PositionFolding.sol#L510-L519](#)

**Description:** When calling `deleverage()` to unwind a folded position, the following flow takes place:

- 1) We call `withdrawByPositionFolding()` on the CToken, which burns our CToken shares and transfers us the underlying CToken asset (usually some kind of LP token).
- 2) The CToken calls `onRedeem()` on PositionFolding.
- 3) This first uses the Zapper to convert the LP token into its constituent tokens.
- 4) The Swapper is then used to convert these constituent tokens into the borrow token.
- 5) A specified amount of the borrow token is repaid, while the rest is sent back to the redeemer.
- 6) If there is any leftover of the underlying CToken asset (LP token), it is sent back to the user.

As we can see, we are careful to ensure that the LP token and the borrow token are both sent back to the user. However, there are no checks that the constituent tokens of the LP token are sent back to the user.

This is especially important, because these are the values that are (a) returned from a zap and then (b) swapped out. There isn't a way to know the precise amounts that will be returned from the zap before executing on chain, and therefore it is very possible that all of these tokens are not included in the swap.

**Recommendation:** The constituent tokens of the CToken's underlying asset should be sent back to the user in addition to the underlying asset and the borrow token.

### 3.2.30 `deleverage()` only allows one swap, which isn't sufficient for the use case

*Submitted by Zach Obront*

**Severity:** Medium Risk

**Context:** [PositionFolding.sol#L476](#)

**Description:** When calling `deleverage()` to unwind a folded position, we burn our CTokens, claim the CToken's underlying asset (usually an LP token), and zap it back into its constituent tokens.

We then make a single call to `SwapperLib.swap(centralRegistry, deleverageData.swapData)`; in order to swap into the borrow token.

However, in the case that we received multiple constituent tokens (and need to repay a large portion of the value as the borrow tokens), a single swap will not be sufficient to convert our multiple tokens into the borrow token.

Further, it is important that each constituent token is able to be swapped, because if these tokens are not swapped into the borrow token, there is no way to reclaim them in the function, and they will remain locked in the Position Folding contract.

**Recommendation:** Allow the user to submit an array of `swapDatas`, which can be executed sequentially to swap each token into the `borrowToken`.

### 3.2.31 Reward distribution fails for all chains if a single chain would receive zero fees

*Submitted by Bernd Artmüller*

**Severity:** Medium Risk

**Context:** [FeeAccumulator.sol#L806-L810](#)

**Description:** The `FeeAccumulator.executeEpochFeeRouter` function reports the epoch rewards to all remote chains and distributes the fees (i.e., USDC) pro-rata to all chains based on the locked `veCVE` tokens. Internally, the `_executeEpochFeeRouter` function calculates the pro-rata fees per chain (`feeTokenBalanceForChain`) and bridges them to the various chains via the `ProtocolMessagingHub.sendFees` function.

In case the fee tokens are bridged via Circle's CCTP, i.e., calling the Circle Token Manager's `depositForBurnWithCaller` function in lines 139-145 of the `FeeTokenBridgingHub._transferFeeTokenViaCCTP` function, the number of tokens to be bridged (`amount`) is checked to be non-zero and reverts otherwise.



Consequently, if the amount of fees bridged to one of the remote chains is zero, e.g., due to the chain having zero locked veCVE tokens, the bridging attempt fails for that and all other chains, preventing the fees from getting distributed and the epoch from getting settled.

**Impact:** Epoch rewards are not distributed to veCVE token holders and the new epoch is not started.

**Recommendation:** In the `_executeEpochFeeRouter` function, consider checking if `feeTokenBalanceForChain` is zero. If so, skip the `messagingHub.sendFees` call to ensure all other chains and the veCVE token holders receive their entitled rewards.

### 3.2.32 Harvested Convex rewards cannot be swapped to native tokens due to missing `receive` function

Submitted by [Bernd Artmüller](#), also found by [kodyvim](#) and [ladboy233](#)

**Severity:** Medium Risk

**Context:** [Convex2PoolCToken.sol#L301](#), [Convex3PoolCToken.sol#L301](#)

**Description:** In both the `Convex2PoolCToken` and `Convex3PoolCToken` contracts, the `harvest` function claims the Convex rewards and swaps them to the pool's underlying tokens (configurable by the harvester) before adding them as liquidity to the pool.

If the pool has native tokens (e.g., ETH, as in the [stETH/ETH pool](#)) as one of the underlying tokens, swapping rewards to native tokens is not possible because the contracts do not have a `receive` function. As a result, native tokens (as a result of swapping the rewards) cannot be added as liquidity to the pool, which can amplify the imbalance of the pool.

Curve pools are sensitive to imbalances and incentivize users to add liquidity in the desired ratio to balance the pool. If liquidity is added, which further imbalances the pool, a penalty is applied, resulting in fewer LP tokens and thus less yield for Curvature `Convex2PoolCToken` and `Convex3PoolCToken` token holders.

This oversight was potentially missed due to the tests not covering swaps to native tokens, instead, native tokens are provided to the contract directly via the `deal` Foundry cheatcode.

**Impact:** Rewards cannot be swapped to the pool's underlying native token, preventing it from being added as liquidity to the pool. Only the other pool token can be supplied as liquidity. If the Curve pool is imbalanced, e.g., the pool has more ETH than the other token, adding liquidity with the other token will result in a penalty and, thus, fewer LP tokens.

**Proof of concept:** Adapting the tests in [TestConvex2PoolCToken.t.sol](#) as follows shows the issue:

```
function testConvexStethEthPool() public {
 uint256 assets = 100e18;
 deal(address(CONVEX_STETH_ETH_POOL), address(user1), assets);

 vm.prank(address(user1));
 CONVEX_STETH_ETH_POOL.approve(address(cSTETH), assets);

 vm.prank(address(user1));
 cSTETH.deposit(assets, user1);

 assertEq(
 cSTETH.totalAssets(),
 assets + 42069,
 "Total Assets should equal user deposit."
);

 // Advance time to earn CRV and CVX rewards
 vm.warp(block.timestamp + 3 days);

 // Mint some extra rewards for Vault.
 deal(address(CRV), address(cSTETH), 100e18);
 deal(address(CVX), address(cSTETH), 100e18);
 deal(address(cSTETH), 1 ether);

 cSTETH.harvest(abi.encode(new SwapperLib.Swap[] (0)));

 assertEq(
 cSTETH.totalAssets(),
 assets + 42069,
 "Total Assets should equal user deposit."
);
}
```

```

);

 vm.warp(block.timestamp + 8 days);

 // Mint some extra rewards for Vault.
 deal(address(CRV), address(cSTETH), 100e18);
 deal(address(CVX), address(cSTETH), 100e18);
- deal(address(cSTETH), 1 ether);
+ assertGt(address(this).balance, 1 ether);
+ payable(address(cSTETH)).transfer(1 ether); // @audit-info Fails due to missing receive function
+

 cSTETH.harvest(abi.encode(new SwapperLib.Swap[] (0)));
 vm.warp(block.timestamp + 7 days);

 uint256 totalAssets = cSTETH.totalAssets();

 assertGt(
 totalAssets,
 assets + 42069,
 "Total Assets should greater than original deposit."
);

 vm.startPrank(address(user1));
 cSTETH.withdraw(cSTETH.balanceOf(user1), user1, user1);
 vm.stopPrank();
}

```

**Recommendation:** Consider adding a receive function to the Convex2PoolCToken and Convex3PoolCToken cToken contracts to allow swapping to native tokens (e.g., ETH).

### 3.2.33 CToken yield stops compounding if the reward rate is set to zero

Submitted by *Bernd Artmüller*

**Severity:** Medium Risk

**Context:** CTokenCompounding.sol#L750-L753

**Description:** The CTokenCompounding.harvest function is regularly called by the harvester (i.e., Gelato bot) to harvest and compound outstanding vault rewards (yield). This yield is then vested over a period of `vestPeriod = 1 days`.

The `harvest` function only harvests "new" rewards if the last vesting period has ended, checked via the `_checkVestStatus` function. If the last vesting period has ended, rewards are claimed (specific to each cToken and the underlying protocol), the new reward rate per second is calculated and stored along with the new vesting period end timestamp.

Looking at the `_checkVestStatus` function, it checks if the last vesting claim timestamp (`lastVestClaim`) is greater or equal to the vesting period end timestamp (`vestingPeriodEnd`). If it is, the function returns `true`, indicating that the current vesting period has ended and rewards can be harvested.

If for some reasons the `lastVestClaim` timestamp is not updated, i.e., the timestamp remains less than the `vestingPeriodEnd`, the `_checkVestStatus` function will return `false`, preventing the harvesting of rewards and the start of a new vesting period.

This `lastVestClaim` timestamp is updated in the `_vestRewards` function, which is called from within the `_vestIfNeeded` function (and also [here](#), and [here](#)). In all instances, the `_vestRewards` function is **only called if there are pending rewards to be vested**.

There are multiple scenarios where the resulting reward rate could be set to zero:

1. If a cToken's underlying protocol has no yield for a given period.
2. If a cToken has no underlying assets deposited in the protocol to generate yield.
3. Due to slippage during reward token swapping or adding liquidity.

In all cases, as the `harvest` function in all cTokens has some form of slippage control, the likelihood of this issue depends on the Gelato bot not setting an appropriate slippage control.

If the `harvest` function is called during this period, the reward rate will be set to zero.

As a result, there will not be any pending rewards and thus the `_vestRewards` function will not be called on subsequent `harvest` calls. Consequently, the `lastVestClaim` timestamp will not be updated, and the `_checkVestStatus` function will keep returning `false`, preventing the harvesting of future rewards.

In conclusion, if the reward rate is ever set to 0, the vesting period will not advance, and any new rewards will not be harvested and vested to cToken holders.

**Impact:** cToken yield vesting period is stuck, resulting in a loss of yield for cToken holders.

**Recommendation:** In the `_vestIfNeeded` function, consider calling the `_vestRewards` function even if there are no pending rewards (i.e., `pending = 0`) to ensure the `lastVestClaim` timestamp is updated and the vesting period advances.

### 3.2.34 If the DAO deposits dToken reserves, `totalReserves` will be miscalculated resulting in an incorrect exchange and borrow rate

*Submitted by Bernd Artmüller, also found by tvdung94, Zach Obront, alix40 and KupiaSec*

**Severity:** Medium Risk

**Context:** `DToken.sol#L618`, `DToken.sol#L643`

**Description:** The `DToken.totalReserves` variable keeps track of the total underlying reserves (e.g., USDC). This reserve consists of protocol fees or funds transferred by the Curvance DAO via the `depositReserves` function.

This `totalReserves` value **must be in the underlying token**, as it's used in multiple places where the value is expected to represent an underlying token amount:

1. In the `exchangeRateCached` function where `totalReserves` is deducted from `totalBorrow` (which is in underlying tokens).
2. In the `accrueInterest` function, where `totalReserves` is supplied to the `getBorrowRateWithUpdate` function to calculate the current borrow rate. Specifically, the `totalReserves` value is used to calculate the utilization rate.

However, the `depositReserves`, `withdrawReserves`, and `processWithdrawReserves` functions assume that `totalReserves` is in DToken shares, i.e., tokens, as seen in [line 618](#), [line 643](#), and [line 669](#).

As a result, once the Curvance DAO deposits or withdraws reserves, the `totalReserves` value is not updated correctly. For example, if the dToken exchange rate (shares to underlying) is 2, and the Curvance DAO deposits 100 USDC, the `totalReserves` value will be incremented by  $100 / 2 = 50$  shares, instead of 100 USDC. Among other impacts, this will inflate the exchange rate as the reserves are not correctly accounted for and will lead to a lower borrow rate than expected.

**Impact:** `totalReserves` incorrectly accounts for the underlying reserve amount. As a result, the exchange rate is incorrectly calculated, the borrow rate is wrong, and the DAO would accidentally withdraw more reserves than intended.

**Recommendation:** Consider not mixing underlying and shares in the `totalReserves` variable. Specifically, use the underlying token amount in both the `depositReserves` and `withdrawReserves` function when updating the `totalReserves` variable.

Additionally, the `processWithdrawReserves` function should withdraw the underlying token amount stored in `totalReserves` and calculate the corresponding shares to be [withdrawn from the gauge pool](#) based on the current exchange rate.

### 3.2.35 Balancer oracle should not be used while in recovery mode

*Submitted by 0xffchain*

**Severity:** Medium Risk

**Context:** `ComposableStablePool.sol#L1058`,

**Description:** Balancer pool rates is a function of balance changing actions on the balancer pool, like deposit, withdrawal/exit and swap. The rate increases and decreases based on these actions. But in the case of when a pool's recovery mode is turned on, the effect of these actions on the invariants that dictate the rates are skipped.

An example is when a user exits a pool, the onExit hook is called.

- [BasePool.sol#L351-L403](#):

```
function onExitPool(
 bytes32 poolId,
 address sender,
 address recipient,
 uint256[] memory balances,
 uint256 lastChangeBlock,
 uint256 protocolSwapFeePercentage,
 bytes memory userData
) external override onlyVault(poolId) returns (uint256[] memory, uint256[] memory) {
 uint256[] memory amountsOut;
 uint256 bptAmountIn;
 // When a user calls `exitPool`, this is the first point of entry from the Vault.
 // We first check whether this is a Recovery Mode exit - if so, we proceed using this special
 ↳ lightweight exit
 // mechanism which avoids computing any complex values, interacting with external contracts, etc.,
 ↳ and generally
 // should always work, even if the Pool's mathematics or a dependency break down.
 if (userData.isRecoveryModeExitKind()) {
 // This exit kind is only available in Recovery Mode.
 _ensureInRecoveryMode();
 // Note that we don't upscale balances nor downscale amountsOut - we don't care about scaling
 ↳ factors during
 // a recovery mode exit.
 (bptAmountIn, amountsOut) = _doRecoveryModeExit(balances, totalSupply(), userData);
 } else {
 // Note that we only call this if we're not in a recovery mode exit.
 _beforeSwapJoinExit();
 uint256[] memory scalingFactors = _scalingFactors();
 _upscaleArray(balances, scalingFactors);
 (bptAmountIn, amountsOut) = _onExitPool(
 poolId,
 sender,
 recipient,
 balances,
 lastChangeBlock,
 inRecoveryMode() ? 0 : protocolSwapFeePercentage, // Protocol fees are disabled while in
 ↳ recovery mode
 scalingFactors,
 userData
);
 // amountsOut are amounts exiting the Pool, so we round down.
 _downscaleDownArray(amountsOut, scalingFactors);
 }
 // Note we no longer use `balances` after calling `_onExitPool`, which may mutate it.
 _burnPoolTokens(sender, bptAmountIn);
 // This Pool ignores the `dueProtocolFees` return value, so we simply return a zeroed-out array.
 return (amountsOut, new uint256[](balances.length));
}
```

It checks if the pool is on recovery mode, and if it is it does the minimum to get users funds back without making changes to the invariants.

- [ComposableStablePool.sol#L949-L973](#):

```

function _doRecoveryModeExit(
 uint256[] memory registeredBalances,
 uint256,
 bytes memory userData
) internal view override returns (uint256, uint256[] memory) {
 // Since this Pool uses premined BPT, we need to replace the total supply with the virtual total
 ↪ supply, and
 // adjust the balances array by removing BPT from it.
 // Note that we don't compute the actual supply, which would require a lot of complex calculations
 ↪ and
 // interactions with external components. This is fine because virtual and actual supply are the
 ↪ same while
 // recovery mode is enabled (since all protocol fees are forfeit and the fee percentages zeroed
 ↪ out).
 (uint256 virtualSupply, uint256[] memory balances) = _dropBptItemFromBalances(registeredBalances);
 uint256 bptAmountIn = userData.recoveryModeExit();
 uint256[] memory amountsOut = new uint256[](balances.length);
 uint256 bptRatio = bptAmountIn.divDown(virtualSupply);
 for (uint256 i = 0; i < balances.length; i++) {
 amountsOut[i] = balances[i].mulDown(bptRatio);
 }
 // The vault requires an array including BPT, so add it back in here.
 return (bptAmountIn, _addBptItem(amountsOut, 0));
}

```

But when the pool is not in recovery mode, it makes changes to the invariants.

- [ComposableStablePool.sol#L631-L697](#)

```

function _onExitPool(
 bytes32,
 address,
 address,
 uint256[] memory registeredBalances,
 uint256,
 uint256,
 uint256[] memory scalingFactors,
 bytes memory userData
) internal override returns (uint256, uint256[] memory) {
 return _onJoinExitPool(false, registeredBalances, scalingFactors, userData);
}
/**
 * @dev Pay protocol fees before the operation, and call `_updateInvariantAfterJoinExit` afterward, to
 ↪ establish
 * the new basis for protocol fees.
 */
function _onJoinExitPool(
 bool isJoin,
 uint256[] memory registeredBalances,
 uint256[] memory scalingFactors,
 bytes memory userData
) internal returns (uint256, uint256[] memory) {
 (
 uint256 preJoinExitSupply,
 uint256[] memory balances,
 uint256 currentAmp,
 uint256 preJoinExitInvariant
) = _beforeJoinExit(registeredBalances);
 function(uint256[] memory, uint256, uint256, uint256, uint256[] memory, bytes memory)
 internal
 view
 returns (uint256, uint256[] memory) _doJoinOrExit
 = (isJoin ? _doJoin : _doExit);
 (uint256 bptAmount, uint256[] memory amountsDelta) = _doJoinOrExit(
 balances,
 currentAmp,
 preJoinExitSupply,
 preJoinExitInvariant,
 scalingFactors,
 userData
);
 // Unlike joinswaps, explicit joins do not mutate balances into the post join-exit balances so we
 ↪ must perform
 // this mutation here.

```

```

function(uint256, uint256) internal pure returns (uint256) _addOrSub = isJoin ? FixedPoint.add :
↳ FixedPoint.sub;
_mutateAmounts(balances, amountsDelta, _addOrSub);
uint256 postJoinExitSupply = _addOrSub(preJoinExitSupply, bptAmount);
// Pass in the post-join balances to reset the protocol fee basis.
// We are minting bptAmount, increasing the total (and virtual) supply post-join
_updateInvariantAfterJoinExit(
 currentAmp,
 balances,
 preJoinExitInvariant,
 preJoinExitSupply,
 postJoinExitSupply
);
// For clarity and simplicity, arrays used and computed in lower level functions do not include BPT.
// But the amountsIn array passed back to the Vault must include BPT, so we add it back in here.
return (bptAmount, _addBptItem(amountsDelta, 0));
}

```

So meaning when recovery mode is turned on, the rates does not reflect the true market value of the pool.

**Recommendation:** Before calls to the balancer pools for rate it should call.

- [BasePool.sol#L217-L219](#)

```

function inRecoveryMode() public view override returns (bool) {
 return _miscData.decodeBool(_RECOVERY_MODE_BIT_OFFSET);
}

```

On the pool to find out if the pool is on recovery mode, and if so then revert or use an alternative oracle.

### 3.2.36 Malicious Liquidity Providers, could artificially spike Interest rates, to steal value from Borrowers

Submitted by [alix40](#)

**Severity:** Medium Risk

**Context:** [DynamicInterestRateModel.sol#L290-L339](#)

**Description:** To calculate the interest rates in the DToken contract, the method `getBorrowRateWithUpdate()` from the `DynamicInterestRateModel` contract is used. This method is highly dependent on the utilization rate of the DToken. To make it simple the higher the utilization rate, and the less funds are available for lending, the higher the interest rate will be. Also thanks to the vertex multiplier introduced in the contract, if the utilization rate crosses a certain threshold, the interest rates will exponentially grow. This will lead to Liquidity providers and the protocol getting more yield and borrowers paying higher fees. This is an expected behaviour and is also implemented by other battle-tested lending protocols.

Please also note that if the utilization rate is above the `vertexStartingPoint`, will continue to exponentially grow. The vertex Multiplier will however only be update following an `adjustmentRate`. The `adjustmentRate` in the contract is **forced** to be at least 1 hour and max 12 hours.

This allows LP providers to spike the `vertexMultiplier` artificially leading to really high interest rates, which in the short-term would steal value from Borrowers and in the long term make the curvance Protocol unattractive because the manipulated interest rates.

Note this attack is only relevant in periods of higher utilization rate, as if the utilization is under the `vertexStartingPoint`, only **the base rate** would be used and not **the vertex rate**

**Proof of concept:** According to the sponsor, a realistic `adjustmentRate` that would be used in the production is 4 hours. This means that the vertex rate are updated at a max of once each 4 hours. Liquidity Providers in the DToken could deploy MEV Bots to withdraw their liquidity at the block before the block where the vertex multiplier are updated. This would lead to less liquidity available to borrow, so the calculations with the `getBorrowRateWithUpdate()` would result in an artificially spiked interest rate. The Malicious Liquidity Provider could simply redeposit his Liquidity in the next block in which interest rates are artificially spiked and would then benefit from the higher interest rates for the next 4 hours.

**Financial Feasibility of the Attack:** The attack would only cost the LP provider 5 seconds of not receiving interests and gas fees for withdrawing liquidity and redepositing it. => The cost of the attack < 5 USD in gas fees.



The attacker after artificially spiking the interest rates, would get the higher interest rates for the next 4 hours. Depending of the reached interest rate and the amount of Liquidity provided, the attacker would receive somewhere from 2 figures to 5 figures more in fees paid for the next 4 hours.

**Impact:** Having spiked interest rates artificially, would lead to higher yield for LP providers and protocol short-term. Long-term this behaviour would discourage Borrowers from using the Curvature Protocol, as they will always have unrealistic and spiked up interest rates.

Secondly the state as it is now, would encourage all LP providers to constantly withdraw and supply their liquidity, which only brings instability for the protocol.

**Recommendation:** The logic implemented in the `DynamicInterestRateModel` contract, brings a lot of innovations and could benefit the protocol and its users long-term. So I wouldn't recommend necessarily changing the model. A possible solution, that I would recommend to avoid this attack vector, is to introduce an exit fee for the `DToken`. The exit fee could be as small as 0.1%, this would discourage LP providers from constantly withdrawing their liquidity, as it makes it financially unfeasible. This would also encourage Liquidity Providers to hold their positions for a longer period of time.

Interest rates are updated each hour, so basically an LP provider who wants way more interest paid, could follow the following strategy, Before the block is reached in which the rates are updated LP, will withdraw his liquidity to spike up the interest rates. This will lead to the interest rates paid by borrowers spiked up artificially leading into high yield for LP providers and borrowers finding themselves locked in a high interest space (specially with the 20 min lock)

### 3.2.37 `GMCToken` incorrectly attempts to claim funding fees from GMX

Submitted by *Bernd Artmüller*

**Severity:** Medium Risk

**Context:** `GMCToken.sol`#L331-L335

**Description:** The `GMCToken` `cToken` contract is supposed to claim funding fees from GMX and distribute them to the `cToken` holders via vesting.

However, there is a misunderstanding about the GMX GM token and how rewards are accumulated. According to the [GMX docs](#), fees collected are automatically reflected in the GM token price:

Fees from leverage trading and swaps will automatically increase the price of GM tokens.

The `GMCToken` contract attempts to claim funding fees by calling the `IGMXExchangeRouter.claimFundingFees` function in lines 331-335 of the `_claimReward` function (as part of harvesting rewards). As the GM token price increases to reflect accumulated fees, this is the wrong mechanism to claim fees. Funding fees are claimable by long or short positions, not by GM token holders.

**Impact:** `GMCToken` incorrectly attempts to claim funding fees from GMX, while the GM token price automatically reflects the accumulated fees.

**Recommendation:** Consider removing the `GMCToken` contract as the GMX GM token can also be used as a `CTokenPrimitive` `cToken`.

### 3.2.38 The `averagePrice` algorithm will report inaccurate prices in the event of a depeg

Submitted by *cuthalion0x*, also found by *n1punp*, *Zach Obront* and *tdung94*

**Severity:** Medium Risk

**Context:** `BalancerStablePoolAdaptor.sol`#L121-L122

**Description:** The `averagePrice` algorithm used here will fail drastically in the event of any constituent token depegging. Instead, we should use the same `minPrice` method as in the `Curve2PoolLPAdaptor`. The original source of this methodology is [Chainlink's article](#).

In short, liquidity pools do not represent an arithmetic average of the constituent token set, so it is inaccurate to treat the price as an average. Instead, any losses incurred by a single token within the pool are **amplified** in the pool share price by subsequent arbitrage (often called "impermanent loss"). Consider a trivial example:

A pool holds two constituent tokens, A and B. Token A experiences a dramatic price crash towards 0. Arbitrageurs can now acquire a near-infinite balance of Token A for near-zero value at some other venue (another DEX, CEX, OTC, etc.). Arbitrageurs sell these tokens into the pool to extract nearly the entire balance of Token B virtually for free. In the end state, the pool holds nearly 100% of its underlying balance in the worthless Token A, and nearly 0 balance in the valuable Token B. The value of a single pool share, therefore, should be near-zero, and a `minPrice` algorithm would indeed report that answer as it would use the price of Token A as a worst-case estimate. But the `averagePrice` method would return a price near half of Token B's price  $((\sim 0 + \text{priceB}) / 2)$ , which is almost completely irrelevant because the pool is so lopsided toward Token A.

This would be extremely incorrect and would defeat the purpose of having a robust oracle price in the first place. If the oracle cannot withstand a depeg event, then we might as well use the pool's own virtual price (`getRate()`) directly rather than bothering with an oracle (not a good idea).

**Recommendation:** Use the `minPrice` algorithm instead of an `averagePrice` algorithm. Note that prices must first be normalized by their `RateProvider` rates or else the min-value comparisons will be meaningless, but that is a separate finding.

### 3.2.39 `FeeAccumulator.multiSwap` may transfer too much `CompoundFee`

*Submitted by [yttriumzz](#), also found by [kaka](#)*

**Severity:** Medium Risk

**Context:** `FeeAccumulator.sol#L248`

**Description:** `FeeAccumulator.multiSwap` allows Harvester to swap other earning tokens for `FeeToken`. A certain proportion of this `feeToken` will be transferred to compounding vaults.

However, when `multiSwap` transfers, it uses `IERC20(feeToken).balanceOf(address(this))` as the base instead of the `FeeToken` got from this swap. This is wrong because there may already be some `FeeTokens` in the contract (for example, interfaces such as `executeOTC` will generate some `FeeTokens`).

For users, rewards for compounding vaults have increased while `VeCVE` locking rewards have decreased.

**Recommendation:**

```
function multiSwap(
 bytes calldata data,
 address[] calldata tokens
) external nonReentrant {
 if (!centralRegistry.isHarvester(msg.sender)) {
 revert FeeAccumulator__Unauthorized();
 }

 SwapperLib.Swap[] memory swapDataArray = abi.decode(
 data,
 (SwapperLib.Swap[])
);

 uint256 numTokens = swapDataArray.length;
 if (numTokens != tokens.length) {
 revert FeeAccumulator__SwapDataAndTokenLengthMismatch(
 numTokens,
 tokens.length
);
 }
 address currentToken;

+ uint256 balanceBefore = IERC20(feeToken).balanceOf(address(this));
 for (uint256 i; i < numTokens; ++i) {
 currentToken = tokens[i];
 // Make sure we are not earmarking this token for DAO OTC.
 if (rewardTokenInfo[currentToken].forOTC == 2) {
 continue;
 }
 if (rewardTokenInfo[currentToken].isRewardToken != 2) {
 revert FeeAccumulator__SwapDataCurrentTokenIsNotRewardToken(
 i,
 currentToken
);
 }
 }
}
```



```

 if (swapDataArray[i].inputToken != currentToken) {
 revert FeeAccumulator__SwapDataInputTokenIsNotCurrentToken(
 i,
 swapDataArray[i].inputToken,
 currentToken
);
 }
 if (swapDataArray[i].outputToken != feeToken) {
 revert FeeAccumulator__SwapDataOutputTokenIsNotFeeToken(
 i,
 swapDataArray[i].outputToken,
 feeToken
);
 }
 if (!centralRegistry.isSwapper(swapDataArray[i].target)) {
 revert FeeAccumulator__SwapDataInvalidSwapper(
 i,
 swapDataArray[i].target
);
 }

 // Swap from token to output token (fee token).
 // Note: Because this is ran directly from Gelato Network we know
 // we will not have a malicious actor on swap routing.
 // We route liquidity to 1Inch with tight slippage
 // requirement, meaning we do not need to separately check
 // for slippage here.
 SwapperLib.swap(centralRegistry, swapDataArray[i]);
 }

 SafeTransferLib.safeTransfer(
 feeToken,
 oneBalanceFeeManager,
 - (IERC20(feeToken).balanceOf(address(this)) * vaultCompoundFee()) /
 + (IERC20(feeToken).balanceOf(address(this) - balanceBefore) * vaultCompoundFee()) /
 vaultYieldFee()
);
}

```

### 3.2.40 FeeAccumulator.executeEpochFeeRouter repeats payment of CompoundFee

Submitted by [yttriumzz](#), also found by [tvdung94](#)

**Severity:** Medium Risk

**Context:** [FeeAccumulator.sol#L777-L782](#)

**Description:** FeeAccumulator.executeEpochFeeRouter will transfer the FeeToken to CVELocker as a reward for the next epoch. Prior to this, a portion of FeeToken will be transferred to compounding vaults. However, these FeeTokens have already been distributed to compounding vaults when they are transferred to FeeAccumulator. For example, the following code.

```

///// https://github.com/curvance/Curvance-CantinaCompetition/blob/c01e6bda82a60ac4842bf0d3363d63e89c870758/co
↪ ntracts/architecture/FeeAccumulator.sol#L245-L250
SafeTransferLib.safeTransfer(
 feeToken,
 oneBalanceFeeManager,
 (IERC20(feeToken).balanceOf(address(this)) * vaultCompoundFee()) /
 vaultYieldFee()
);

```

That is, FeeAccumulator.executeEpochFeeRouter repeatedly paid reward tokens to compounding vaults. For users, rewards for compounding vaults have increased while VeCVE locking rewards have decreased.

**Recommendation:**

```

uint256 feeTokenBalance = IERC20(feeToken).balanceOf(address(this));

- // In terms of funds inside fee accumulator, 1/16 or 6.25% of fee token
- // should be sent and deposited to Gelato 1Balance on polygon.
- SafeTransferLib.safeTransfer(
- feeToken,
- oneBalanceFeeManager,
- (feeTokenBalance * vaultCompoundFee()) /
- centralRegistry.protocolHarvestFee()
-);
-
- feeTokenBalance = IERC20(feeToken).balanceOf(address(this));
-
uint256 chainId;
uint256 feeTokenBalanceForChain;

```

### 3.2.41 Liquidators are not incentivized to socialize bad debt since `liquidate` is always cheaper than `liquidateAccount`

Submitted by JCN

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** When an account becomes underwater, liquidators are incentivized to liquidate that account via the `liquidateAccount` function. Via this function the liquidator will pay down part of the debt, receive all the account's collateral, and then socialize the remaining bad debt. At the end of this function call the account should be empty. However, liquidators also have the option of using the `liquidate` function when liquidating underwater accounts. Via this function the liquidator will pay down as much debt as possible in order to liquidate as much of the account's collateral as possible. No bad debt is socialized via this function call since the intention is to have bad debt handled via `liquidateAccount`.

The following equations are used when liquidating an underwater account:

Equation used during `DToken::liquidate`:

$$\text{debtAmountToPay} = \frac{\text{collateralValue}}{(1 + \text{liqIncHard}) \cdot \text{debtTokenPrice}}$$

Equation used during `MarketManager::liquidateAccount`:

$$\text{debtAmountToPay} = \frac{\text{collateralValue}}{(1 + \text{liqIncSoft}) \cdot \text{debtTokenPrice}}$$

Below is the relationship between `liqIncSoft` and `liqIncHard`:

- `MarketManager::updateCollateralToken`:

```

988: if (liqIncSoft >= liqIncHard) { // @audit: liqIncSoft always smaller than liqIncHard
989: _revert(_INVALID_PARAMETER_SELECTOR);
990: }

```

Based on the above information, a liquidator will always pay less when liquidating an underwater account via `liquidate` versus when using `liquidateAccount` since `liqIncSoft` will always be smaller than `liqIncHard`. The greater the difference between these two values, the more of a discount the liquidator will receive via `liquidate`.

I will now step through each function to show how we can derive the equations above.

`DToken::liquidate`: The `liquidate` function has the following call path:

`DToken_liquidate` → `MarketManager::canLiquidateWithExecution` → `MarketManager::_canLiquidate`, with the main calculations occurring in the `_canLiquidate` internal function:

- `MarketManager::_canLiquidate`:

```

1559: LiqData memory data = _LiquidationStatusOf(// @audit: calculates `lFactor`
1560: account,
1561: debtToken,
1562: collateralToken
1563:);

```

As shown above, the data.lFactor is calculated in the \_LiquidationStatusOf internal function:

- LiquidationManager::\_LiquidationStatusOf:

```

550: if (accountDebt >= accountCollateralHard) {
551: result.lFactor = WAD;
552: return result;
553: }

```

An underwater account will always undergo a hard liquidation since the account debt is greater than the total collateral value of the account. Therefore, the lFactor will be 1e18.

Below are the main calculations for determining how many debt tokens the liquidator will pay:

- MarketManager::\_canLiquidate:

```

1573: uint256 cFactor = cToken.baseCFactor +
1574: ((cToken.cFactorCurve * data.lFactor) / WAD);
1575: uint256 incentive = cToken.liqBaseIncentive +
1576: ((cToken.liqCurve * data.lFactor) / WAD);
1577: maxAmount =
1578: (cFactor * IMToken(debtToken).debtBalanceCached(account)) /
1579: WAD;
1580:
1581: // Get the exchange rate, and calculate the number of
1582: // collateral tokens to seize.
1583: debtToCollateralRatio =
1584: (incentive * data.debtTokenPrice * WAD) /
1585: (data.collateralTokenPrice *
1586: IMToken(collateralToken).exchangeRateCached());
1587: }
1588:
1589: // If they want to liquidate an exact amount, liquidate `debtAmount`,
1590: // otherwise liquidate the maximum amount possible.
1591: if (!liquidateExact) { // @audit: liquidators will want to liquidate the max amount so
 ↳ `liquidateExact = false`
1592: debtAmount = maxAmount;
1593: }
1594:
1595: // Adjust decimals if necessary.
1596: uint256 amountAdjusted = (debtAmount *
1597: (10 ** IERC20(collateralToken).decimals())) /
1598: (10 ** IERC20(debtToken).decimals());
1599: // Calculate how many cTokens should be liquidated.
1600: uint256 liquidatedTokens = (amountAdjusted * debtToCollateralRatio) /
1601: WAD;
1602:
1603: // Cache `account`s collateral posted of `collateralToken`.
1604: uint256 collateralAvailable = cToken
1605: .accountPositions[account]
1606: .collateralPosted;
1607: // If the user wants to liquidate an exact amount, make sure theres
1608: // enough collateral available to liquidate,
1609: // otherwise liquidate as much as possible.
1610: if (liquidateExact) { // @audit: liquidates will want to liquidate max amount so `liquidateExact =
 ↳ `false`
1611: if (
1612: debtAmount > maxAmount ||
1613: liquidatedTokens > collateralAvailable
1614:) {
1615: // Make sure that the liquidation limit,
1616: // and collateral posted >= amount.
1617: _revert(_INVALID_PARAMETER_SELECTOR);
1618: }
1619: } else {
1620: if (liquidatedTokens > collateralAvailable) {
1621: debtAmount =
1622: (debtAmount * collateralAvailable) /

```

```

1623: liquidatedTokens;
1624: liquidatedTokens = collateralAvailable;

```

For underwater liquidations, the `cFactor`, computed on lines 1573 - 1574, will always be equal to `1e18` since the `baseCFactor + cFactorCurve` is always equal to `1e18`:

- `MarketManager::updateCollateralToken`:

```

1056: marketToken.baseCFactor = baseCFactor;
1057: // Store the distance between base cFactor and 100%,
1058: // that way we can quickly scale between [base, 100%] based on lFactor.
1059: marketToken.cFactorCurve = WAD - baseCFactor;

```

The `incentive`, computed on lines 1575 - 1576, is defined as follows:

$$\text{incentive} = \text{liqBaseIncentive} + \text{liqCurve}$$

The `maxAmount`, computed on lines 1577 - 1579, is simply the `debtBalanceCached(account)` (total debt amount), since the `cFactor` is equal to `1e18`.

The `debtToCollateralRatio`, computed on lines 1583 - 1586, is defined as follows:

$$\text{debtToCollateralRatio} = \frac{\text{incentive} \cdot \text{debtTokenPrice}}{\text{collateralTokenPrice} \cdot \text{collateralExchangeRate}}$$

`debtAmount` is set equal to `maxAmount (debtBalanceCached(account))` on lines 1592. Lines 1596 - 1600 will adjust the `debtAmount` to the collateral token's decimals and then compute the amount of tokens to liquidate. For demonstration purposes, let's assume the collateral token and debt token have the same decimals:

$$\text{liquidatedTokens} = \text{debtAmount} \cdot \text{debtToCollateralRatio}$$

This can be expanded to

$$\text{liquidatedTokens} = \text{debtAmount} \cdot \frac{\text{incentive} \cdot \text{debtTokenPrice}}{\text{collateralTokenPrice} \cdot \text{collateralExchangeRate}}$$

Which can be rewritten as

$$\text{liquidatedTokens} = \frac{\text{incentive} \cdot (\text{debtTokenPrice} \cdot \text{debtAmount})}{\text{collateralTokenPrice} \cdot \text{collateralExchangeRate}}$$

And simplified to

$$\text{liquidatedTokens} = \frac{\text{incentive} \cdot \text{debtValue}}{\text{collateralSharePrice}}$$

Where `debtValue` is the total value of the account's debt and `collateralSharePrice` is the value of a collateral share.

Since the account is underwater, the `debtValue` is inflated with respect to the `collateralSharePrice`. Thus, `liquidatedTokens` will be greater than the actual amount of collateral shares that the account has posted.

From line 1610, the second branch will execute since the liquidator is trying to liquidate the maximum amount via `liquidate`. Since `liquidatedTokens` is greater than `collateralAvailable`, the `debtAmount` is adjusted.

Therefore,

$$\text{debtAmount} = \frac{\text{debtAmount} * \text{collateralAvailable}}{\text{liquidatedTokens}}$$

Which can be expanded to

$$\text{debtAmount} = \frac{\text{debtAmount} \cdot \text{collateralAvailable}}{\text{debtAmount} \cdot \text{debtToCollateralRatio}}$$

The `debtAmount` values cancel out

$$\text{debtAmount} = \frac{\text{collateralAvailable}}{\text{debtToCollateralRatio}}$$

This can be expanded to

$$\text{debtAmount} = \frac{\text{collateralAvailable}}{\left( \frac{\text{incentive} \cdot \text{debtTokenPrice}}{\text{collateralTokenPrice} \cdot \text{collateralExchangeRate}} \right)}$$

Which can be rewritten as:

$$\text{debtAmount} = \frac{\text{collateralAvailable} \cdot \text{collateralTokenPrice} \cdot \text{collateralExchangeRate}}{\text{incentive} \cdot \text{debtTokenPrice}}$$

The numerator evaluates to the `collateralValue` of the account:

$$\text{debtAmount} = \frac{\text{collateralValue}}{\text{incentive} \cdot \text{debtTokenPrice}}$$

The equation for the `incentive` can be expanded given the following information:

- `MarketManager::updateCollateralToken:`

```
1050: marketToken.liqCurve = liqIncHard - liqIncSoft;
1051: // We use the liquidation incentive values as a premium in
1052: // `calculateLiquidatedTokens`, so it needs to be 1 + incentive.
1053: marketToken.liqBaseIncentive = WAD + liqIncSoft;
```

Based on the above,

$$\text{liqBaseIncentive} = 1 + \text{liqIncSoft}$$

$$\text{liqCurve} = \text{liqIncHard} - \text{liqIncSoft}$$

Now we can evaluate `incentive` as

$$\text{incentive} = (1 + \text{liqIncSoft}) + (\text{liqIncHard} - \text{liqIncSoft})$$

`liqIncSoft` values cancel out

$$\text{incentive} = 1 + \text{liqIncHard}$$

So now the equation for `debtAmount` can finally be rewritten as:

$$\text{debtAmount} = \frac{\text{collateralValue}}{(1 + \text{liqIncHard}) \cdot \text{debtTokenPrice}}$$

`MarketManager::liquidateAccount:` The `liquidateAccount` function invokes the `_BadDebtTermsOf` internal function, which is where the value of debt the liquidator will repay is calculated:

- `MarketManager::liquidateAccount:`

```
768: (
769: BadDebtData memory data,
770: uint256[] memory assetBalances
771:) = _BadDebtTermsOf(account);
```

- `LiquidationManager::_BadDebtTermsOf:`

```
617: result.debtToPay +=
618: (collateralValue * WAD) /
619: tokenData[snapshot.asset].liqBaseIncentive;
```

Based on the above,

$$\text{debtToPay} = \frac{\text{collateralValue}}{\text{liqBaseIncentive}}$$

Execution continues back in the `liquidateAccount` function in order to determine the ratio of debt tokens that the liquidator will pay:

- `MarketManager::liquidateAccount:`

```
778: uint256 repayRatio = (data.debtToPay * WAD) / data.debt;
 // ...
805: mToken.repayWithBadDebt(msg.sender, account, repayRatio);
```

Based on the above,

$$\text{repayRatio} = \frac{\text{debtToPay}}{\text{debtValue}}$$

Finally, the `repayWithBadDebt` function is invoked in the `DToken(s)` in question and the `repayRatio` is used to determine exactly how many debt tokens to pull from the liquidator:

- `DToken::repayWithBadDebt:`

```
411: uint256 accountDebt = debtBalanceCached(account);
412: uint256 repayAmount = (accountDebt * repayRatio) / WAD;
```

Based on the above,

$$\text{repayAmount} = \text{accountDebt} \cdot \text{repayRatio}$$

`repayRatio` can be expanded

$$\text{repayAmount} = \text{accountDebt} \cdot \frac{\text{debtToPay}}{\text{debtValue}}$$

We can rewrite the equation as

$$\text{repayAmount} = \text{debtToPay} \cdot \frac{\text{accountDebt}}{\text{debtValue}}$$

`debtValue` is the total value of the account's debt. Which can be evaluated as:

$$\text{debtValue} = \text{accountDebt} \cdot \text{debtTokenPrice}$$

Therefore,

$$\text{repayAmount} = \text{debtToPay} \cdot \frac{\text{accountDebt}}{\text{accountDebt} \cdot \text{debtTokenPrice}}$$

The `accountDebt` values cancel out

$$\text{repayAmount} = \text{debtToPay} \cdot \frac{1}{\text{debtTokenPrice}}$$

We can expand `debtToPay`

$$\text{repayAmount} = \frac{\text{collateralValue}}{\text{liqBaseIncentive}} \cdot \frac{1}{\text{debtTokenPrice}}$$

We can rewrite as

$$\text{repayAmount} = \frac{\text{collateralValue}}{\text{liqBaseIncentive} \cdot \text{debtTokenPrice}}$$

We can now expand `liqBaseIncentive` to receive our final equation:

$$\text{repayAmount} = \frac{\text{collateralValue}}{(1 + \text{liqIncSoft}) \cdot \text{debtTokenPrice}}$$

Lastly, we will notice that when the `liquidateAccount` function is called, the remaining debt (bad debt) is socialized by decrementing the `totalBorrows` by the entire account's debt:

- `DToken::repayWithBadDebt`:

```
429: delete _debtOf[account].principal;
430: totalBorrows -= accountDebt;
```

However, the `liquidate` function does not implement this mechanism and also does not validate that the account is healthy post liquidation:

- `DToken::liquidate`:

```
1377: _debtOf[account].principal = accountDebt - amount;
1378: _debtOf[account].accountExchangeRate = marketData.exchangeRate;
1379: totalBorrows -= amount;
1380:
1381: emit Repay(liquidator, account, amount);
1382:
1383: // ...
1386: collateralToken.seize(
1387: liquidator,
1388: account,
1389: liquidatedTokens,
1390: protocolTokens
1391:);
```

Therefore, the bad debt is not socialized during the `liquidate` call and the account is left with bad debt and no collateral. Additionally, since the liquidator is paying back less debt during this `liquidate` call (compared to the `liquidateAccount` call), the amount of bad debt that remains in the account will be greater than the amount of bad debt that should have been created via `liquidateAccount`:

```
badDebt_created_from_liquidateAccount = badDebt

badDebt_created_from_liquidate = badDebt + discount
```

**Impact:** Liquidators are not incentivized to call the `liquidateAccount` function in order to socialize bad debt for underwater accounts since `liquidate` is always cheaper. When liquidators call the `liquidate` function instead they will receive all of the account's collateral and the account will be left with the expected bad debt plus the discount that the liquidator received for using the `liquidate` function instead of `liquidateAccount`.

Therefore, every underwater account will produce an inflated amount of bad debt when liquidated. After this occurs, liquidators will no longer be incentivized to socialize the bad debt (plus discount) remaining in the account post-liquidation, since the account in question no longer has any collateral. If no one calls `liquidateAccount` for the account in this state, then bad debt will go untreated and the protocol will be at risk of insolvency. Anyone who calls `liquidateAccount` in this state will call it at a loss since they will incur gas costs and receive no collateral in return. Depending on which chain this occurs on, the gas costs can be a non-negligible amount.

Finally, when `liquidateAccount` is called in this state, an inflated amount of bad debt will be socialized, which results in a greater loss for lenders.

**Proof of concept:** Add the following test inside of the `./tests/` directory and run with `forge --mc UnderwaterLiquidationPoC -vvv`:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.17;

import { MockDataFeed } from "contracts/mocks/MockDataFeed.sol";
import { TestBaseMarket } from "tests/market/TestBaseMarket.sol";
import { IMToken } from "contracts/interfaces/market/IMToken.sol";

import "forge-std/console.sol";

contract UnderwaterLiquidationPoC is TestBaseMarket {
 MockDataFeed public mockUsdcFeed;
 MockDataFeed public mockWethFeed;
 MockDataFeed public mockRethFeed;

 uint256 liqIncSoft = 200;
 uint256 liqIncHard = 400;

 function setUp() public virtual override {
 super.setUp();

 // use mock pricing for testing
 mockUsdcFeed = new MockDataFeed(_CHAINLINK_USDC_USD);
 chainlinkAdaptor.addAsset(
 _USDC_ADDRESS,
 address(mockUsdcFeed),
 0,
 true
);
 dualChainlinkAdaptor.addAsset(
 _USDC_ADDRESS,
 address(mockUsdcFeed),
 0,
 true
);
 mockWethFeed = new MockDataFeed(_CHAINLINK_ETH_USD);
 chainlinkAdaptor.addAsset(
 _WETH_ADDRESS,
 address(mockWethFeed),
 0,
 true
);
 dualChainlinkAdaptor.addAsset(
 _WETH_ADDRESS,
 address(mockWethFeed),
 0,
 true
);
 mockRethFeed = new MockDataFeed(_CHAINLINK_RETH_ETH);
 chainlinkAdaptor.addAsset(
 _RETH_ADDRESS,
 address(mockRethFeed),
 0,
 false
);
 dualChainlinkAdaptor.addAsset(
 _RETH_ADDRESS,
 address(mockRethFeed),
 0,
 false
);

 gaugePool.start(address(marketManager));
 vm.warp(gaugePool.startTime());
 vm.roll(block.number + 1000);

 chainlinkEthUsd.updateAnswer(1500e8);
 mockUsdcFeed.setMockUpdatedAt(block.timestamp);
 mockWethFeed.setMockUpdatedAt(block.timestamp);
 mockRethFeed.setMockUpdatedAt(block.timestamp);

 _prepareUSDC(address(this), _ONE);
 usdc.approve(address(dUSDC), _ONE);
 marketManager.listToken(address(dUSDC));
 }
}
```



```

_prepareBALRETH(address(this), _ONE);
balRETH.approve(address(cBALRETH), _ONE);

marketManager.listToken(address(cBALRETH));
marketManager.updateCollateralToken(
 IMToken(address(cBALRETH)),
 7000,
 4000,
 3000,
 liqIncSoft,
 liqIncHard,
 0,
 1000
);
address[] memory tokens = new address[](1);
tokens[0] = address(cBALRETH);
uint256[] memory caps = new uint256[](1);
caps[0] = 100_000e18;
marketManager.setCTokenCollateralCaps(tokens, caps);
}

function testLiquidateUnderwaterAccountViaLiquidate() public {
 // underwater liquidation via `liquidate`
 _liquidateUnderwaterAccount(false);
}

function testLiquidateUnderwaterAccountViaLiquidateAccount() public {
 // underwater liquidation via `liquidateAccount`
 _liquidateUnderwaterAccount(true);
}

function _liquidateUnderwaterAccount(bool socializeBadDebt) internal {
 address borrower = address(0x01010101);
 address liquidator = address(0x02020202);

 // supply funds in market
 deal(address(usdc), address(dUSDC), 3_000_000e6);

 // borrower deposits collateral
 uint256 collateralAmt = 1_000 ether;
 _prepareBALRETH(borrower, collateralAmt);
 vm.startPrank(borrower);
 balRETH.approve(address(cBALRETH), collateralAmt);
 cBALRETH.depositAsCollateral(collateralAmt, borrower);
 vm.stopPrank();

 // borrower borrows usdc
 console.log("--- Borrower creates debt position ---");
 uint256 borrowAmt = 1_000_000e6;
 vm.startPrank(borrower);
 dUSDC.borrow(borrowAmt);
 vm.stopPrank();
 _getStatus(borrower);

 console.log("");
 console.log("--- Borrower's position becomes unhealthy ---");
 //mockUsdcFeed.setMockAnswer(200000000); // borrowed asset increases in value
 mockWethFeed.setMockAnswer(1000e8); // ETH price drops
 mockRethFeed.setMockAnswer(1000e8);
 (uint256 collateralValue, uint256 debtValue) = _getStatus(borrower);

 // liquidator liquidates borrower
 uint256 liquidatorStartingBal = 10_000_000e6;
 uint256 cBALRETHBalBefore = cBALRETH.balanceOf(liquidator);
 _prepareUSDC(liquidator, liquidatorStartingBal);

 console.log("");
 vm.startPrank(liquidator);
 usdc.approve(address(dUSDC), liquidatorStartingBal);

 if (socializeBadDebt) {
 console.log("--- Liquidator liquidates borrower via `liquidateAccount` ---");
 marketManager.liquidateAccount(borrower);
 } else {
 console.log("--- Liquidator liquidates borrower via `liquidate` ---");
 }
}

```

```

 dUSDC.liquidate(borrower, IMToken(address(cBALRETH)));
 }

 vm.stopPrank();

 uint256 debtPaid = liquidatorStartingBal - usdc.balanceOf(liquidator);
 uint256 cBALRETHBalAfter = cBALRETH.balanceOf(liquidator);

 emit log_named_uint("Amount Liquidator paid in USDC", debtPaid);
 emit log_named_uint("Amount Liquidator received in cToken shares", cBALRETHBalAfter -
 ↪ cBALRETHBalBefore);

 console.log("");
 console.log("--- Borrower account status ---");
 _getStatus(borrower);

 console.log("");
 console.log("--- DToken exchange rate and total borrows ---");
 emit log_named_uint("exchangeRate", dUSDC.exchangeRateCached());
 emit log_named_uint("totalBorrows", dUSDC.totalBorrows());

 // log how much bad debt was created post-liquidation
 (uint256 debtTokenPrice,) = oracleRouter.getPrice(address(usdc), true, false);
 uint256 debtPaidInUSD = (debtPaid * debtTokenPrice) / 1e6;
 console.log("");
 emit log_named_uint("Amount of bad debt created in USD", debtValue - debtPaidInUSD);

 // compare actual debtPaid versus expected debtPaid from equations in report
 if (socializeBadDebt) {
 // equation: collateralValue / (1 + liqIncSoft) * debtTokenPrice
 uint256 expectedDebtPaid = collateralValue * 1e18 * 1e6 / ((1e18 + (liqIncSoft * 1e14)) *
 ↪ debtTokenPrice);
 assertEq(expectedDebtPaid, debtPaid);
 } else {
 // equation: collateralValue / (1 + liqIncHard) * debtTokenPrice
 uint256 expectedDebtPaid = collateralValue * 1e18 * 1e6 / ((1e18 + (liqIncHard * 1e14)) *
 ↪ debtTokenPrice);
 assertEq(expectedDebtPaid, debtPaid);
 }
}

function _getStatus(address account) internal returns (uint256 collateralValue, uint256 debtValue) {
 (
 collateralValue,
 debtValue
) = marketManager.solvencyOf(account);

 emit log_named_uint("total collateral", collateralValue);
 emit log_named_uint("total debt", debtValue);
}
}

```

Logs from the above tests:

```

[PASS] testLiquidateUnderwaterAccountViaLiquidate() (gas: 2387960)
Logs:
--- Borrower creates debt position ---
total collateral: 1705260154818651337913000
total debt: 1000064150000000000000000

--- Borrower's position becomes unhealthy ---
total collateral: 512717863574320963497000
total debt: 1000064150000000000000000

--- Liquidator liquidates borrower via `liquidate` ---
Amount Liquidator paid in USDC: 492966321954
Amount Liquidator received in cToken shares: 1000000000000000000000000

--- Borrower account status ---
total collateral: 0
total debt: 507066204256446650900000

--- DToken exchange rate and total borrows ---
exchangeRate: 71311416957854952577907723
totalBorrows: 507033678046

```

```

Amount of bad debt created in USD: 507066204256446650900000

[PASS] testLiquidateUnderwaterAccountViaLiquidateAccount() (gas: 2468957)
Logs:
--- Borrower creates debt position ---
total collateral: 1705260154818651337913000
total debt: 10000641500000000000000000

--- Borrower's position becomes unhealthy ---
total collateral: 512717863574320963497000
total debt: 10000641500000000000000000

--- Liquidator liquidates borrower via `liquidateAccount` ---
Amount Liquidator paid in USDC: 502632328267
Amount Liquidator received in cToken shares: 10000000000000000000000000

--- Borrower account status ---
total collateral: 0
total debt: 0

--- DToken exchange rate and total borrows ---
exchangeRate: 59488752484418455394708692
totalBorrows: 0

Amount of bad debt created in USD: 497399577869141671950000

```

Note that the account in the above test is large, with a debt value of \$1\_000\_064 and a collateral value of \$512\_717.

Based on the above logs, we can see that the liquidator paid ~\$9\_666 less when liquidating the account via the `liquidate` function. Additionally, we can observe that the `liquidateAccount` function creates \$497\_399 worth of bad debt, while the `liquidate` function creates \$507\_066, which includes the discount of \$9\_666.

Thus, the liquidator paid 50.26% of the total debt when using the `liquidateAccount` function and paid 49.29% of the total debt when using the `liquidate` function. In both cases the liquidator received all of the account's collateral, but received a 0.96% discount when using `liquidate`. Note that this discount is in proportion to the difference between the `liqIncHard` and `liqIncSoft` collateral values for the collateral in question.

**Recommendation:** A possible mitigation for this issue is to only allow `liquidate` to be called for accounts that are *not* underwater. I.e. if `debtValue > collateralValue`, then the call can revert early.

Below is an example implementation (not optimized):

```

diff --git a/./contracts/market/LiquidityManager.sol b/./contracts/market/LiquidityManager.sol
index 0ee505f4..78bb6d9c 100644
--- a/./contracts/market/LiquidityManager.sol
+++ b/./contracts/market/LiquidityManager.sol
@@ -502,6 +502,8 @@ abstract contract LiquidityManager {
 uint256 accountCollateralHard;
 // Current outstanding account debt.
 uint256 accountDebt;
+ // Current total collateral value
+ uint256 accountTotalCollateral;

 for (uint256 i; i < numAssets; ++i) {
 snapshot = snapshots[i];
@@ -515,13 +517,15 @@ abstract contract LiquidityManager {
 if (tokenData[snapshot.asset].collRatio != 0) {
 (
 accountCollateralSoft,
 accountCollateralHard,
+ accountCollateralHard,
+ accountTotalCollateral
) = _addLiquidationValues(
 snapshot,
 account,
 underlyingPrices[i],
 accountCollateralSoft,
 accountCollateralHard,
+ accountCollateralHard,
+ accountTotalCollateral
);

```

```

 }
 } else {
@@ -541,6 +545,11 @@ abstract contract LiquidityManager {
 }
}

+ // Revert if account is currently underwater
+ if (accountDebt > accountTotalCollateral) {
+ revert("Account Underwater");
+ }
+
+ // Indicates no liquidation.
+ if (accountCollateralSoft >= accountDebt) {
+ return result;
@@ -746,8 +755,9 @@ abstract contract LiquidityManager {
 address account,
 uint256 price,
 uint256 softSumPrior,
- uint256 hardSumPrior
-) internal view returns (uint256, uint256) {
+ uint256 hardSumPrior,
+ uint256 totalCollateralPrior
+) internal view returns (uint256, uint256, uint256) {
 uint256 assetValue = _assetValue(
 ((tokenData[snapshot.asset].accountPositions[account].collateralPosted *
 snapshot.exchangeRate) / WAD),
@@ -758,7 +768,8 @@ abstract contract LiquidityManager {
 return (
 softSumPrior +
 (assetValue / tokenData[snapshot.asset].collReqSoft),
- hardSumPrior + (assetValue / tokenData[snapshot.asset].collReqHard)
+ hardSumPrior + (assetValue / tokenData[snapshot.asset].collReqHard),
+ totalCollateralPrior + (assetValue / 1e18)
);
}

```

### 3.2.42 FeeAccumulator.migrateFeeAccumulator did not transfer native token

Submitted by *yttriumzz*, also found by *Haxatron* and *another*

**Severity:** Medium Risk

**Context:** FeeAccumulator.sol#L491

**Description:** When the FeeAccumulator is updated, the migrateFeeAccumulator function of the old contract can be called to transfer all tokens to the new contract. However, the migrateFeeAccumulator function does not transfer the native token.

```

///// https://github.com/curvature/Curvature-CantinaCompetition/blob/c01e6bda82a60ac4842bf0d3363d63e89c870758/co
↪ ntracts/architecture/FeeAccumulator.sol#L40-L44
/// Native gas tokens are stored inside the contract to pay for all
/// crosschain actions. Locked token data actions are intended to be
/// moved over to Wormhole's CCQ prior to mainnet deployment.
/// At this time, payload/MessageType configuration + encoding/decoding
/// are not production ready.

```

Please see the documentation of the FeeAccumulator contract, which states that native tokens are stored in the contract for payment of cross-chain operations. Specifically, the behavior of sendWormholeMessages and executeEpochFeeRouter functions.

If the native tokens are not transferred during the migrateFeeAccumulator, these native tokens will be stuck in the old contract.

#### Recommendation:

```

function migrateFeeAccumulator() external {
 address newFeeAccumulator = centralRegistry.feeAccumulator();
 if (newFeeAccumulator == address(this)) {
 revert FeeAccumulator__NewFeeAccumulatorIsNotChanged();
 }

 address[] memory currentRewardTokens = rewardTokens;
 uint256 numTokens = currentRewardTokens.length;
 uint256 tokenBalance;

```

```

// Send remaining fee tokens to new fee accumulator, if any.
for (uint256 i; i < numTokens;) {
 tokenBalance = IERC20(currentRewardTokens[i]).balanceOf(
 address(this)
);

 if (tokenBalance > 0) {
 SafeTransferLib.safeTransfer(
 currentRewardTokens[i],
 newFeeAccumulator,
 tokenBalance
);
 }

 unchecked {
 ++i;
 }
}

tokenBalance = IERC20(feeToken).balanceOf(address(this));

// Send remaining fee token to new fee accumulator, if any.
if (tokenBalance > 0) {
 SafeTransferLib.safeTransfer(
 feeToken,
 newFeeAccumulator,
 tokenBalance
);
}

+
+ // just a demo, you can use `call` or `send` function :)
+ newFeeAccumulator.transfer(address(this).balance);
}

```

### 3.2.43 Convex2PoolCToken pool info's struct implementaion is completely broken on Arbitrum/Polygon

Submitted by [Bauchibred](#), also found by [Bauer](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Proof of concept:** First note that protocol has been hinted to be deployed on Eth/Arbitrum/Optimism/Base/Polygon ZkEVM/Blast minimum, and then later on, on chains like monad/berachain confirmed [here](#) and in the [docs](#).

Now, take a look how the Convex pool configuration data is being queried.

- [Convex2PoolCToken.sol#L83-L86](#)

```

(address pidToken, , , address crvRewards, , bool shutdown) = IBooster(
 booster_
).poolInfo(strategyData.pid);

```

This is from the constructors of the 2pool Convex tokens and as the name suggests this implementation is used to query the Convex pool cofiguration data, problem however is that these functionalities would be broken on some chains, see below.

From Convex's docs on the sidechain implementation [here](#), note that Convex lists both Arbitrum and Polygon under it's side chains as can be seen [here](#) indicating that his is directly impacting this protocol.

Now back to the [Convex's official docs](#), we can see that on side chains, unlike mainnet, the PoolInfo struct has been slightly changed, i.e it was changed from:

```
//mainnet
struct PoolInfo {
 address lptoken;
 address token;
 address gauge;
 address crvRewards;
 address stash;
 bool shutdown;
}
```

To:

```
//sidechains
struct PoolInfo {
 address lptoken;
 address gauge;
 address rewards;
 bool shutdown;
 address factory;
}
```

Now from how the struct is queried, i.e (address pidToken, , , address crvRewards, , bool shutdown) we can see that protocol assumes this is going to be valid for all chains it's to be deployed on, i.e:

- The shutdown bool is attached as the last element in the struct this is only valid for the mainnet version.
- The crvRewards bool is attached as the fourth element in the struct this is only valid for the mainnet version, as it's the 3rd on the sidechain's struct.

This would end up completely breaking the integration, since on the sidechains.

#### Impact:

Impact: High - Inability to correctly integrate on sidechains. Likelihood: High/Medium, this protocol is going to get deployed on two of the current sidechains that Convex supports (*Arbitrum & Polygon*).

Convex integration on withdrawing rewards will be completely broken on Arbitrum/Polygon due to unaccounted for implementation differences by protocol

**Recommendation:** Consider creating a separate contract for Convex2PoolCToken for Arbitrum/Polygon that correctly accounts for the change in the booster's pool info struct.

### 3.2.44 CVE bridge will be replayed when chain hard fork

Submitted by [pks271](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** When there's a hard fork of the chain, CVE/VeCVE bridge functions like [CVE#bridge](#), [Child-CVE#bridge](#) and [VeCVE#bridgeVeCVELock](#) can be replayed in the hard fork chain because the functions don't consider whether the chain id has changed or not.

Although the possibility is low, but the effect is enormous, malicious users can gain lots of profit due to this issue. See the omni bridge [vulnerability](#) with Ethereum's transition to the PoS consensus.

**Recommendation:** Consider checking if chain id is the same as the source chain id, and reverted the bridge transaction if chain is not the same, as suggested by wormhole's NttManager solution [here](#).

### 3.2.45 Utilization ratio Can Be Manipulated Above 100%

Submitted by [pks271](#), also found by [kankodu](#), [oceansky](#) and [Bernd Artmüller](#)

**Severity:** Medium Risk

**Context:** [DToken.sol#L977-L1041](#), [DToken.sol#L812-L819](#)

**Description:** In A low liquidity market, UtilizationRatio can be manipulated above 100%.

When borrowing from DToken, the utilization rate [calculation](#) based on `marketUnderlyingHeld`, `totalBorrows` and `totalReserves`, the latter two parameters are updated when `DToken#accrueInterest` function is called:

```
function accrueInterest() public {
 // ...
 // Calculate the interest compound cycles to update,
 // in `interestCompounds`. Rounds down natively.
 uint256 interestCompounds = (block.timestamp -
 cachedData.lastTimestampUpdated) / cachedData.compoundRate;
 // Calculate the interest and debt accumulated.
 uint256 interestAccumulated = borrowRate * interestCompounds;
 uint256 debtAccumulated = (interestAccumulated * borrowsPrior) / WAD;
 // Calculate new borrows, and the new exchange rate, based on
 // accumulation values above.
 uint256 totalBorrowsNew = debtAccumulated + borrowsPrior;
 uint256 exchangeRateNew = ((interestAccumulated * exchangeRatePrior) /
 WAD) + exchangeRatePrior;
 // Update update timestamp, exchange rate, and total outstanding
 // borrows.
 marketData.lastTimestampUpdated = uint40(
 cachedData.lastTimestampUpdated +
 (interestCompounds * cachedData.compoundRate)
);
 marketData.exchangeRate = uint216(exchangeRateNew);
 totalBorrows = totalBorrowsNew;
 // ...
 uint256 newReserves = ((interestFactor * debtAccumulated) / WAD);
 if (newReserves > 0) {
 totalReserves = newReserves + reservesPrior;

 // Update gauge pool values for new reserves.
 _gaugePool().deposit(
 address(this),
 centralRegistry.daoAddress(),
 newReserves
);
 }
}
```

As time goes by, `totalBorrowsNew` and `totalReserves` will be greater due to accumulated debt.

So if `totalBorrowsNew` is nearly or equal to `marketUnderlyingHeld()` at first in low liquidity market, the utilization ratio can be manipulated above 100%.

The utilization rate is basically `assets_borrowed / assets_loaned`, the higher the utilization ratio, the higher the interest rate, such that borrowers are discouraged from borrowing from DToken, it shouldn't exceed 100% in any markets.

**Proof of concept:**

```

function testUtilizationRateInLowLiquidity() public {
 uint256 _BASE_UNDERLYING_RESERVE = 42069;
 uint256 initialUsdcReserves = 1000e6;
 _setCbalRETHCollateralCaps(100_000e18);

 uint256 addUsdcAmount = 1150e6;
 deal(_USDC_ADDRESS, address(dUSDC), _BASE_UNDERLYING_RESERVE + initialUsdcReserves + addUsdcAmount);

 assertEq(dUSDC.interestRateModel().compoundRate(), 600);
 assertEq(dUSDC.utilizationRate(), 0);

 // 1. user post collateral and borrow 1150 usdc
 marketManager.postCollateral(address(this), address(cBALRETH), 1e18 - 1);
 dUSDC.borrow(addUsdcAmount);
 assertLt(dUSDC.utilizationRate(), 1 ether);
 assertEq(dUSDC.totalBorrows(), addUsdcAmount);
 assertEq(dUSDC.totalSupply(), _BASE_UNDERLYING_RESERVE);

 // 2. accrue interest
 for (uint i; i < 300; ++i) {
 skip(1 days);
 dUSDC accrueInterest();
 }

 // 3. dUSDC.utilizationRate will exceed 100%
 assertGt(dUSDC.totalBorrows(), addUsdcAmount);
 assertGt(dUSDC.utilizationRate(), 1 ether);
 console2.log("Final utilizationRate", dUSDC.utilizationRate());
}

```

Result:

```

[PASS] testUtilizationRateInLowLiquidity() (gas: 9275139)
Logs:
 final utilizationRate 1009843285420908248

```

Insert the case into tests/market/collateral/DToken/functions/Borrow.t.sol file.

**Recommendation:** Limit DToken utilization ratio so that it can't exceed 100%, many lending protocols have such a limitation, you can see that [here](#).

### 3.2.46 DToken debt can't be repaid/liquidated all due to overflow

Submitted by [pks271](#)

**Severity:** Medium Risk

**Context:** DToken.sol#L1244-L1255, DToken.sol#L1277-L1304, DToken.sol#L1365-L1379, DToken.sol#L411-L430

**Description:** When borrowers [borrow](#) from DToken, the borrow position principal will increase, the code `_debtOf[account].principal = debtBalanceCached(account) + amount` calculates the borrower's debt principal based on `debtBalanceCached(account)` and borrow amount.

Inside `debtBalanceCached()` function:

```

function debtBalanceCached(address account) public view returns (uint256) {
 // Cache borrow data to save gas.
 DebtData storage accountDebt = _debtOf[account];

 // If theres no principal owed, can return immediately.
 if (accountDebt.principal == 0) {
 return 0;
 }

 // Calculate debt balance using the interest index:
 // debtBalanceCached calculation:
 // ((Account's principal * DToken's exchange rate) /
 // Account's exchange rate).
 return
 (accountDebt.principal * marketData.exchangeRate) /
 accountDebt.accountExchangeRate;
}

```



As we can see, first borrowers's debt principal will be `_debtOf[account].principal = totalBorrows = amount`. After some time goes by, when repaying or `_liquidating` all the borrower's full debt, the calculation steps are:

```
function _repay(
 address payer,
 address account,
 uint256 amount
) internal returns (uint256) {
 // Validate that the payer is allowed to repay the loan.
 marketManager.canRepay(address(this), account);

 // Cache how much the account has to save gas.
 uint256 accountDebt = debtBalanceCached(account);

 // Validate repayment amount is not excessive.
 if (amount > accountDebt) {
 revert DToken__ExcessiveValue();
 }

 // If amount == 0, repay max; amount = accountDebt.
 amount = amount == 0 ? accountDebt : amount;

 SafeTransferLib.safeTransferFrom(
 underlying,
 payer,
 address(this),
 amount
);

 // We calculate the new account and total borrow balances,
 // we check that amount is <= accountDebt so we can skip
 // underflow check here.
 unchecked {
 _debtOf[account].principal = accountDebt - amount;
 }
 _debtOf[account].accountExchangeRate = marketData.exchangeRate;
 totalBorrows -= amount;
 // ...
}
```

we can see if the borrowers call `repay(0)` function to repay all debt, the debt is equal to `debtBalanceCached(account)` equal to  $(\text{accountDebt.principal} * \text{marketData.exchangeRate}) / \text{accountDebt.accountExchangeRate}$ , the `marketData.exchangeRate` and `totalBorrows` will update when call `DToken#accrueInterest` every time, the function be called by `DToken#repay/liquidate` functions.

```
function accrueInterest() public {
 // ...
 // Calculate the interest compound cycles to update,
 // in `interestCompounds`. Rounds down natively.
 // @audit - here `interestCompounds` can round down.
 uint256 interestCompounds = (block.timestamp - // <<
 cachedData.lastTimestampUpdated) / cachedData.compoundRate;
 // Calculate the interest and debt accumulated.
 uint256 interestAccumulated = borrowRate * interestCompounds;
 // @audit - here `debtAccumulated` can round down again.
 uint256 debtAccumulated = (interestAccumulated * borrowsPrior) / WAD; // <<
 // Calculate new borrows, and the new exchange rate, based on
 // accumulation values above.
 uint256 totalBorrowsNew = debtAccumulated + borrowsPrior;
 // @audit - here `exchangeRateNew` only round down once.
 uint256 exchangeRateNew = ((interestAccumulated * exchangeRatePrior) / // <<
 WAD) + exchangeRatePrior;
 // Update update timestamp, exchange rate, and total outstanding
 // borrows.
 marketData.lastTimestampUpdated = uint40(
 cachedData.lastTimestampUpdated +
 (interestCompounds * cachedData.compoundRate)
);
 marketData.exchangeRate = uint216(exchangeRateNew);
 totalBorrows = totalBorrowsNew;
 // ...
}
```

Because of the rounding issue here, the `totalBorrows` may be slightly less than `debtBalance-Cached(account)` after first borrower borrows from DToken some time, accumulated debt will increase as time goes by, such that the borrower can't repay all his debt or can't be liquidated by others if no other borrowers borrow from DToken during this period time because `totalBorrows -= amount` will overflow.

This causes two issues here:

- First borrower can't repay/liquidated all his debt if there is no other borrowers during this period time.
- The DToken's debt can't be repaid/liquidated all, the last borrower's repay/liquidation action can't be done if accumulate some debt there.

### Proof of concept:

- Issue 1: first borrower can't repay/liquidated all his debt:

```
function testFirstBorrowerRevertedAfterAccrueInterests() public {
 uint256 _BASE_UNDERLYING_RESERVE = 42069;
 uint256 initialUsdcReserves = 1000e6;
 _setCbalRETHCollateralCaps(100_000e18);

 uint256 addUsdcAmount = 1150e6;
 deal(_USDC_ADDRESS, address(dUSDC), _BASE_UNDERLYING_RESERVE + initialUsdcReserves + addUsdcAmount);

 // 1. post collateral and borrow 1e18 - 1
 marketManager.postCollateral(address(this), address(cBALRETH), 1e18 - 1);
 dUSDC.borrow(addUsdcAmount);

 // can be called by malicious users
 for (uint i; i < 2; ++i) {
 skip(1 days);
 dUSDC.accrueInterest();
 }

 // 2. repay all debt would revert because overflow
 // give address(this) enough usdc to repay their debt because accumulated interest
 deal(_USDC_ADDRESS, address(this), 10000e6);
 IERC20(_USDC_ADDRESS).approve(address(dUSDC), type(uint256).max);
 // vm.expectRevert();
 dUSDC.repay(0);
}
```

Result:

```
[FAIL. Reason: panic: arithmetic underflow or overflow (0x11)]
↪ testFirstBorrowerRevertedAfterAccrueInterests() (gas: 1493803)
```

- Issue 2 - the last borrower can't repay/liquidated all his debt:

```
function testBorrowersCannotRepayAllDebts() public {
 uint256 _BASE_UNDERLYING_RESERVE = 42069;
 uint256 initialUsdcReserves = 1000e6;
 _setCbalRETHCollateralCaps(100_000e18);

 uint256 addUsdcAmount = 1500e6;
 deal(_USDC_ADDRESS, address(dUSDC), _BASE_UNDERLYING_RESERVE + initialUsdcReserves + addUsdcAmount);

 address user101 = address(101);
 address user102 = address(102);
 address user103 = address(103);
 address[] memory users = new address[](3);
 users[0] = user101;
 users[1] = user102;
 users[2] = user103;

 // 1. users post collateral and borrow 100 usdc
 for (uint i; i < 3; ++i) {
 address user = users[i];
 deal(address(cBALRETH), user, 1e18);
 vm.startPrank(user);
 marketManager.postCollateral(user, address(cBALRETH), 1e18 - 1);
 dUSDC.borrow(100e6);
 vm.stopPrank();
 }
}
```

```

 }

 // 2. repay user101 and user102 all debt after two days
 skip(2 days);
 for (uint i; i < 2; ++i) {
 address user = users[i];
 vm.startPrank(user);
 // give users enough usdc to repay their debt because accumulated interest
 deal(_USDC_ADDRESS, user, 1000e6);
 IERC20(_USDC_ADDRESS).approve(address(dUSDC), type(uint256).max);
 dUSDC.repay(0);
 vm.stopPrank();
 }

 // can be called by malicious users
 for (uint i; i < 2; ++i) {
 skip(1 days);
 dUSDC.accrueInterest();
 }

 deal(_USDC_ADDRESS, users[2], 1000e6);
 vm.startPrank(users[2]);
 IERC20(_USDC_ADDRESS).approve(address(dUSDC), type(uint256).max);
 // 3. user103 repay all his debt would revert because overflow
 // vm.expectRevert();
 dUSDC.repay(0);
 vm.stopPrank();
}

```

Result:

```

[FAIL. Reason: panic: arithmetic underflow or overflow (0x11)] testBorrowersCannotRepayAllDebts() (gas:
↳ 3486652)

```

Insert the two cases into tests/market/collateral/DToken/functions/Borrow.t.sol file.

### Recommendation:

1. Change totalBorrows -= accountDebt to:

```

if (totalBorrows < accountDebt) {
 totalBorrows = 0;
}

```

2. Fix the rounding issue in the DToken#accrueInterest function.

### 3.2.47 DTokens cannot be minted until gaugePool has started

Submitted by *KupiaSec*

**Severity:** Medium Risk

**Context:** GaugePool.sol#L334-L391, GaugePool.sol#L603-L673, GaugeController.sol#L145-L150, GaugeController.sol#L186-L190

**Description:** If dTokens are minted before starting the gauge pool, then `deposit` function of `GaugePool` is called.

```

```solidity
function deposit(
    address token,
    address user,
    uint256 amount
) external nonReentrant {
    // [...]
    updatePool(token); // <<

    _calcPending(user, token);

    balanceOf[token][user] += amount;
    totalSupply[token] += amount;

    // If the gauge has not started yet no need to check whether
    // first deposit has been set.
    if (block.timestamp > startTime) { // <<
        // If first deposit has not occurred we will need to send
        // excess rewards to the DAO.
        if (firstDeposit == 0) { // <<
            // If first deposit, the new rewards from gauge start to this
            // point will be unallocated rewards.
            firstDeposit = block.timestamp;
            updatePool(token); // <<
            // [...]
        }
    }
    // [...]
}
}
```

```

Since `startTime` is 0 and it is the first deposit, `updatePool` function is called twice.

The first call is not reverted because `totalSupply[token]` is 0, but `totalSupply[token]` becomes greater than 0 in the second call. So, in the second call of `updatePool`, `epochOfTimestamp` is called and `epochOfTimestamp` function is reverted when `gaugePool` is not started.

```

```solidity
function updatePool(address token) public override {
    // If rewards have not started yet, there is nothing to update.
    if (block.timestamp <= startTime) {
        return;
    }

    uint256 _lastRewardTimestamp = poolLastRewardTimestamp[token];
    // If nobody has updated reward timestamp, time to set it up to startTime.
    if (_lastRewardTimestamp == 0) {
        _lastRewardTimestamp = startTime;
    }

    // Make sure time has passed since the last update.
    if (block.timestamp <= _lastRewardTimestamp) {
        return;
    }

    // Is there are no deposits, there is nothing to update.
    uint256 totalDeposited = totalSupply[token]; // <<
    if (totalDeposited == 0) { // <<
        return;
    }

    // Cache rewardTokens length.
    uint256 rewardTokensLength = rewardTokens.length;
    for (uint256 i; i < rewardTokensLength; ) {
        uint256 lastRewardTimestamp = _lastRewardTimestamp;

        // Query rewardToken then increment i.
        address rewardToken = rewardTokens[i++];
        uint256 accRewardPerShare = poolAccRewardPerShare[token][
            rewardToken
        ];
        uint256 lastEpoch = epochOfTimestamp(lastRewardTimestamp); // <<
        uint256 currentEpoch = currentEpoch();
        // [...]
    }
}
```

```

```

}

function epochOfTimestamp(
 uint256 timestamp
) public view returns (uint256) {
 _checkGaugeHasStarted(); // <<
 return (timestamp - startTime) / EPOCH_WINDOW;
}

function _checkGaugeHasStarted() internal view {
 if (startTime == 0) { // <<
 revert GaugeErrors.NotStarted();
 }
}
...

```

DTokens cannot be minted until gaugePool has started. The likelihood is HIGH because DTokens are intended to be minted before starting the gauge pool. However, attempting to mint DToken before starting the gauge pool will always result in a revert.

#### Proof of concept:

1. The gaugePool has not started yet.
2. A user deposit some dTokens and it will be reverted.

**Recommendation:** Apply the following modification:

```

function updatePool(address token) public override {
 // If rewards have not started yet, there is nothing to update.
 if (startTime == 0 || block.timestamp <= startTime) {
 return;
 }
 // [...]
}

```

### 3.2.48 epochReported is never set in BlastNativeYieldManager

Submitted by *0xEVom*, also found by *krikolkk*, *Zach Obront* and *kaka*

**Severity:** Medium Risk

**Context:** [BlastNativeYieldManager.sol#L190-L194](#)

**Description:** In the BlastNativeYieldManager contract, the epochReported mapping is used to track whether yield has already been reported and claimed for a given token and epoch when calling the claimYieldForGauge() function.

However, epochReported is never actually set to true after claiming yield. This means that the check:

```

if (epochReported[msg.sender][nextEpoch]) {
 _revert(_UNAUTHORIZED_SELECTOR);
}

```

will always pass, since epochReported will remain false.

This has several consequences:

1. If claimYieldForGauge() is called multiple times in the same epoch for a given token, the rewards will be transferred back out of GaugePool in subsequent calls instead of accumulating. This is partly due to the fact that GaugePool does not aggregate rewards for calls in the same epoch, even though it will be called several times in the same epoch by the different tokens, which is reported in a separate finding.
2. It will lead to pending yield for a given epoch being claimable more than once since the pending yield is never cleared. This is possible for all yield (not just that in a given epoch) and is also reported in a separate submission.
3. Tokens that are depending on the epochReported check to ensure yield is not claimed multiple times or misaccounted for will not function as intended, since the check will never fail.

**Recommendation:** epochReported[msg.sender][nextEpoch] should be set to true at the end of claimYieldForGauge().

### 3.2.49 Users voting power is zero in VeCVE, causing offchain voting failure

Submitted by [Josephdara](#), also found by [0xTiwa](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** The VeCVE contract is a system for managing the larger Curvance Voting Escrow System within Curvance Protocol. It is quite similar to other voting contracts except for few differences outlined in the codebase. The relevant one is stated below:

```
/// - Offchain Voting:
/// By moving from an onchain voting mechanism we minimize expenses
/// to users and can aggregate votes across all chains at the same
/// time, via calling getVotes() on each chain for a user.
```

The aggregation of user votes is done by getting the votes for each user through getVotes(), however there is a critical oversight from vote boost, which makes getVotes() return zero for users with CONTINUOUS\_LOCK.

The issue is with the voteBoostMultiplier gotten from the central registry. In the centralRegistry.setVoteBoostMultiplier(), it is seen below that the voteBoostMultiplier can be set to ZERO if the boosting feature is to be turned off.

```
function setVoteBoostMultiplier(uint256 value) external {
 _checkElevatedPermissions();

 // Voting power boost cannot be less than 1,
 // unless its being turned off.
 if (value < DENOMINATOR && value != 0) {
 _revert(_PARAMETERS_MISCONFIGURED_SELECTOR);
 }

 voteBoostMultiplier = value;

 emit MultiplierSet("Vote Boost", value);
}
```

However, without regard for its value, all voting power gotten for any user is directly multiplied. Therefore whenever the feature is turned off to disable voting boosts, all offchain votes will be null for users with the CONTINUOUS\_LOCK.

**Proof of concept:** Below is the code snippet for calculating all votes for users, as we can see the currentLockBoost is directly multiplied by the user's lock.amount, resulting in 0.

```
function getVotes(address user) external view returns (uint256) {
 uint256 numLocks = userLocks[user].length;

 if (numLocks == 0) {
 return 0;
 }

 uint256 currentLockBoost = centralRegistry.voteBoostMultiplier();
 uint256 votes;

 for (uint256 i; i < numLocks;) {
 // Based on CVE maximum supply this cannot overflow.
 unchecked {
 votes += getVotesForSingleLockForTime(
 user,
 i++,
 block.timestamp,
 currentLockBoost
);
 }
 }

 return votes;
}
```

```

}

function getVotesForSingleLockForTime(
 address user,
 uint256 lockIndex,
 uint256 time,
 uint256 currentLockBoost
) public view returns (uint256) {
 Lock storage lock = userLocks[user][lockIndex];

 if (lock.unlockTime < time) {
 return 0;
 }

 if (lock.unlockTime == CONTINUOUS_LOCK_VALUE) {
 unchecked {
 return ((lock.amount * currentLockBoost) / 10000);
 }
 }

 // Equal to epochsLeft = (lock.unlockTime - time) / EPOCH_DURATION
 // (lock.amount * epochsLeft) / LOCK_DURATION_EPOCHS.
 return
 (lock.amount * ((lock.unlockTime - time) / EPOCH_DURATION)) /
 LOCK_DURATION_EPOCHS;
}

```

**Recommendation:** Change the currentLockBoost to 10000 if it is equal to 0 on the central registry

### 3.2.50 supportsInterface fails to return all supported interfaces in BlastCentralRegistry

Submitted by [Sujith Somraaj](#), also found by [krikolkk](#) and [0xEVom](#)

**Severity:** Medium Risk

**Context:** [BlastCentralRegistry.sol#L148](#)

**Description:** The function supportsInterface in BlastCentralRegistry overrides all base implementations of the function in CentralRegistry and ERC165 contracts.

However it does not return all the supported interfaces, leading to problems with external integrations as it does not return valid supported interfaces in the blast implementation.

```

function supportsInterface(
 bytes4 interfaceId
) public pure override returns (bool) {
 return
 interfaceId == type(IBlastCentralRegistry).interfaceId ||
 interfaceId == type(ICentralRegistry).interfaceId;
}

```

**Recommendation:** Return all supported interface IDs.

```

function supportsInterface(
 bytes4 interfaceId
) public view virtual override returns (bool) {
 return
 interfaceId == type(IBlastCentralRegistry).interfaceId ||
+ super.supportsInterface(interfaceId);
}

```

### 3.2.51 Rewards will be lost when calling setRewardPerSec() more than once for a given epoch

Submitted by [0xEVom](#), also found by [Zach Obront](#)

**Severity:** Medium Risk

**Context:** [BlastNativeYieldManager.sol#L148](#), [BlastNativeYieldManager.sol#L192-L194](#), [BlastNativeYieldManager.sol#L210-L215](#), [GaugePool.sol#L227-L240](#)

**Description:** In the GaugePool contract, the setRewardPerSec() function allows the reward rate for a token to be updated. If the new reward rate is lower than the previous rate in the same epoch, the contract will

transfer the difference in total rewards for the epoch from itself to the caller, while if it is higher it will only pull the difference.

However, there is no logic in the `BlastNativeYieldManager` contract to aggregate multiple reward rate changes in the same epoch. So if `setRewardPerSec()` is called multiple times in the same epoch with the same `yieldDestination`, the total rewards will be less than expected.

This can happen in practice in at least two scenarios:

- `claimYieldForGauge()` in `BlastNativeYieldManager` will be called by many different tokens, each triggering a call to `setRewardPerSec()`. If multiple tokens route their yield to the same token as mapped in `cTokenToDTokenYieldRouted`, the second reward will overwrite the first one.
- The `epochReported` mapping in `BlastNativeYieldManager` is never updated, so a token can claim yield multiple times in an epoch (as reported in a separate finding).

As a result, a portion of the rewards can be lost, leading to less rewards available for distribution to users than expected.

**Recommendation:** `setRewardPerSec()` should always transfer the amount derived from `newRewardPerSec` from the caller, rather than treating the new call as an update. If this functionality is needed, it should be implemented as a separate function that can only be called by the DAO address.

Alternatively, `epochReported` should track reported yield per `yieldDestination`, and not per `msg.sender`.

### 3.2.52 CVX rewards are not accounted for

Submitted by [99Crits](#)

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** Convex mints CVX when rewards are claimed, but the address is not part of the extra reward tokens that are queried. This will cause harvesting to not deduct the protocol fees for CVX.

This is what the process looks like on Convex' side:

```
function getReward(address _account, bool _claimExtras) public updateReward(_account) returns(bool){
 uint256 reward = earned(_account);
 if (reward > 0) {
 rewards[_account] = 0;
 rewardToken.safeTransfer(_account, reward); // CRV
 IDeposit(operator).rewardClaimed(pid, _account, reward); // this will mint CVX
 emit RewardPaid(_account, reward);
 }

 //also get rewards from linked rewards
 if(_claimExtras){
 for(uint i=0; i < extraRewards.length; i++){
 IRewards(extraRewards[i]).getReward(_account);
 }
 }
 return true;
}

//Booster.rewardClaimed (aka "operator")
function rewardClaimed(uint256 _pid, address _address, uint256 _amount) external returns(bool){
 address rewardContract = poolInfo[_pid].crvRewards;
 require(msg.sender == rewardContract || msg.sender == lockRewards, "!auth");

 //mint reward tokens
 ITokenMinter(minter).mint(_address, _amount); //CVX mint

 return true;
}
```

**Proof of concept:** (just logging reward tokens and convex balances before and after):

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.17;

import { SwapperLib } from "contracts/libraries/SwapperLib.sol";
```



```

import { ICentralRegistry } from "contracts/interfaces/ICentralRegistry.sol";
import { Convex2PoolCToken, IERC20 } from "contracts/market/collateral/Convex2PoolCToken.sol";
import "forge-std/console2.sol";

import "tests/market/TestBaseMarket.sol";

contract TestConvex2PoolCToken is TestBaseMarket {
 address internal constant _UNISWAP_V2_ROUTER =
 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;

 IERC20 public constant CVX =
 IERC20(0x4e3FBD56CD56c3e72c1403e103b45Db9da5B9D2B);
 IERC20 public constant CRV =
 IERC20(0xD533a949740bb3306d119CC777fa900bA034cd52);
 IERC20 public CONVEX_STETH_ETH_POOL =
 IERC20(0x21E27a5E5513D6e65C4f830167390997aA84843a);
 uint256 public CONVEX_STETH_ETH_POOL_ID = 177;
 address public CONVEX_STETH_ETH_REWARD =
 0x6B27D7BC63F1999D14fF9bA900069ee516669ee8;
 address public CONVEX_BOOSTER = 0xF403C135812408BFbE8713b5A23a04b3D48AAE31;

 Convex2PoolCToken public cSTETH;

 /*
 LP token address 0x21E27a5E5513D6e65C4f830167390997aA84843a
 Deposit contract address 0xF403C135812408BFbE8713b5A23a04b3D48AAE31
 Rewards contract address 0x6B27D7BC63F1999D14fF9bA900069ee516669ee8
 Convex pool id 177
 Convex pool url https://www.convexfinance.com/stake/ethereum/177
 */

 receive() external payable {}

 fallback() external payable {}

 function setUp() public override {
 super.setUp();

 centralRegistry.addHarvester(address(this));
 centralRegistry.setFeeAccumulator(address(this));

 // start epoch
 gaugePool.start(address(marketManager));
 vm.warp(gaugePool.startTime());
 vm.roll(block.number + 1000);

 cSTETH = new Convex2PoolCToken(
 ICentralRegistry(address(centralRegistry)),
 CONVEX_STETH_ETH_POOL,
 address(marketManager),
 CONVEX_STETH_ETH_POOL_ID,
 CONVEX_STETH_ETH_REWARD,
 CONVEX_BOOSTER
);

 address owner = address(this);
 deal(address(CONVEX_STETH_ETH_POOL), address(owner), 1 ether);
 CONVEX_STETH_ETH_POOL.approve(address(cSTETH), 1 ether);
 marketManager.listToken(address(cSTETH));
 }

 function testConvexStethEthPool() public {
 uint256 assets = 100e18;
 deal(address(CONVEX_STETH_ETH_POOL), address(user1), assets);

 vm.prank(address(user1));
 CONVEX_STETH_ETH_POOL.approve(address(cSTETH), assets);

 vm.prank(address(user1));
 cSTETH.deposit(assets, user1);

 console2.log("reward token1:", cSTETH.rewardTokens()[0]);
 console2.log("reward token2:", cSTETH.rewardTokens()[1]);
 console2.log("convex address:", address(CVX));

 console2.log("cvx balance before warp", CVX.balanceOf(address(cSTETH)));
 }
}

```

```

 assertEq(
 cSTETH.totalAssets(),
 assets + 42069,
 "Total Assets should equal user deposit."
);

 // Advance time to earn CRV and CVX rewards
 vm.warp(block.timestamp + 3 days);

 // Mint some extra rewards for Vault.
 deal(address(CRV), address(cSTETH), 100e18);
 deal(address(CVX), address(cSTETH), 100e18);
 deal(address(cSTETH), 1 ether);

 cSTETH.harvest(abi.encode(new SwapperLib.Swap[] (0)));

 assertEq(
 cSTETH.totalAssets(),
 assets + 42069,
 "Total Assets should equal user deposit."
);

 vm.warp(block.timestamp + 8 days);

 // Mint some extra rewards for Vault.
 deal(address(CRV), address(cSTETH), 100e18);
 deal(address(CVX), address(cSTETH), 100e18);
 deal(address(cSTETH), 1 ether);
 cSTETH.harvest(abi.encode(new SwapperLib.Swap[] (0)));

 console2.log("cvx balance after warp and harvest", CVX.balanceOf(address(cSTETH)));

 vm.warp(block.timestamp + 7 days);

 uint256 totalAssets = cSTETH.totalAssets();

 assertGt(
 totalAssets,
 assets + 42069,
 "Total Assets should greater than original deposit."
);

 vm.startPrank(address(user1));
 cSTETH.withdraw(cSTETH.balanceOf(user1), user1, user1);
 vm.stopPrank();
}
}

```

**Recommendation:** Like AuraCToken does for AURA token, the ConvexCTokens (2Pool and 3Pool) needs to push CVX additionally to CRV.

```

address private constant _CVX = 0x4e3FBD56CD56c3e72c1403e103b45Db9da5B9D2B;
// ...
strategyData.rewardTokens.push() = _CVX;

```